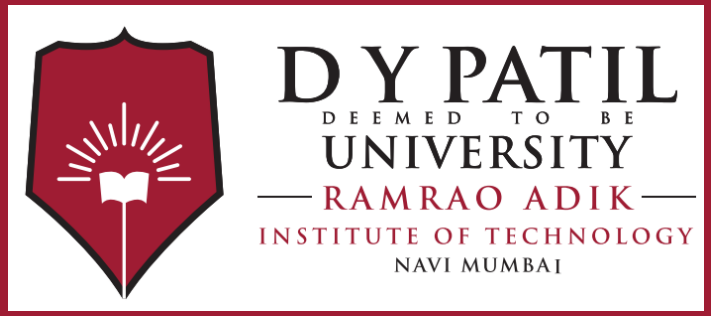


Subject Name: OOP

Unit No:1

Unit Name: Introduction to OOP

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 1 – Basic Concepts of Object-Oriented Programming	7
Lecture 2 – Need of OOP, Object Oriented Programming Paradigm	21
Lecture 3 – Benefits of OOP, Introduction to OOP languages : C++ and Java	34

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Course Schema

Teaching Scheme &Evaluation Scheme:

Course Code	Course Name	Teaching Scheme (Contact Hours 39)			Credits Assigned	
		Theory	Practical		Theory	Total
2413CEC3T1	Object Oriented Programming	03	--		03	03

Course Code	Course Name	Theory Evaluation Scheme (Marks)								
		Internal Assessment (50)						End Sem. Exam	End Sem. Exam Hrs.	Total
		Best 2 (20)			(30)					
		CA1	CA2	CA3	Assignment / Tutorial	Quiz / Seminar	Open Book Test / Surprise Test / Capstone Project			
2413CEC3T1	Object Oriented Programming	10	10	10	10	10	10	100	03	150

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Course Schema

Teaching Scheme &Evaluation Scheme:

Course Code	Course Name	Teaching Scheme (Contact Hours 26)		Credits Assigned	
		Theory	Practical	Practical	Total
2413CEC3L1	Object Oriented Programming Lab	--	02	01	01

Course Code	Course Name	Practical Evaluation Scheme (Marks)					
		Internal Assessment (30)			End Semester Examination (30)		Total
		Continuous Evaluation	Lab Quiz	Attendance	Practical Performance	Oral	
2413CEC3L1	Object Oriented Programming Lab	20	5	5	20	10	60

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Course Objectives

1. To understand the object-oriented programming basics and its features.
2. Able to use a programming language to resolve problems.
3. To understand and apply Object Oriented Programming (OOP) principles using Java.
4. To study various java programming concept like multithreading, exception handling, packages etc.
5. To explain components of GUI based programming.

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Course Outcomes

Students will be able to :

1. Understand basics of OOP and apply fundamental programming constructs.
2. Understand and illustrate the features of classes and objects.
3. Elaborate the concept of strings, arrays and vectors.
4. Develop a program that implements the concept of inheritance, interfaces and packages.
5. Implement the notion of exception handling and multithreading.
6. Develop GUI based application.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 1
Basic Concepts of Object-Oriented Programming

Introduction to Programming

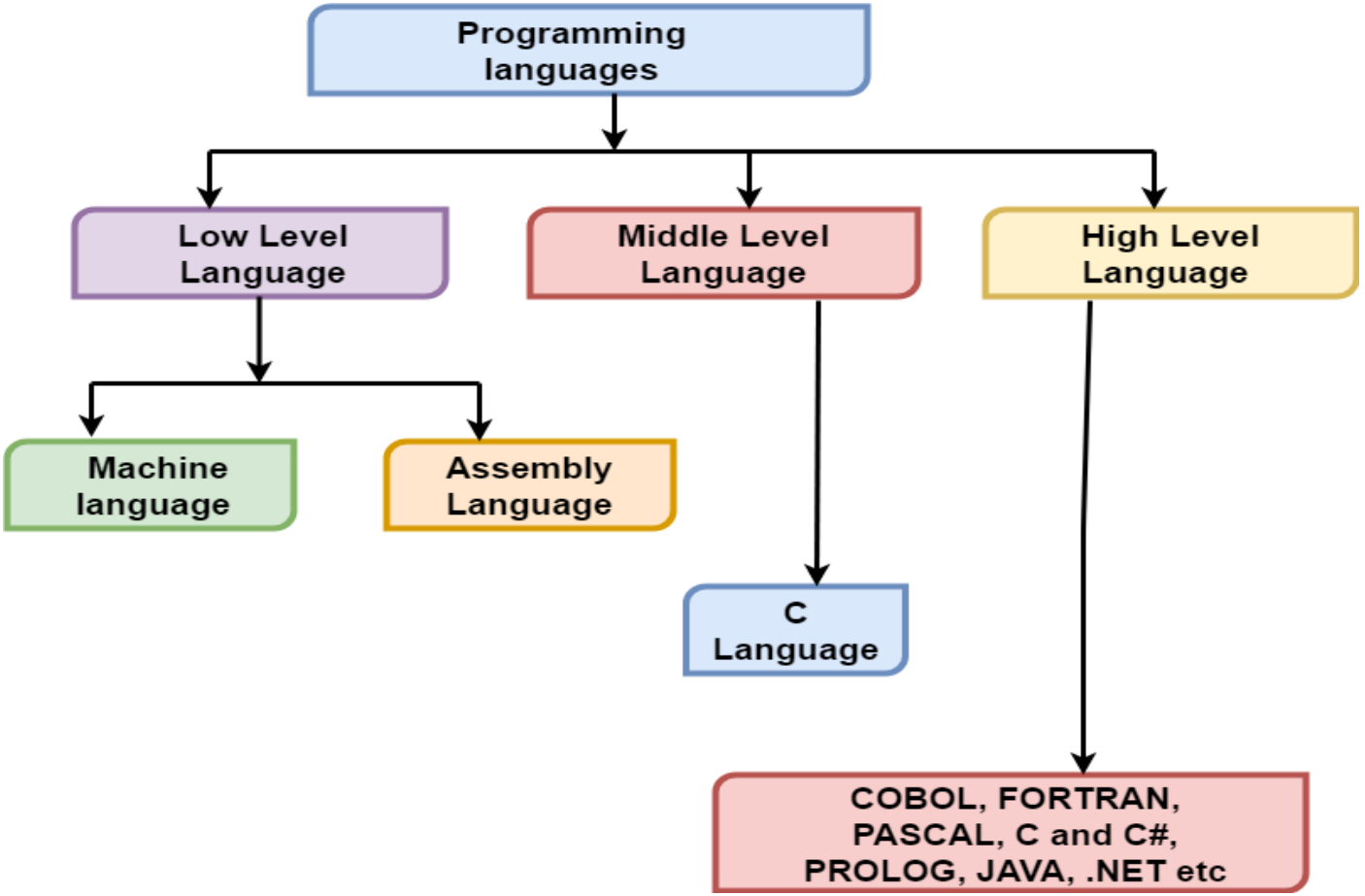
- **Program:**
  - Program (also software or application) is a sequence of instructions written to perform a specified task with a computer.

- **Programmer:**
  - A person who practices this skill of writing program is referred to as a computer programmer or software developer or coder.

- **Programming:**
  - Programming is the iterative process of writing or editing source code. Editing source code involves testing, analyzing, refining and sometimes coordinating with other programmers on a jointly developed program.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

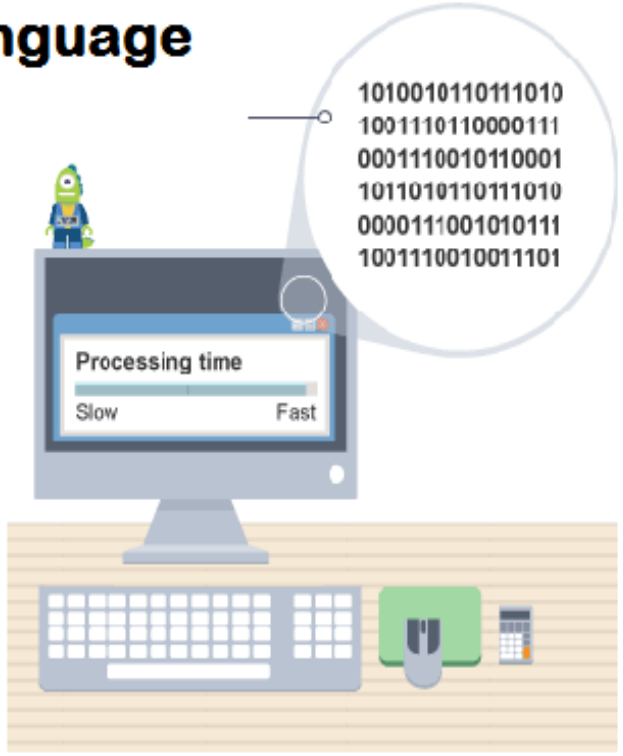
Hierarchy of Programming Languages



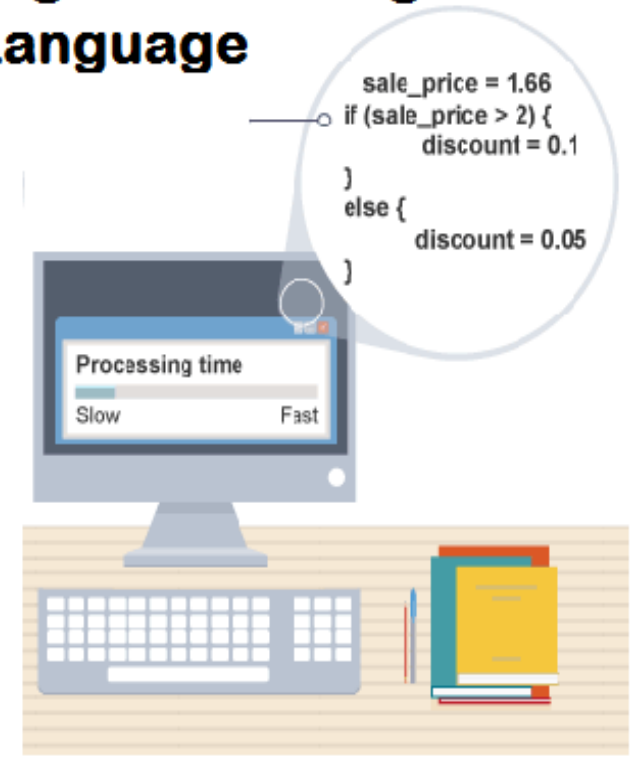
D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Why to Learn High Level Languages

Low Level Programming Language



High Level Programming Language



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Programming Approaches

- Procedure-Oriented: Top-Down Approach
- Object-Oriented: Bottom-Up Approach

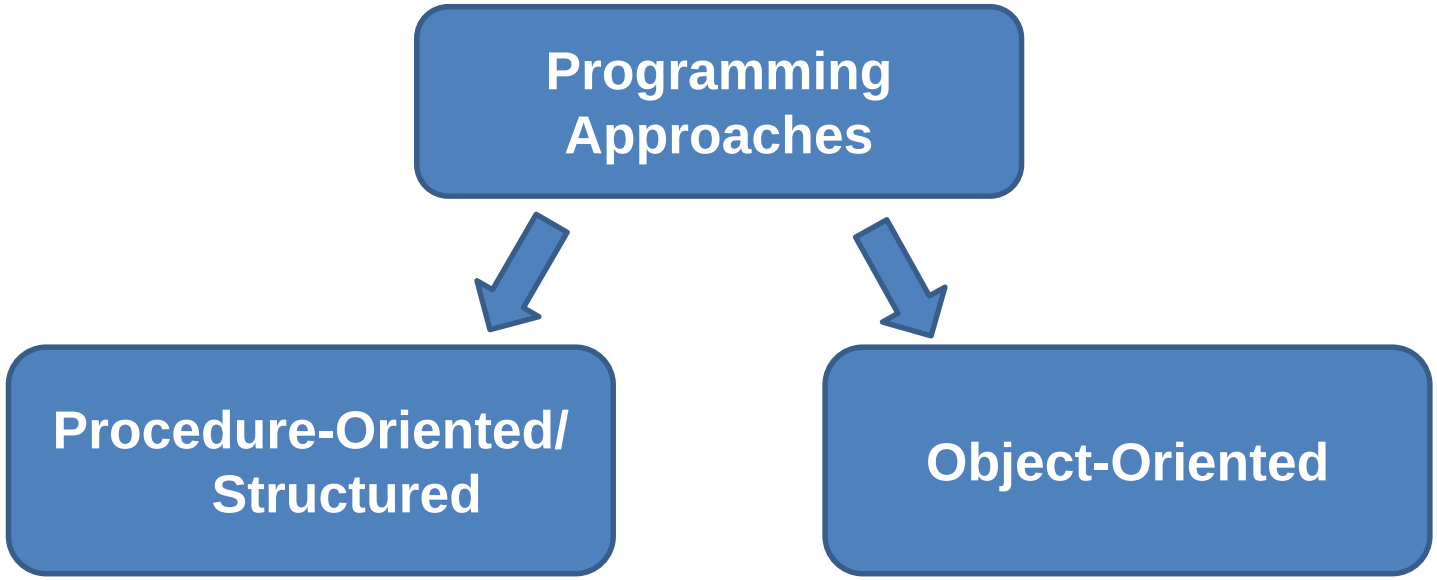


Figure: Programming Approaches

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Procedural Programming

- Traditional programming languages were procedural.
  - E.g. C, Pascal, BASIC, Ada and COBOL
- Programming in procedural languages involves choosing data structures (appropriate ways to store data), designing algorithms, and translating algorithm into code.
- In procedural programming, data and operations on the data are separated.
- This methodology requires sending data to procedure/functions.

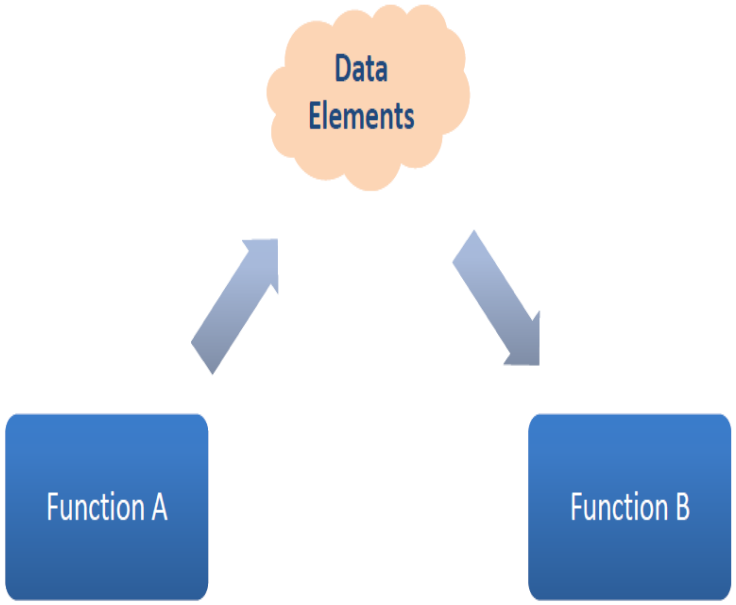
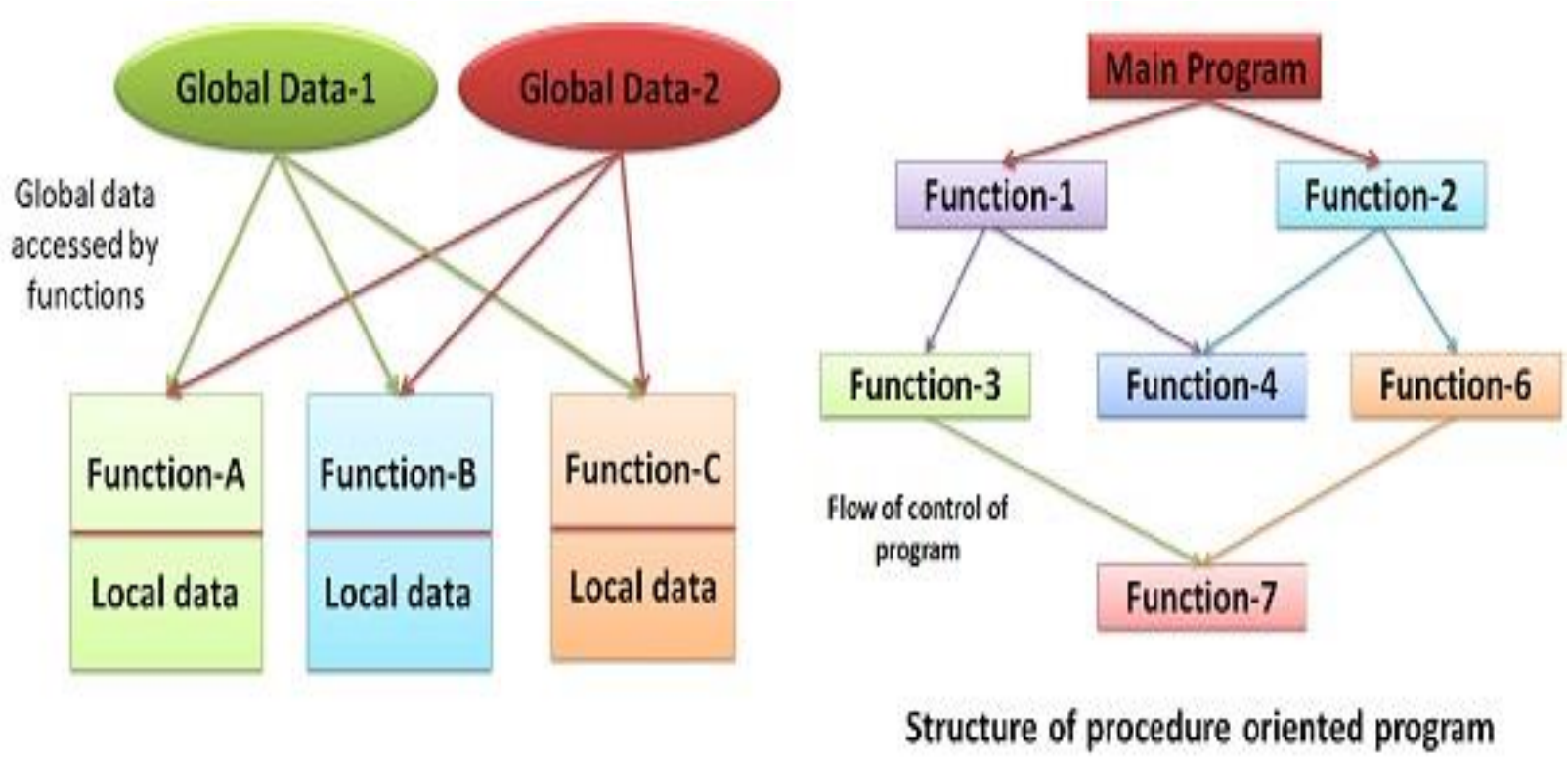


Figure: Procedural Programming

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Structure of Procedure-oriented Programming



Structure of procedure oriented program

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Structured Programming Approach

- The program is made as a single structure i.e. the code will execute the instruction by instruction one after the other.
- It doesn't support the possibility of jumping from one instruction to some other with the help of any statement like GOTO, etc. Therefore, the instructions in this approach will be executed in a serial and structured manner.
- C is called a structured programming language because to solve a large problem, C programming language divides the problem into smaller structural blocks each of which handles a particular responsibility.
- These structural blocks are –
  - Decision making blocks like if-else-elseif, switch-cases,
  - Repetitive blocks like For-loop, While-loop, Do-while loop etc
  - Subroutines/Procedures - functions

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Advantages of Structured Programming

- C structured programming is simple and easy to understand and implement.
- It is well sited for small size implementation. However this is not restricted. A good design can extend it to large size implementation.
- Programmers do not require to know complex design concepts to start a new program.
- Easier to Debug
- These are mainly problem oriented rather than machine based.
- Program written in a higher level language can be translated into many machine languages and therefore can run on any computer for which there exists an appropriate translator.
- It is independent of machine on which it is used i.e. programs developed in high level languages can be run on any computer.

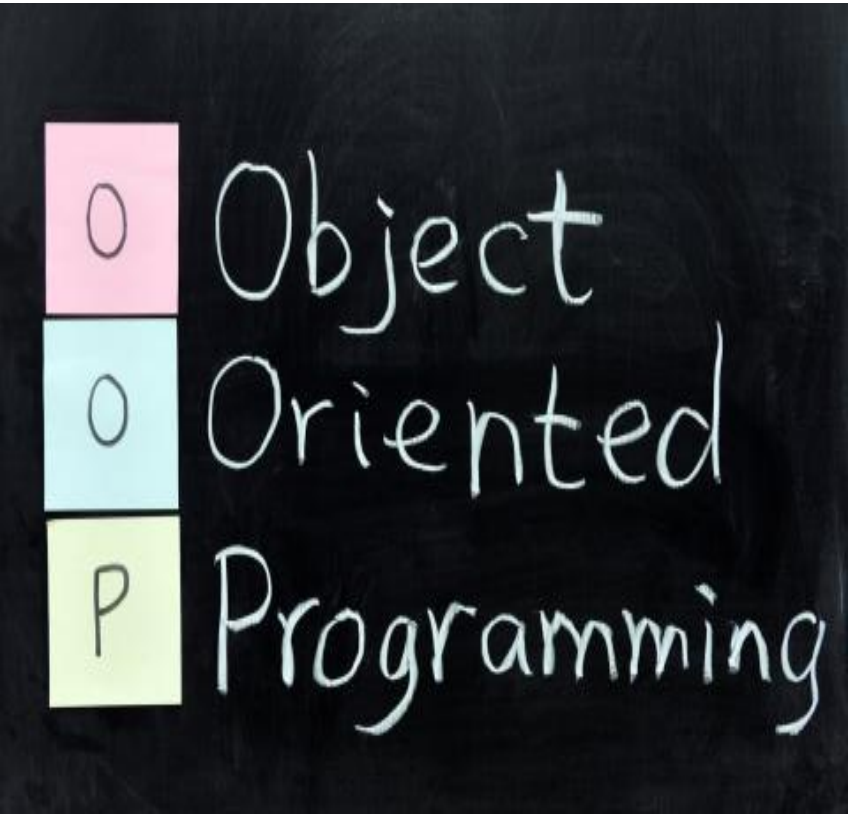
D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Disadvantages of Structured Programming

- A high level language has to be translated into the machine language by translator and thus a price in computer time is paid.
- Since it is Machine-Independent, So it takes time to convert into machine code.
- The code generated by a translator might be inefficient compared to an equivalent assembly language program.
- Data type are proceeds in many functions in a structured program. When changes occur in those data types, the corresponding change must be made to every location that acts on those data types within the program. This is really a very time consuming task if the program is very large.
- Data and methods are not bind together in a module.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Object Oriented Approach



- Object oriented approach organizes a program in the form of objects
- Focus on data
- Program is organized around data
- Data and operations are grouped together

Objects = Data+ Methods

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Object-Oriented Programming

- Object-oriented programming is centered on creating **objects rather than procedures/ functions.**
- Objects are a melding of **data and procedures that manipulate that data.**

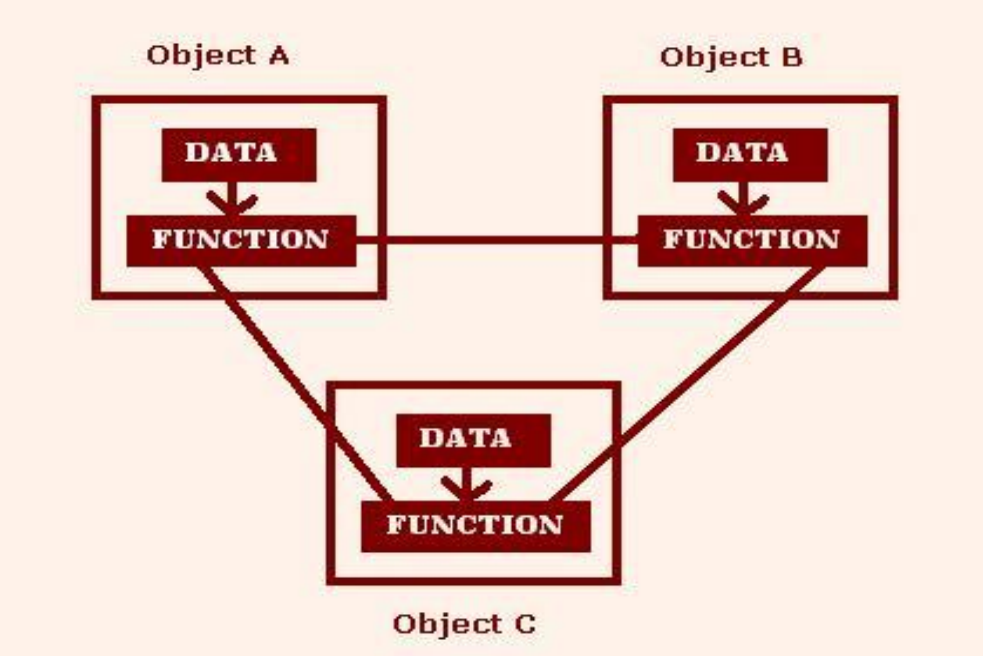
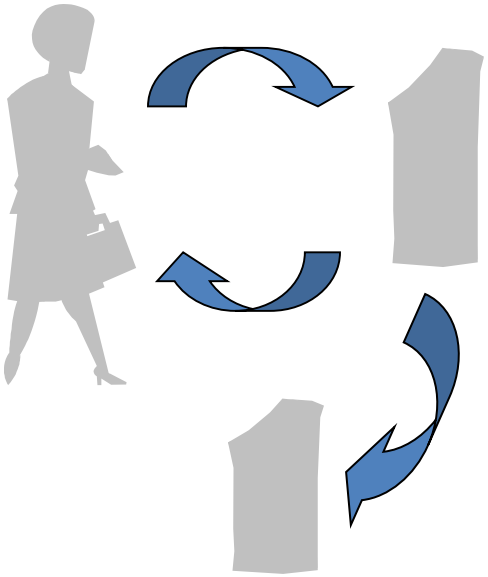


Figure: Object-Oriented Programming

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

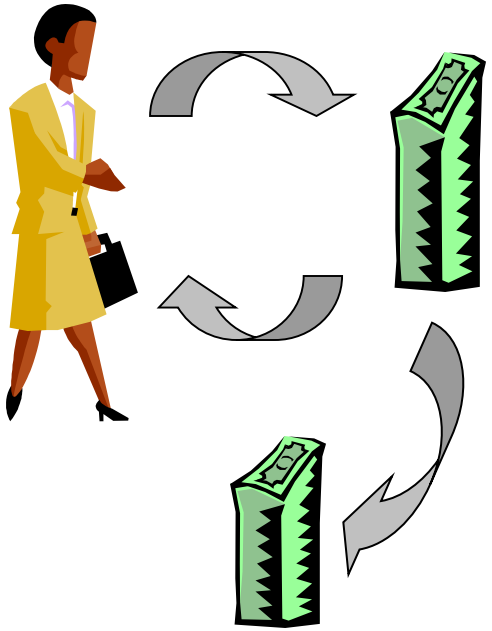
Procedural vs Object oriented programming

- Procedural



Withdraw, deposit, transfer

- Object Oriented



Customer, money, account

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Difference between Structured and Object Oriented Approach

Structured Programming	Object Oriented Programming
• The program is represented as a logical structure.	• The program is written as a collection of objects which communicate with each other.
• The flow of execution of the programming is dependent on the structure of the program.	• The basic entity is object. Each computation is performed using objects only.
• Code is given more importance.	• Data is given more importance.
• Can handle up to moderately complex programs.	• Can handle very complex programs.
• Less data security.	• More data security.
• Less code reusability.	• More reusability.
• Flexibility is less.	• Flexibility is more.
• Abstraction is less.	• Abstraction is more.
• This is top-down approach.	• This is a bottom-up approach.
E.g: C, VB, FORTRAN, Pascal	E.g: C++, JAVA, VB.NET, C#.NET

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Lecture No: 2
Need of OOP, Object Oriented Programming Paradigm

Need of OOP

Hardware	• Language close to hardware
Object Oriented Programming	• Easy to understanding
Software Developers Choice	• Millions of software developers use C++,Java
Application And System Programming.	• C++, Java is the most widely used programming languages
Difference Between Compiler, Linker And Loader	• C++, Java really teaches you the difference between compiler, linker and loader, different data types, storage classes,

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Applications of OOP

Application Software Development	• Operating System: Windows, Mac OSX and Linux. • Browsers : Mozilla Firefox and Chrome • database system: MySQL.
Programming Languages Development	• C#, Java, JavaScript, Perl, UNIX's C Shell, PHP and Python, and Verilog etc.
Computation Programming	• Has a fast speed and computational efficiencies.
Embedded System	• Medical and Engineering Applications like softwares for MRI machines, high-end CAD/CAM systems etc.
Games Development	• Procedural programming for CPU intensive functions and provides greater control over hardware

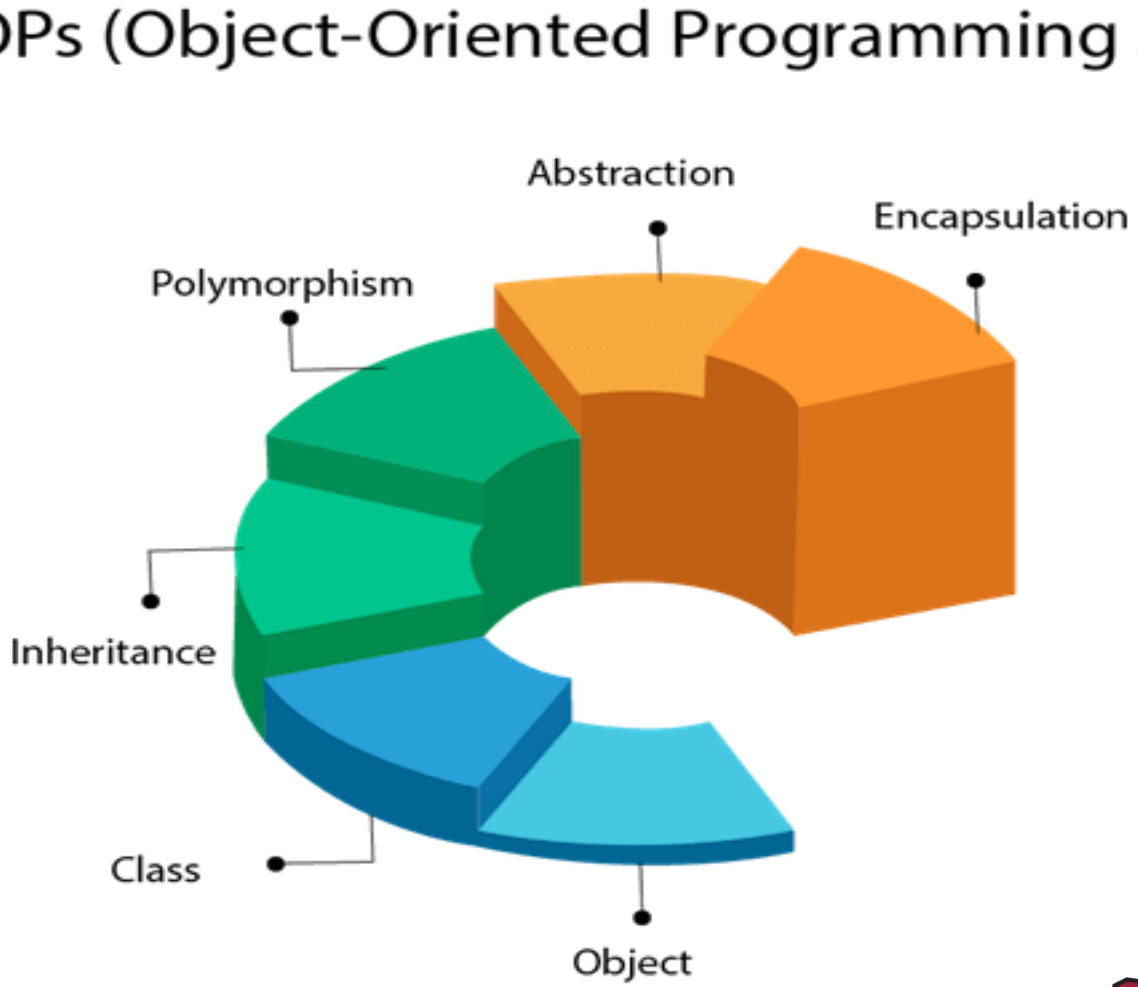
D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Features of OOP

Striking features of OOP	Emphasis data
	Program divided into object
	Data structure
	Function
	Hidden Data
	Object Communication
	New data/function easily add
	Follow bottom of approach

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Object Oriented Paradigm



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Object Oriented Programming

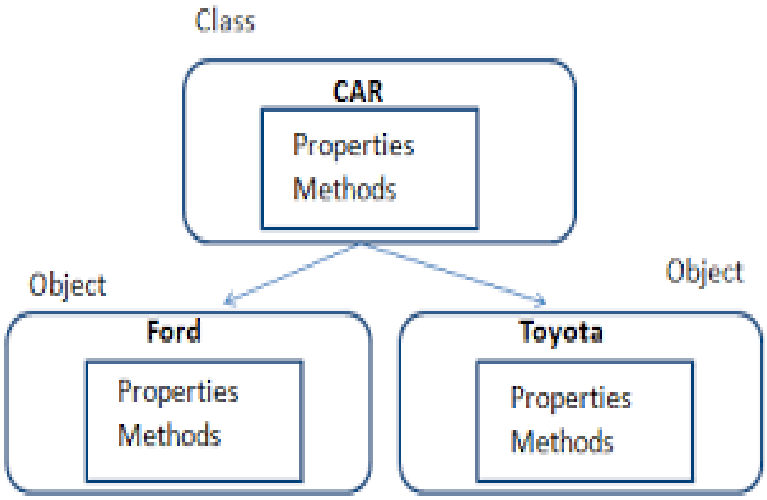
- Classes
- Objects
- Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Dynamic Binding
- Message Passing

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Classes and Objects

➢ Classes

- Basic building blocks OOP and a single entity which has data and operations on data together.
- Classes are user-defined data types and behave like the built-in types of a programming language.



➢ Objects

- Objects are the basic run-time entities.
- The instances of a class which are used in real functionality(properties and methods)
- When a program is executed the objects interact by sending messages to one another.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Abstraction

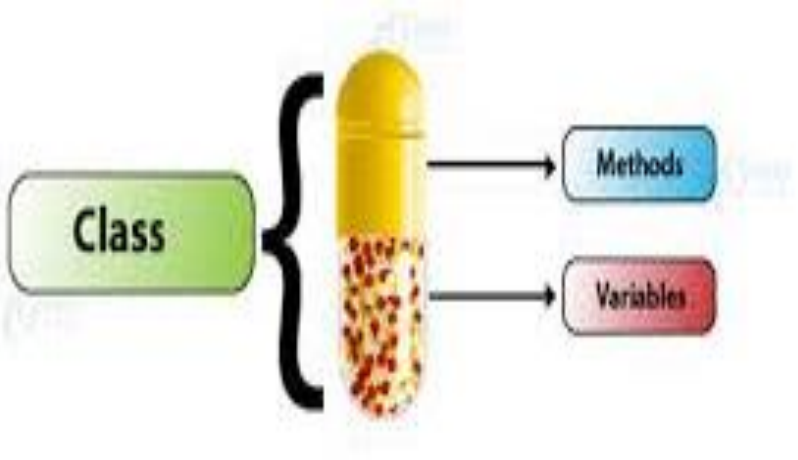
- Abstraction refers to the act of representing essential features without including the background details or explanations.
- Specifying what to do but not how to do.
- It is a flexible feature for having a overall view of an object's functionality.
- Classes use the concept of abstraction and are defined as a list of abstract attributes such as size, weight and cost, and functions to operate on these functions.
- Since the classes use the concept of data abstraction, they are known as Abstract Data Types.

Real Life Example of Abstraction

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Encapsulation

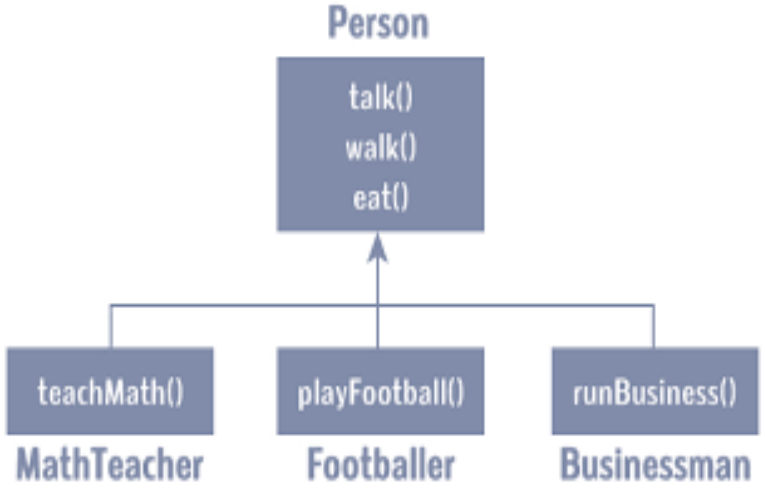
- The wrapping up of data and functions into a single unit is known as Encapsulation.
- The data is not accessible to the outside world and only those functions which are wrapped in the class can access it.
- This insulation of the data from direct access by the program is called data hiding or information hiding.



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Inheritance

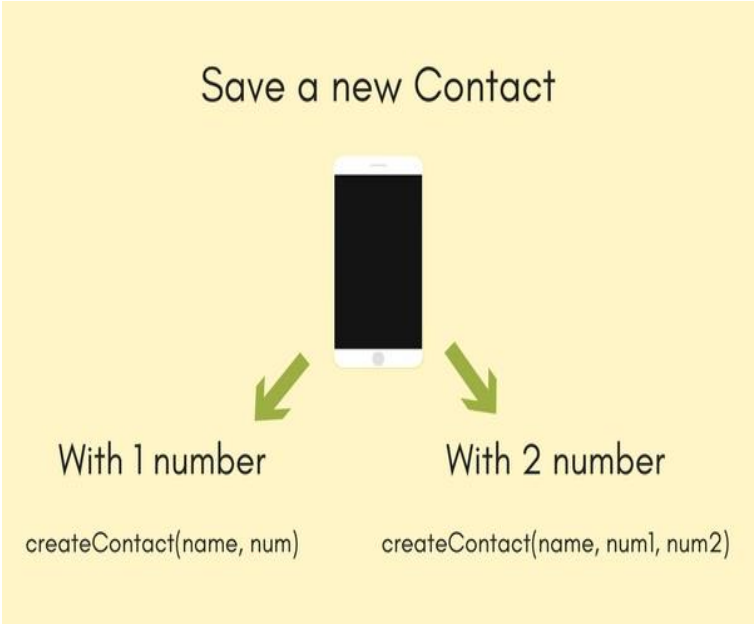
- Inheritance is the process by which objects of one class acquire the properties of objects of another class.
- The principle behind this sort of division is that each derived class shares common characteristics with the class from which it is derived.
- In OOP the concept of inheritance provides the idea of reusability.
- This means that we can add additional features to an existing class without modifying it.



D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Polymorphism

- Polymorphism means the ability to take more than one form.
- Multiple definitions for a single name.
- Functions with same name with different functionality
- That is an operation may exhibit different behaviours in different instances.
- saves time in investing many function names
- The behaviour depends upon the types of data used in the operation.
- Operator and Function overloading
- Example: Operation of addition.



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

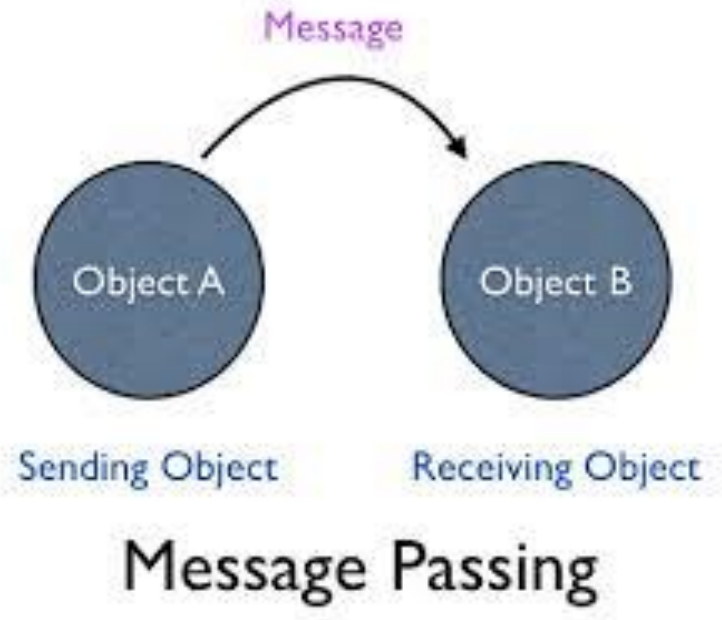
Dynamic Binding

- Dynamic binding refers to the linking of a procedure call to a specific sequence of code.
- The code to be executed for a specific procedure call is not known until run-time.
- Dynamic dispatch is generally used when multiple classes contain different implementations of the same method.
- It provides a mechanism for selecting the function to be executed from various function alternatives at the run-time.
- It is associated with polymorphism and inheritance.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Message Communication

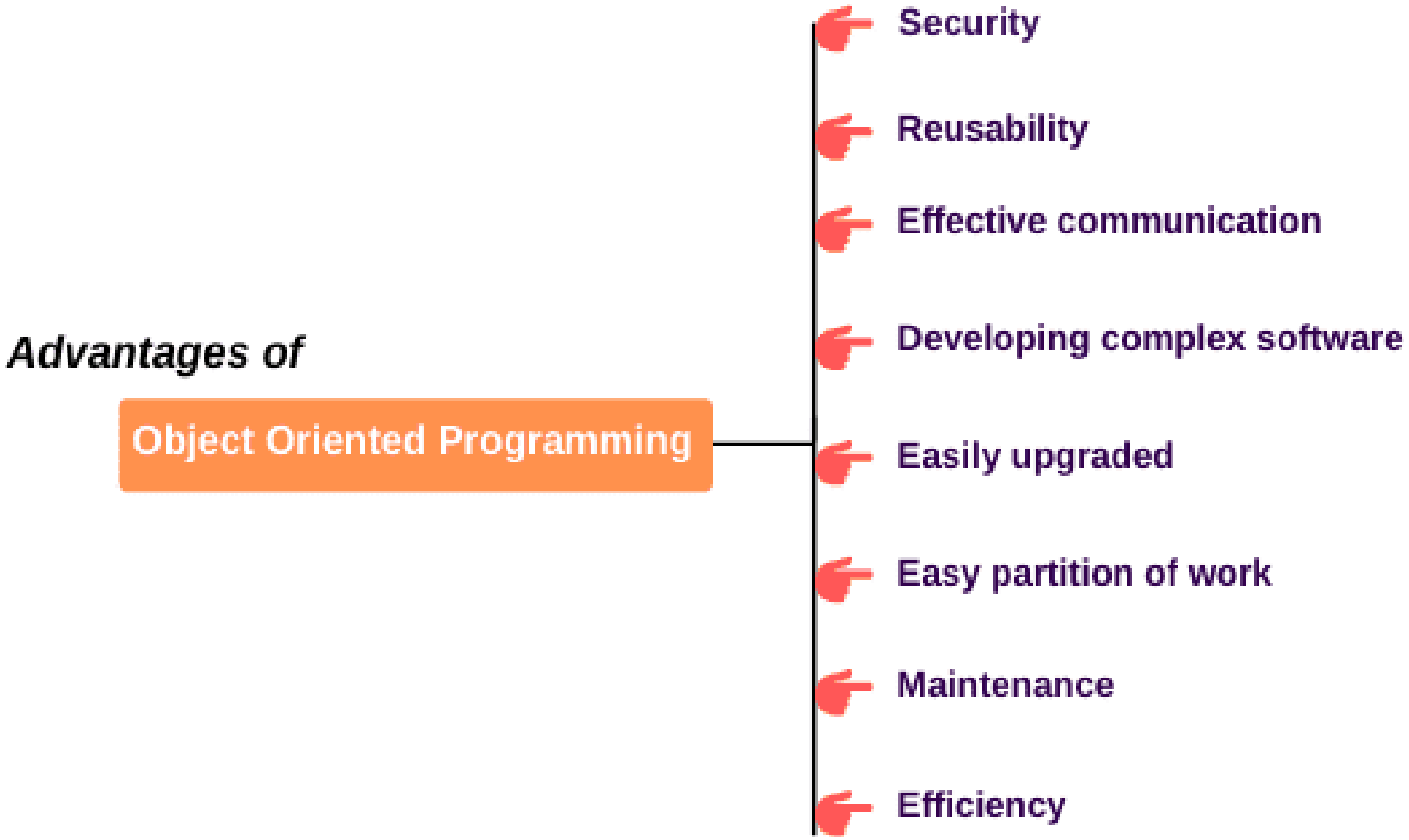
- Object communicate with one another by sending and receiving information much the same way as people pass message to one another.
- Objects communicates through invoking methods and sending data to them.
- This feature of sending and receiving information among objects through function parameters is known as Message Passing.



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 3
Benefits of OOP, Introduction to OOP languages : C++ and Java

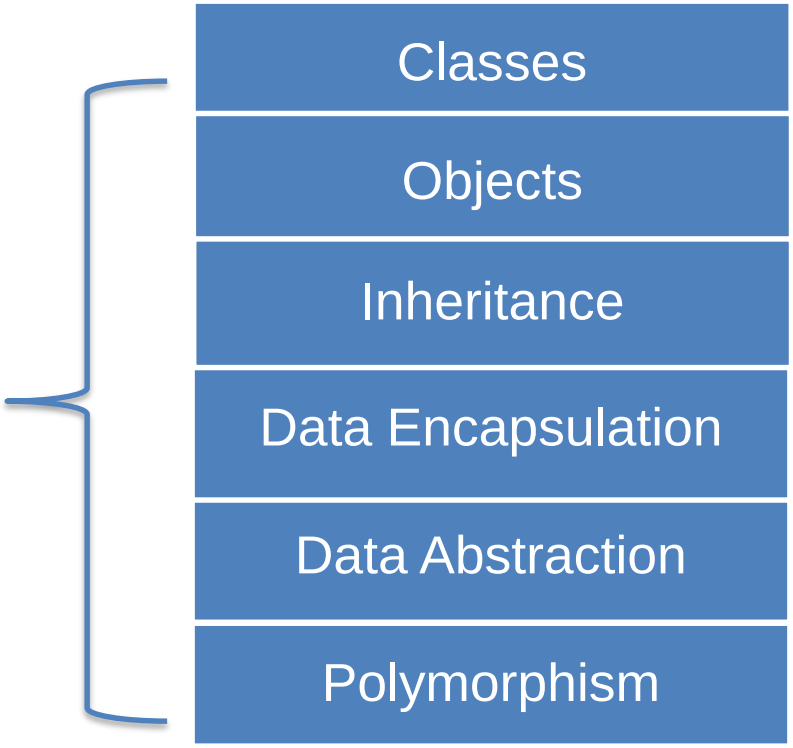
Benefits of OOP



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

OOP: C++ Language

- Developed by Bjarne Stroustrup in 1979.
- an extension / superset of C language.
- Supports both procedural and object oriented paradigm
- The prime purpose of C++ programming was to add **object orientation** to the C programming language.
- While designing C++ modules, we try to see whole world in the form of objects.



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Difference Between C and C++

C	C++
C is the function or procedure programming language.	C++ is the object oriented programming language.
Top down approach is used in C.	Bottom up approach is used in C++.
C is less secure than C++	C++ is secure, because oop's concept is available. Here Data is hidden and can't be accessed by external functions.
In C programming variable declaration is possible only top of program.	In C++ you can any where declare variable.
In C, namespace feature is not available.	In C++, namespace feature is present.
C is a middle level language.	C++ is a high level language.
In this programming, programs are divided into modules and functions.	In this programming, programs are divided into classes and functions.
Here no exception handling concept is available	It support exception handling concept.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

OOP: Java Language

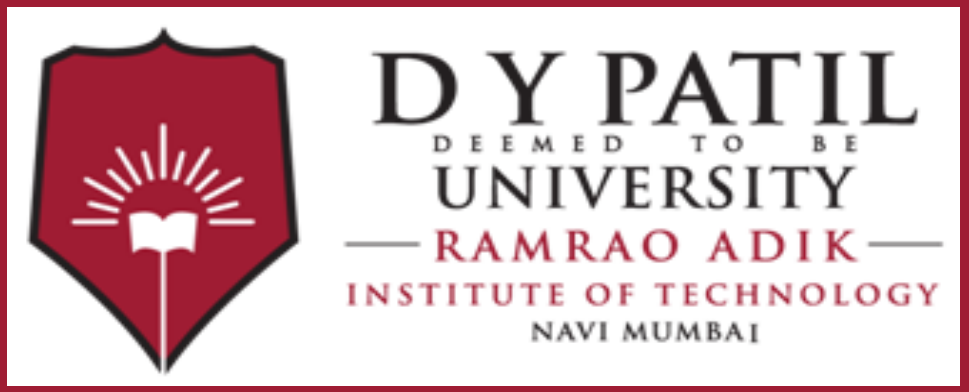
- When we consider a Java program, it can be defined as a collection of objects that communicate via invoking each other's.
- **Object** − Objects have states and behaviors. Example: A dog has states - color, name, breed as well as behaviors - wagging, barking, eating. An object is an instance of a class.
- **Class** − A class can be defined as a template/blueprint that describes the behaviors/states that object of its type support.
- **Methods** − A method is basically a behavior. A class can contain many methods. It is in methods where the logics are written, data is manipulated and all the actions are executed.
- **Instance Variables** − Each object has its unique set of instance variables. An object's state is created by the values assigned to these instance variables.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

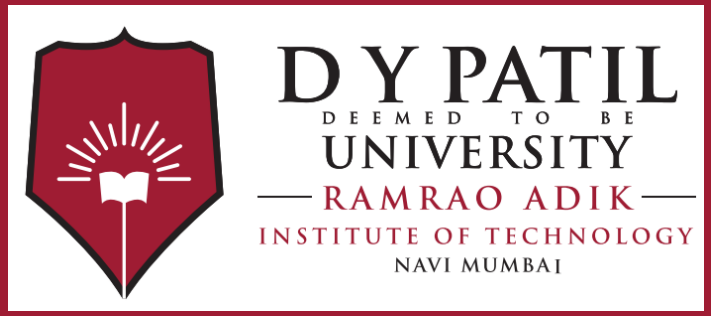
Difference Between C++ and Java

Java	C++
Java doesn't support Pointer concept	It support pointer concept.
It doesn't support multiple inheritances.	It support multiple inheritance
Java does not include structures or unions.	It have structure and union concept.
Java includes automatic garbage collection.	C++ requires explicit memory management
Java has method overloading, but no operator overloading.	C++ supports both method overloading and operator overloading.
It is platform independent programming language	It is platform dependent programming language.
It is mainly used for design web based application but also use for develop desktop application.	It is used for design only desktop application like OS, Compiler etc.
Java uses compiler and interpreter both	C++ use only Compiler.
Java is high level programming language in java we write code like simple English language.	C++ is more nearer to hardware then Java

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI


D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You



Subject Name: OOP

Unit No:1

Unit Name: Introduction to OOP

Faculty Name : Mr. Dayanand Dhongade

Index

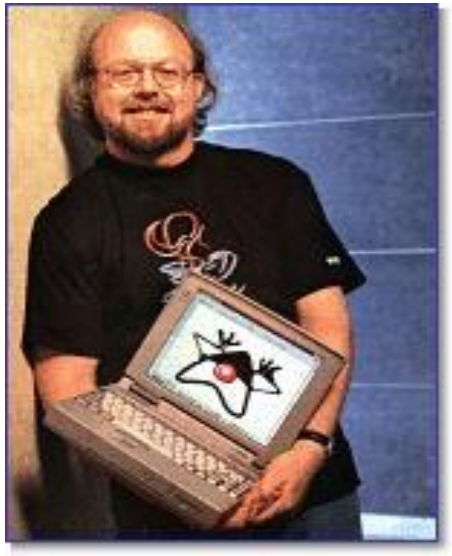
Lecture 4 – Features of Java Language, JDK environment	**3**
Lecture 5 – Basic Constructs of Java Programming	13
Lecture 6 – Input and output functions in Java : Buffered reader class, Scanner class	39

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 4
Features of Java Language, JDK environment

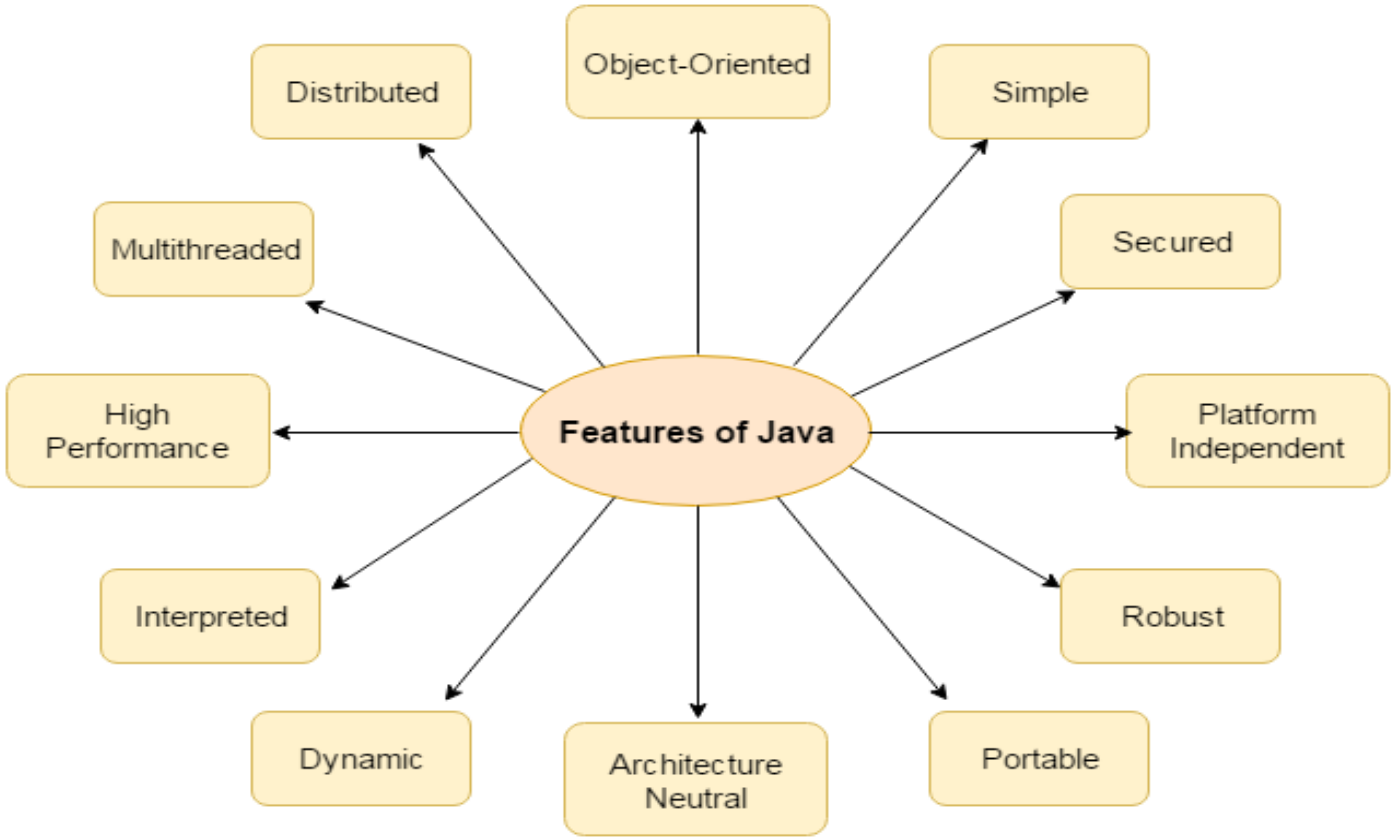
Introduction to JAVA

- A general purpose object oriented programming language developed by Sun Microsystems in 1991.
- Originally called Oak by **James Gosling**, the first designer of Java and implemented its original compiler and virtual machine.
- Renamed as **JAVA** in 1995
- Versions
  - Java 2 Micro Edition (J2ME)
  - Java 2 Standard Edition (J2SE)
  - Java 2 Enterprise Edition (J2EE)



D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Features of Java



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Features of Java

- **Simple:**
  - Easy to learn.
  - removed many confusing and rarely-used features
    e.g., explicit pointers, operator overloading etc.

- **Object-oriented:**
  - Organize our software as a combination of different types of objects that incorporates both data and behaviour.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Features of Java

- **Platform Independent:**
  - A platform is the hardware or software environment in which a program runs.
  - It has two components:
    - ✓ Runtime Environment
    - ✓ API(Application Programming Interface)
  - "Write-Once Run-Anywhere"
    - ✓ Java code can be run on multiple platforms e.g. Windows, Linux, Sun Solaris, Mac/OS etc.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Features of Java

- **Robust:**
  - Strong memory management.
  - No usage of pointers.
  - Exception handling and type checking mechanisms.

  “An Exception is the condition that is caused by run time error in the program. If it occurred java interpreter creates an exception object and throws it.”

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Features of Java

- **Multi-threaded**
  - A thread is like a separate program, executing concurrently.
  - Java programs deal with many tasks at once by defining multiple threads.
  - Doesn't occupy memory for each thread.
  - Shares a common memory area.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Features of Java

- **Distributed and Network Oriented**

  Java grew up in the days of the Internet

  - Inherently network friendly
  - Original release of Java came with Networking libraries
  - Newer releases contain even more for handling distributed applications

    - RMI, Transactions

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

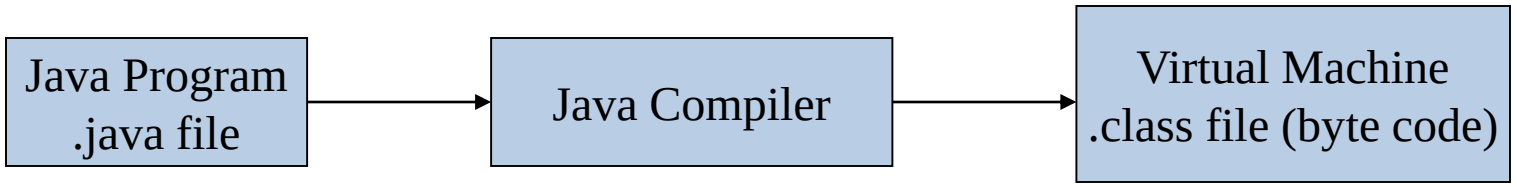
Features of Java

- **Compiled and Interpreted**
  - Java works in two stage
    - **Java compiler** translate the source code into byte code.
    - **Java interpreter** converts the byte code into machine level representation.
  - **Byte Code:**
    - A highly optimized set of instructions to be executed by the java runtime system, known as java virtual machine (JVM).
    - Not executable code.
  - **Java Virtual Machine (JVM):**
    - Need to be implemented for each platform.
    - Although the details vary from machine to machine, all JVM understand the same byte code.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Java Virtual Machine

- ✓ Java compiler produces an intermediate code known as byte code for a machine, known as JVM.
- ✓ It exists only inside the computer memory.



- ✓ Machine code is generated by the java interpreter by acting as an intermediary between the virtual machine and real machine.

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 5
Basic Constructs of Java Programming

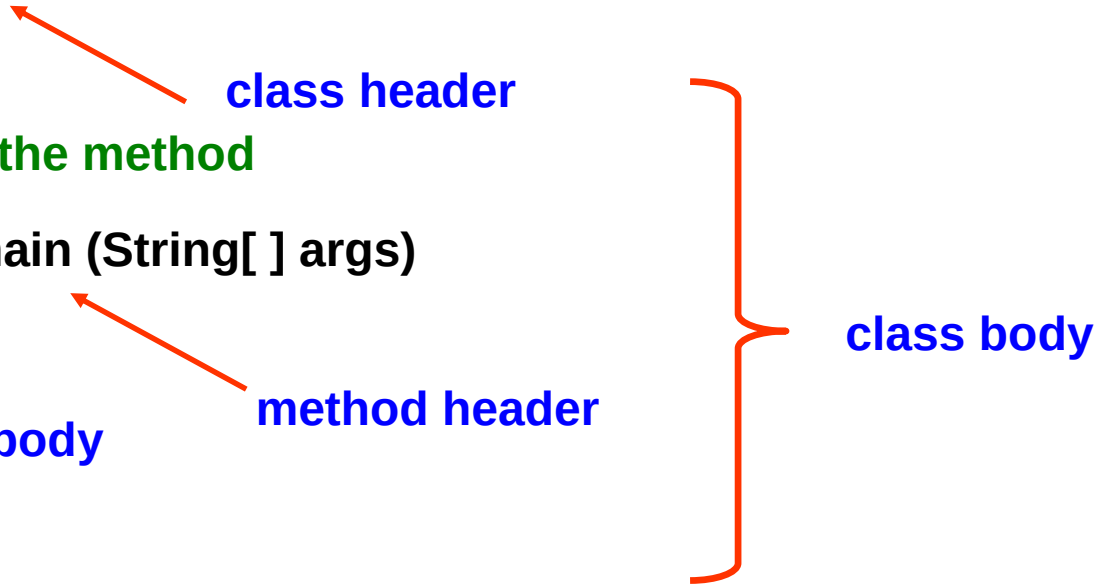
Simple Java Program

```
// Program to print Hello message
public class Hello
{
    public static void main (String[ ] args)
    {
        System.out.print ("Hello");
    }
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

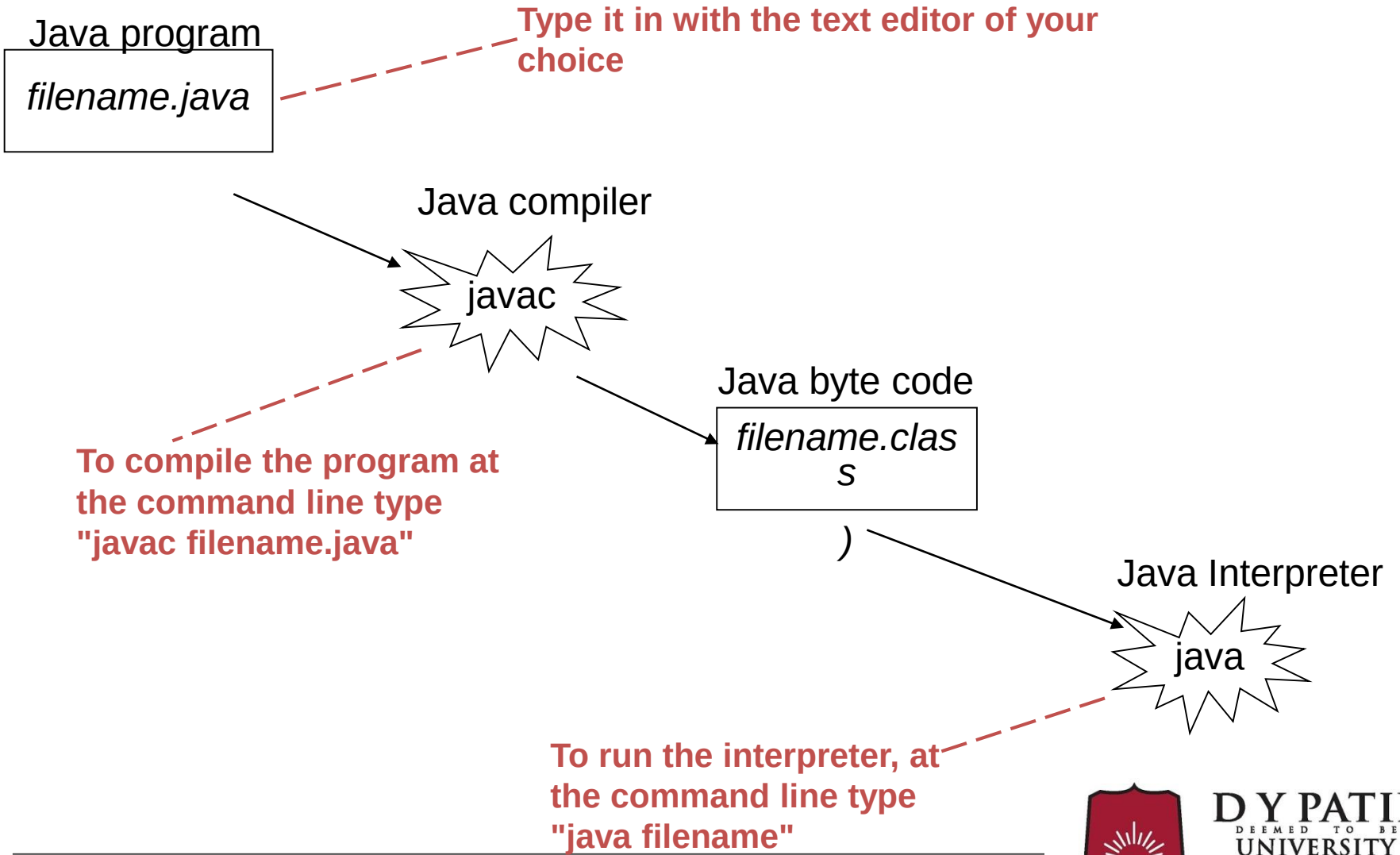
Java Program Structure

```
// comments about the class
public class MyProgram
{
    // comments about the method
    public static void main (String[ ] args)
    {

    }
}
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Compile and Execute Java Program



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Basic Constructs of Java

- Constants
- Variables
- Data Types
- Operators and Expression
- Revision of Branching and Looping

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Constants

- The variable whose value is always constant (does not change).
- The reserved word **final** is used in constant declaration.
- Syntax:

  final data_type constant_name=constant_value;

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Variables

- A variable can be thought of as a container which holds value for you, during the life of a Java program.
- Every variable is assigned a **data type** which designates the type and quantity of value it can hold.
- All variables must be declared before its use.
- Variables can be declared almost anywhere (scope rules apply)
- Variables have default initialization values
  - Integers: 0
  - Reals: 0.0
  - Boolean: False
- Variables can be initialized in the declaration.
- **Syntax**:

  [access_modifier] [**static**] [**final**] type name = [value];

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Data Types

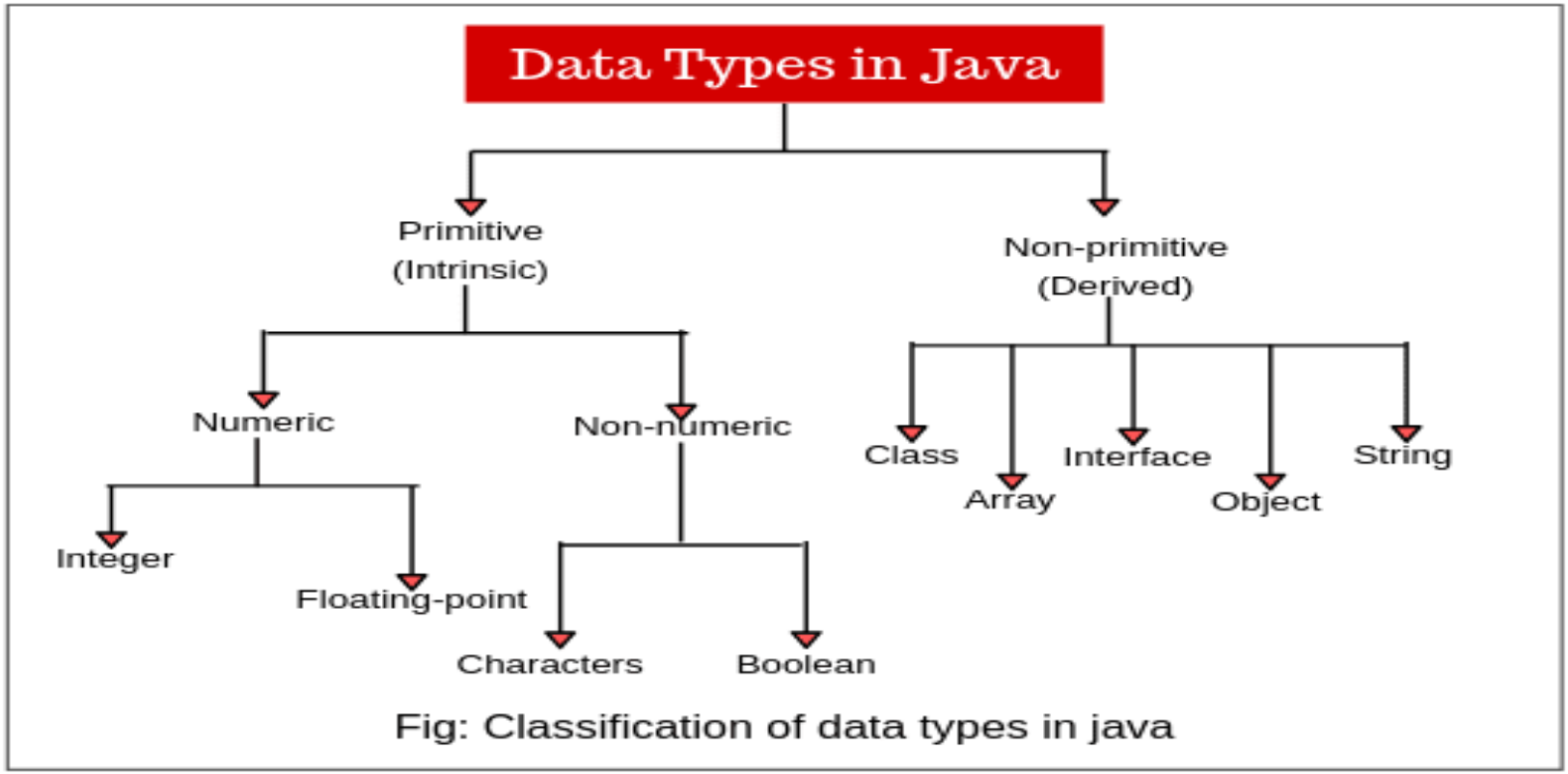


Fig: Classification of data types in java

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Data Types

- **Primitive Data Types**
  - byte (1), -128 to 127
  - short (2), -32768 to 32767
  - int (4), -2147483648 to 2147483647
  - long (8), -9223372036854775808 to 9223372036854775807
  - float (4), -3.4E38 to 3.4E38, 7 digit precision
  - double (8), -1.7E308 to 1.7E308, 17 digits precision
  - char (2), unicode characters
  - boolean (true, false), discrete values

- **Non-primitive/ User-defined Data Types**
  - Array
  - Class
  - interface

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Keywords in Java

- These are the reserved words in JAVA which cannot be used as variable names or identifiers.

abstract	continue	for	new	switch
assert***	default	goto*	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum****	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp**	volatile
const*	float	native	super	while

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Operators

- Operators/Order of Precedence

Operator	Operation	Precedence
() or [ ] or . or ++ or - -	Parens, array indices, object invocation, increment, decrement	1
+ or – or ~	Unary plus, unary minus, complement	2
* or / or %	Multiplication, division, modulus	3
+ or -	Addition, subtraction	4
<<,>>	Bitwise shifts	5
&	Bitwise AND	6
^	Bitwise XOR	7
|	Bitwise OR	8

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Java Operators

- Example Usage

```
int anInt1 = 2, anInt2 = 1, anInt3;
double aDbl1;
anInt3 = anInt1 + anInt2 * 4;          // anInt3 gets 6
anInt3 = (anInt1 + anInt2) * 4;        // anInt3 gets 12
anInt3 = ++anInt2 * 2;                 // anInt3 gets 4
anInt2 = 1;
anInt3 = anInt2++ * 2;                 // anInt3 gets 2
anInt2 = 1;
anInt3 = anInt1 & anInt2;              // anInt3 gets 0
anInt3 = anInt1 ^ anInt2;              // anInt3 gets 3
anInt3 = anInt2 << 1;                  // anInt3 gets 2
anInt3 = 1 / 2;                        // anInt3 gets 0
aDbl1 = 1 / 2;                         // aDbl1 gets 0.0
aDbl1 = 1.0 / 2.0;                     // aDbl1 gets 0.5
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Java Operators…..

- Assignments
  - General Format: variable = expression ;

    Where variable is a previously declared identifier and expression is a valid combo of identifiers, operators, and method (a.k.a. procedure or function) calls
  - Shortcuts:

```
var *= expr ;    // Equivalent to var = var * (expr);
var /= expr ;    // Equivalent to var = var / (expr);
var += expr ;    // Equivalent to var = var + (expr);
var -= expr ;    // Equivalent to var = var - (expr);
var %= expr ;    // Equivalent to var = var % (expr);
var++;           // Equivalent to var = var + 1;
var--;           // Equivalent to var = var - 1;
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Control Statements in Java

1. Conditional Statements
- if-else statements
- switch case statements

2. Iterative Statements
- for loop
- while loop
- do-while loop

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Conditional Constructs

- "if" Statements
  - if with code block

```
if (boolean_expr)
{
      statements
}
```

  - if with single statement

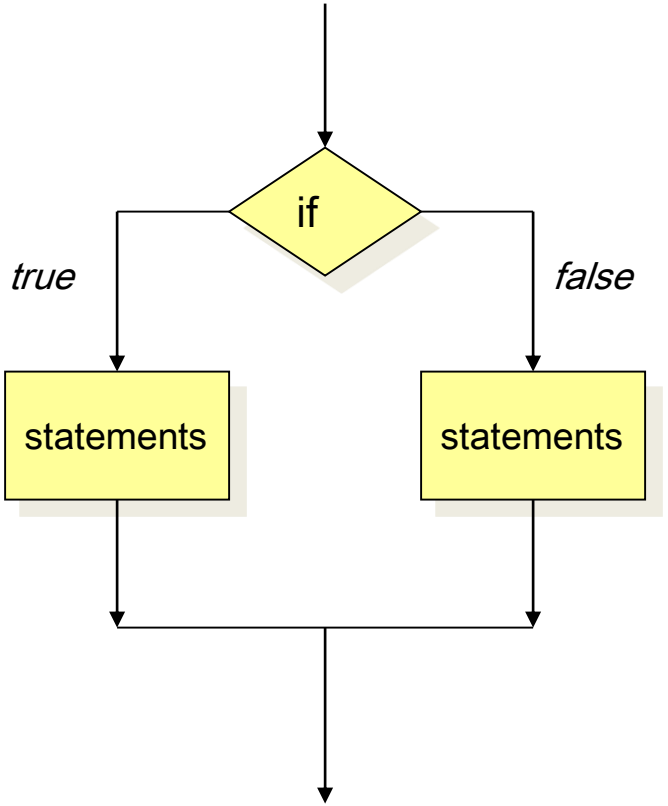
```
if (boolean_expr)
      statement;
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Conditional Constructs

- if" Statements (Continued)
  - if-else

```
if (boolean_expr)
{
     statements for true
}
else
{
     statements for false
}
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Conditional Constructs

- Boolean Expressions
  - Boolean expressions use conditional operators such that they result in a value of true or false
  - Conditional Operators (Not by order of precedence)

Operators	Operations
== or !=	Equality, not equal
> or <	Greater than, less than
>= or <=	Greater than or equal, less than or equal
!	Unary negation (NOT)
& or &&	Evaluation AND, short circuit AND
| or ||	Evaluation OR, short circuit OR

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Java Conditional Constructs

- Boolean Expression Examples

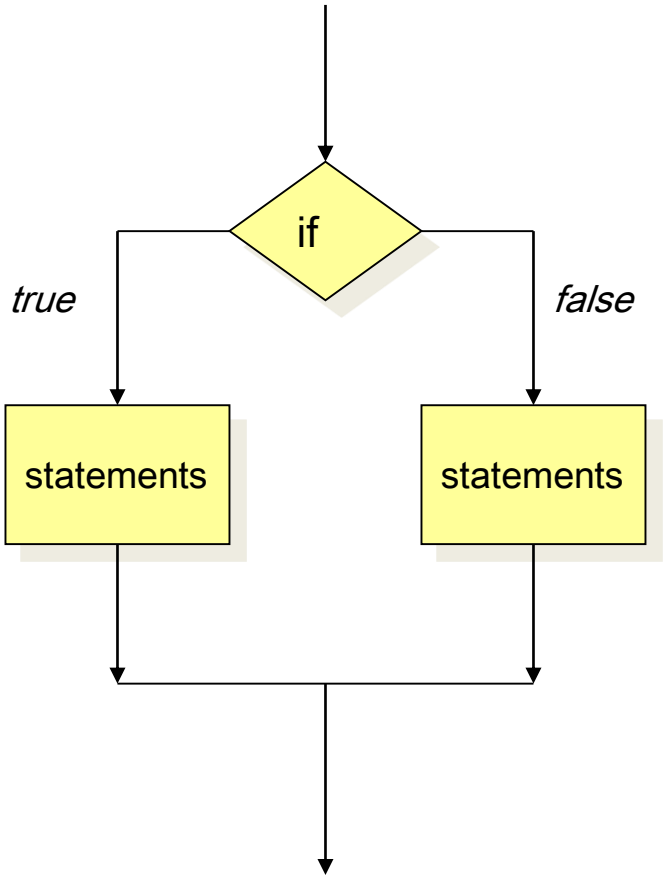
```
int i1 = 1, i2 = 2, i3 = 3, i4 = 0;
boolean b1 = true, b2 = false, b3 = true;

i2 > i1                          // true
(i3 - i2) > i1                   // false
(i3 - i2) >= i1                  // true
(i2 > i1) & (i2 < i3)            // true
(i2 < i1) | (i2 > i1)            // true
i2 != i1                         // true
(i1 < i2) | ((i1/i4) > 1)        // Divide by 0 exception
(i1 < i2) || ((i1/i4) > 1)       // true
(i1 < i2) | (i1++ > 1)           // true, i1 contains 2
(i1 < i2) || (i1++ > 1)          // true, i1 contains 1
b1 && b2 && b3                   // false
(b1 || b2) && b3                 // true
b1 && (i1 == (i3 - i2))          // true
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Java Conditional Constructs

- if-else" Statement Example

```
class Example
{
    static public void main(String args[])
    {
        // A very contrived example
        int i1 = 1, i2 = 2;
         System.out.print("Result: ");
        if (i1 > i2)
        {
            System.out.println("i1 > i2");
        }
        else
        {
            System.out.println("i2 >= i1");
        }
    }
}
```

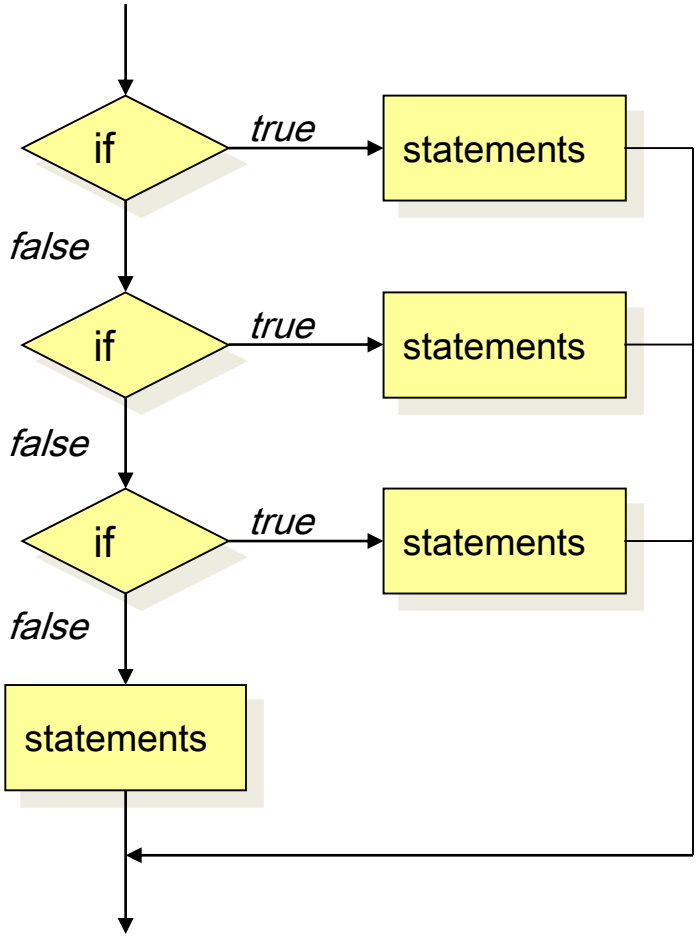


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Conditional Constructs

- The Switch Statement

```
switch (integer_expression)
{
     case int_value_1:
          statements
          break;
     case int_value_2:
          statements
          break;
     …
     case int_value_n:
          statements
          break;
     default:
          statements
}
```

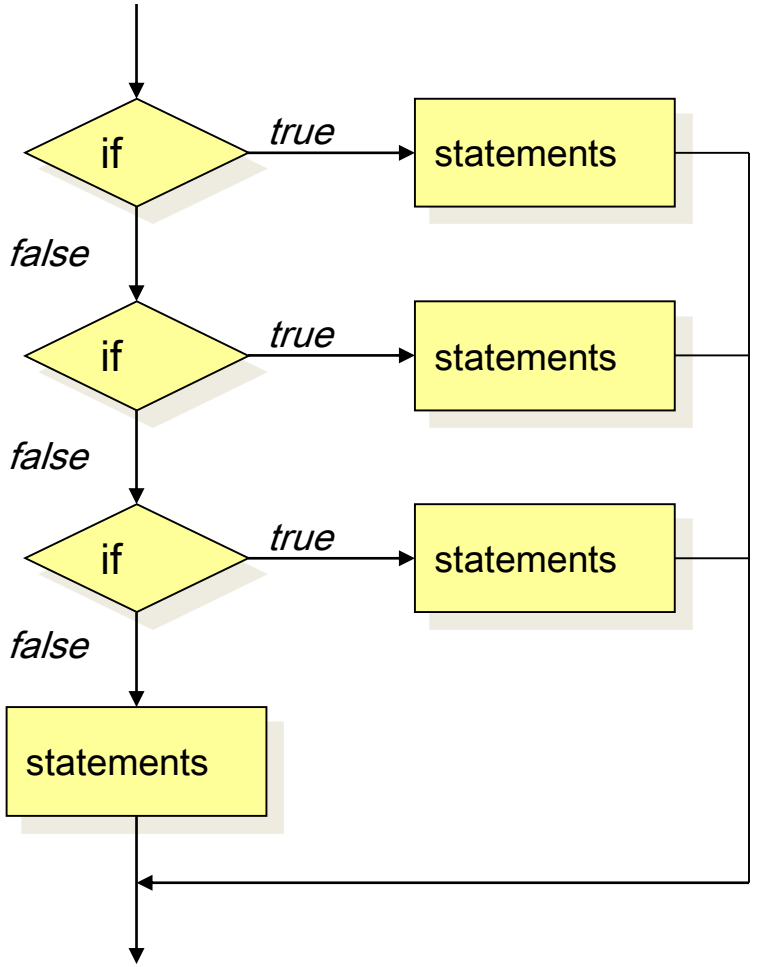


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Java Conditional Constructs

- Don't forget the "break"

```
switch (integer_expression)
{
     case int_value_1:
          statements
          // No break!
     case int_value_2:
          statements
          break;
     …
     case int_value_n:
          statements
          break;
     default:
          statements
}
```

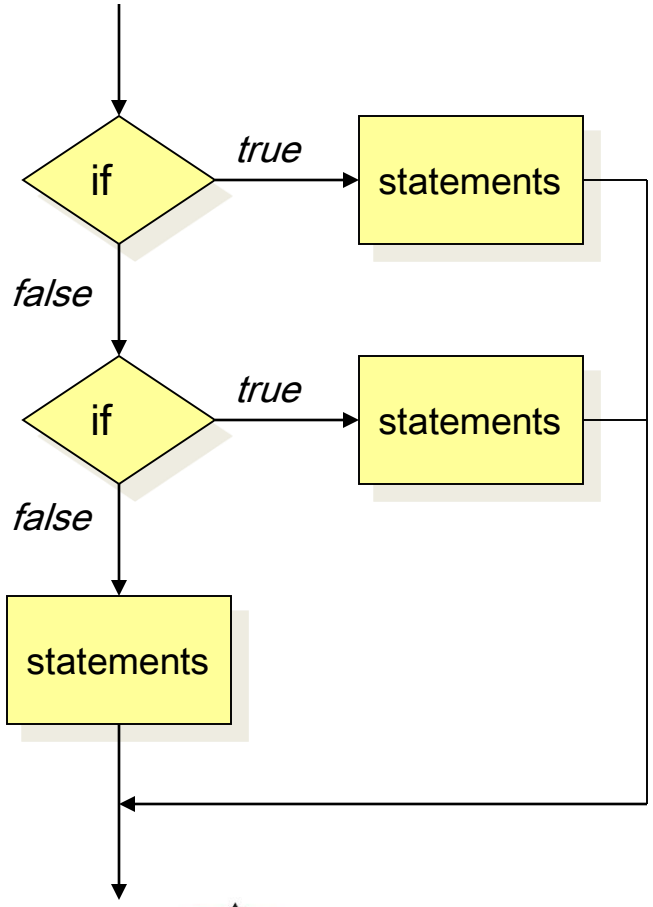


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Conditional Constructs

- Example

```
int n = 5;
switch (n)
{
    case 1:
        n = n + 1;
        break;
    case 5:
        n = n + 2;
        break;
    default:
        n = n - 1;
}
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Looping Constructs

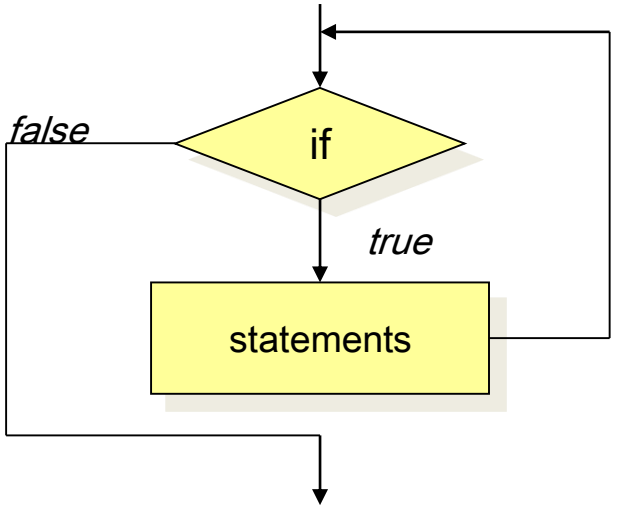
- while loop
  - Exit condition evaluated at top
- do loop
  - Exit condition evaluated at bottom
- for loop
  - Exit condition evaluated at top
  - Includes a initialization statements
  - Includes a update statements for each iteration

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Java Looping Constructs

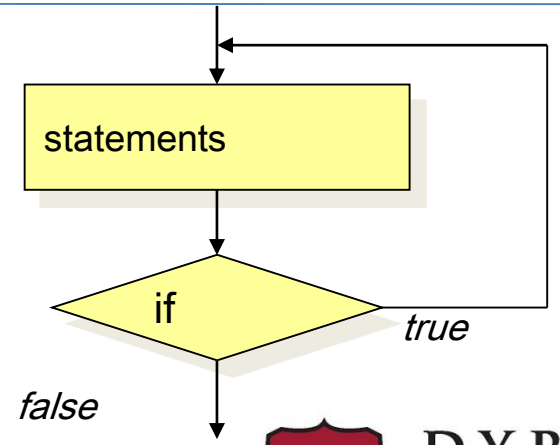
- while loop

```
while (boolean_expr)
{
    statements
}
```



- do loop

```
do
{
    statements
}
while (boolean_expr)
```

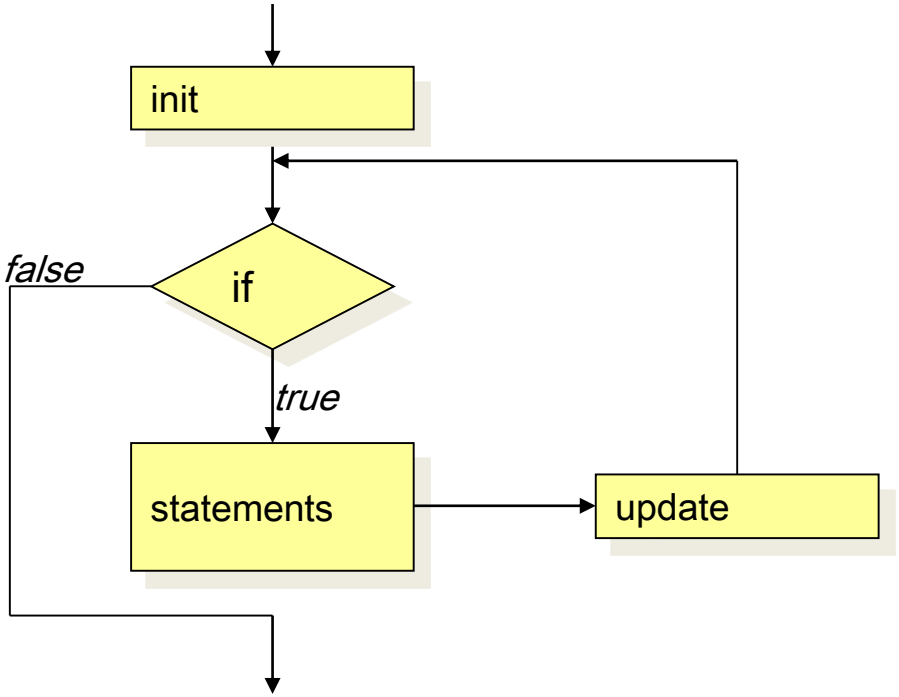


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Looping Constructs

- for loop

```
for (init_stmnt; bool_expr; update_stmnt)
{
    statements
}
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java Looping Constructs

```
class Example
{   static public void main(String args[])
   {
      int i = 0;
      System.out.println("while loop");
      while (i < 10)
      {
         System.out.println(i);
         i++;
      }

      System.out.println("do loop");
      do
      {
         System.out.println(i);
         i--;
      }
      while (i > 0);

     System.out.println("for loop");
      for (i = 0; i < 10; i++)
      {
         System.out.println(i);
      }
   } // End main
} // End Example
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 6
Input and output functions in Java : Buffered reader class, Scanner class

Input/Output

- Java I/O (Input and Output) is used to process the input and produce the output.
- Java uses the concept of a stream to make I/O operation fast.
- The java.io package contains all the classes required for input and output operations.
- We can perform file handling in Java by Java I/O API.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

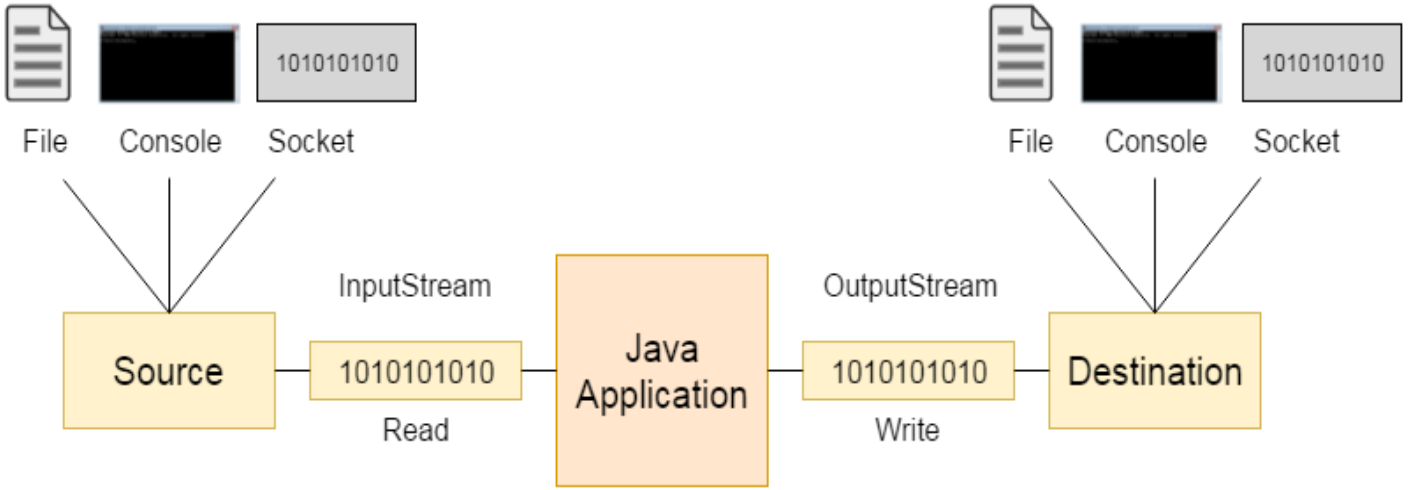
Input/Output

- **Stream -** A stream is a sequence of data. In Java, a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.
- In Java, 3 streams are created for us automatically. All these streams are attached with the console.
  - System.out: standard output stream
  - System.in: standard input stream
  - System.err: standard error stream

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

OutputStream vs InputStream

- **OutputStream -** Java application uses an output stream to write data to a destination; it may be a file, an array, peripheral device or socket.
- **InputStream -** Java application uses an input stream to read data from a source; it may be a file, an array, peripheral device or socket.



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

OutputStream class

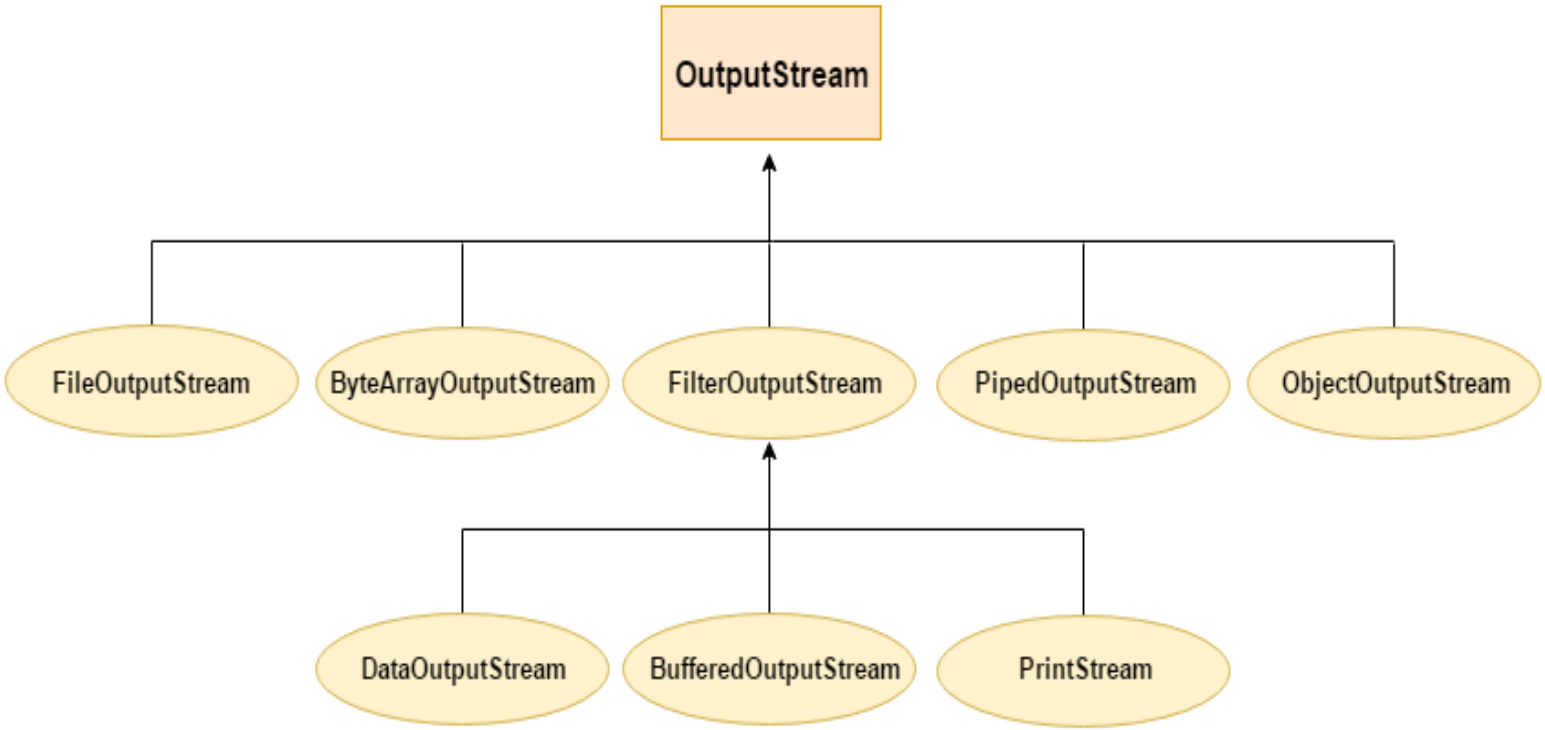
- OutputStream class is an abstract class. It is the superclass of all classes representing an output stream of bytes. An output stream accepts output bytes and sends them to some sink.
- **Useful methods of OutputStream**

Method	Description
1) public void write(int)throws IOException	is used to write a byte to the current output stream.
2) public void write(byte[])throws IOException	is used to write an array of byte to the current output stream.
3) public void flush()throws IOException	flushes the current output stream.
4) public void close()throws IOException	is used to close the current output stream.

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

OutputStream class

- **OutputStream Hierarchy**



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

InputStream class

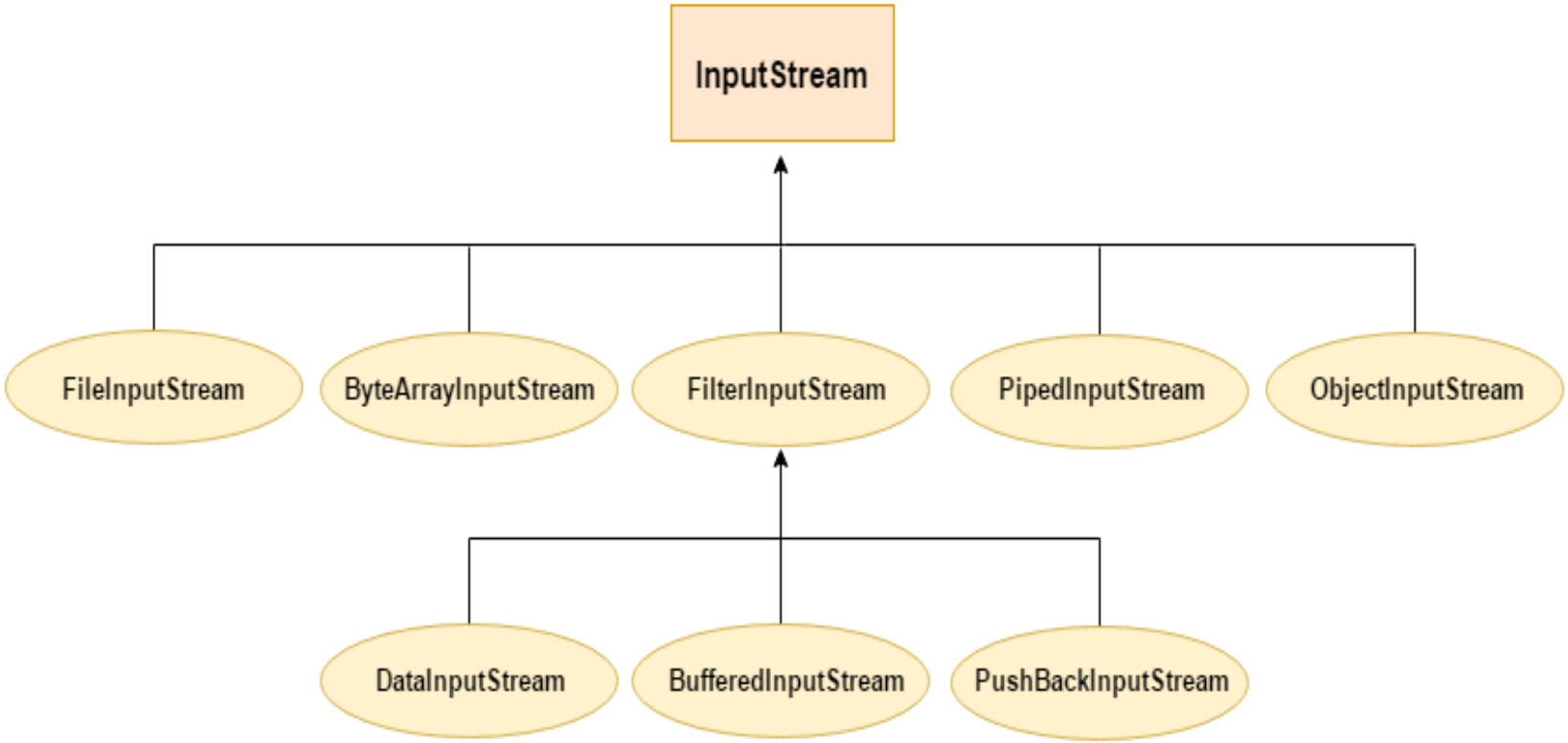
- InputStream class is an abstract class. It is the superclass of all classes representing an input stream of bytes.
- **Useful methods of OutputStream**

Method	Description
1) public abstract int read()throws IOException	reads the next byte of data from the input stream. It returns -1 at the end of the file.
2) public int available()throws IOException	returns an estimate of the number of bytes that can be read from the current input stream.
3) public void close()throws IOException	is used to close the current input stream.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

InputStream class

- **InputStream Hierarchy**



D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Classes used for Reading Information

- **BufferedReader** (has always been there)
  - **import java.io.*;**
- **Scanner** (specific to JAVA 5 and beyond)
  - **import java.util.*;**

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

java.io.BufferedReader

- The BufferedReader class can only read Strings
- However we can transform those Strings into numbers of any kind.
- This one is always safe.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Two Objects and one Exception

- In the class:

  public static void main(String args[ ]) **throws IOException**{

  InputStreamReader isr = new InputStreamReader(System.in);

  BufferedReader br = new BufferedReader(isr);

- The method:

  String readLine( )

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Reading a String with BufferedReader

- Reading a String

```
InputStreamReader isr = new InputStreamReader (System.in);
BufferedReader br = new BufferedReader(isr);
String sName;

System.out.print("What is your name?   ");
sName = br.readLine( );
System.out.println("So your name is " + sName);
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Reading integers with BufferedReader

- Reading an int number

```
String sPlaceHolder;
int iAge =0;
int iYear = 2023;

System.out.print("How old are you?   ");
sPlaceHolder = br.readLine( );
iAge = Integer.parseInt(sPlaceHolder);
System.out.println("So you are  " + iAge + " years old");
System.out.println("you were probably born in "+ (iYear - iAge));
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Reading doubles with BufferedReader

- Reading a double number

```
final double dGoal = 1000000;  //constant
double dEarnings = 0;
double dMissing = 0;

System.out.print("How much do you make per hour?  ");
sPlaceHolder = br.readLine( )
dEarnings = Double.parseDouble(sPlaceHolder);
dMissing = dGoal - dEarnings;
System.out.println("\nIn one hours you will only need " + iMissing + "more to earn your goal");
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Scanner syntax

- The Scanner class is found in the java.util package.

  import java.util.*; // so you can use Scanner

- Constructing a Scanner object to read console input:

  Scanner name = new Scanner(System.in);

- Example:

  Scanner sc = new Scanner(System.in);

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Scanner Class -The Methods

- **Constructor**

  Scanner sc = new Scanner(System.in);

- **Reading a String**

  String sc.nextLine( );

- **Reading an int**

  int sc.nextInt( );

- **Reading a double**

  double sc.nextDouble( );

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Scanner methods

Method	Description
nextInt()	reads an integer from the user
nextDouble()	reads a double from the user
next()	reads a one-word String from the user
nextLine()	reads a one-line String from the user

- Each method waits until the user presses Enter.
- The value typed by the user is returned.
- Example:

```
System.out.print("How old are you? ");  // prompt
int age = sc.nextInt();
System.out.println("You typed " + age);
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Reading a String with the Scanner class

- Reading a String

```
Scanner scan = new Scanner(System.in);

String str;

System.out.print("What is your name?   ");
str = scan.nextLine( );
System.out.println("So your name is " + str);
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Reading integers with the Scanner class

- Reading an int number

```
Scanner scan = new Scanner(System.in);

int iAge = 0;
Int iYear = 2023;

System.out.print("How old are you? ");
iAge = scan.nextInt(  );
iYear = iYear - iAge;
System.out.println("\nSo you were born around " + iYear);
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Reading doubles with the Scanner Class

- Reading a double number

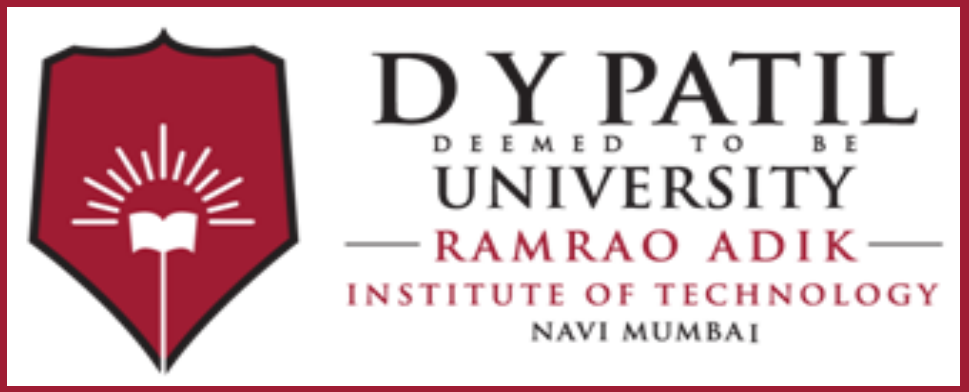
```
Scanner scan = new Scanner(System.in);

final double dGoal = 1000000;  //constant
double dEarnings = 0;
double dMissing = 0;

System.out.print("How much do you make per hour?  ");
dEarnings = scan.nextDouble(  );
dMissing = dGoal - dEarnings;
System.out.println("\nIn one hours you will only need " + iMissing +
   "more to earn your goal");
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

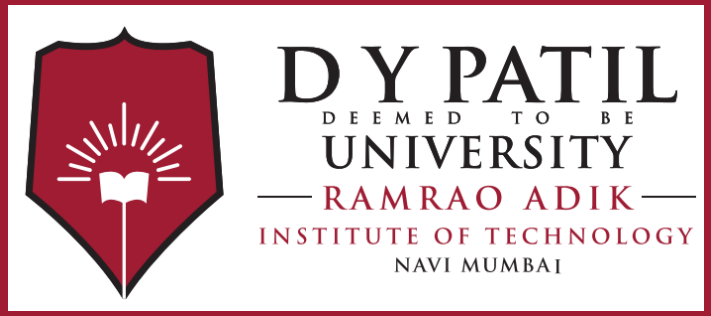
Difference between Scanner and BufferReader Class in Java

Sr. No.	Key	Scanner Class	BufferReader Class
1	Synchronous	Scanner is not synchronous in nature and should be used only in single threaded case.	BufferReader is synchronous in nature. During multithreading environment, BufferReader should be used.
2	Buffer Memory	Scanner has little buffer of 1 KB char buffer.	BufferReader has large buffer of 8KB byte Buffer as compared to Scanner.
3	Processing Speed	Scanner is bit slower as it need to parse data as well.	BufferReader is faster than Scanner as it only reads a character stream.
4	Methods	Scanner has methods like nextInt(), nextShort() etc.	BufferReader has methods like parseInt(), parseShort() etc.
5	Read Line	Scanner has method nextLine() to read a line.	BufferReader has method readLine() to read a line.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI


D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You



Subject Name: OOP

Unit No:2

Unit Name: Classes and Objects

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 7 – Introduction to Class and Object Fundamentals	3
Lecture 8 – Method Definition	12
Lecture 9 – Access Specifier, Class scope and accessing class members	19

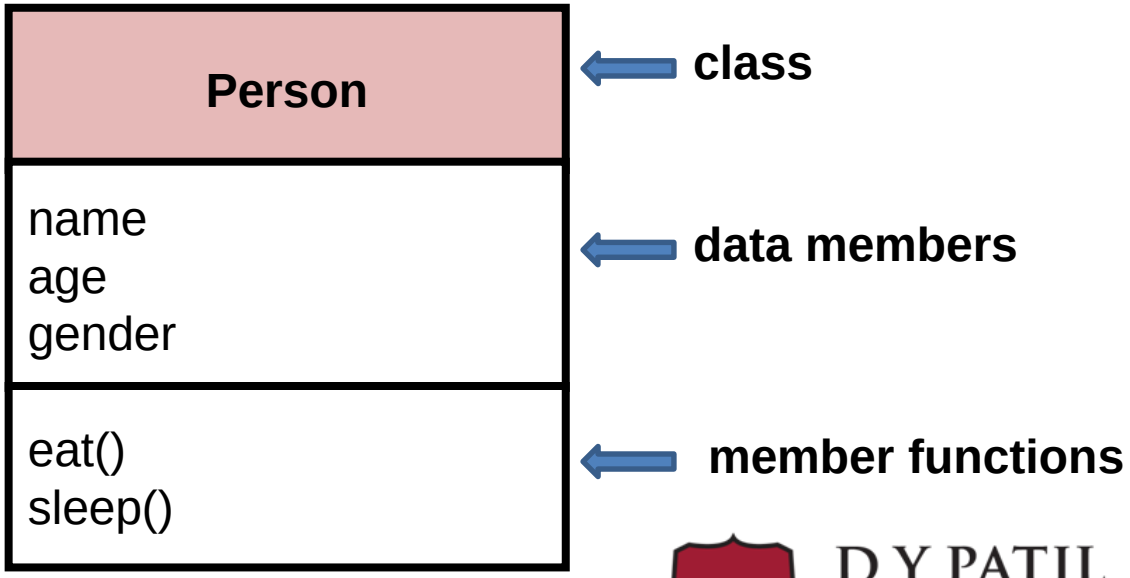
D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 7
Introduction to Class and Object Fundamentals

Class

- A ***class*** can be defined as a template/blue print that describes the behaviors/states that object of its type support.
- It is a user-defined data type.
- It is a collection of *fields/variables* (data members) and *member functions/methods* (procedure or function) that operate on the data.

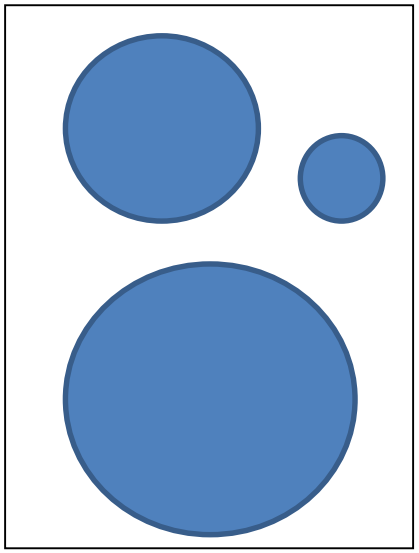
E.g.

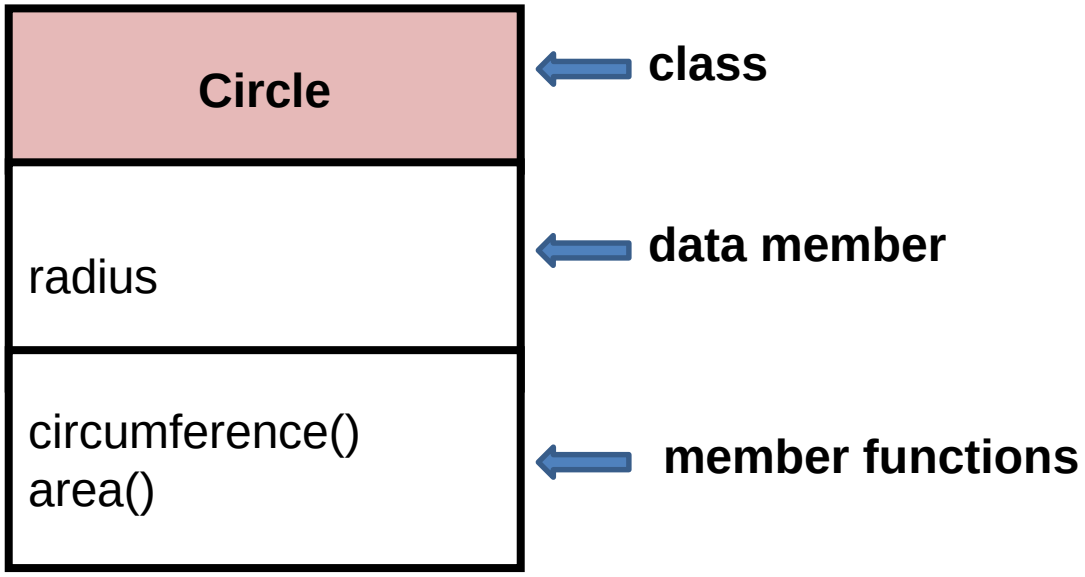


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Class

E.g.



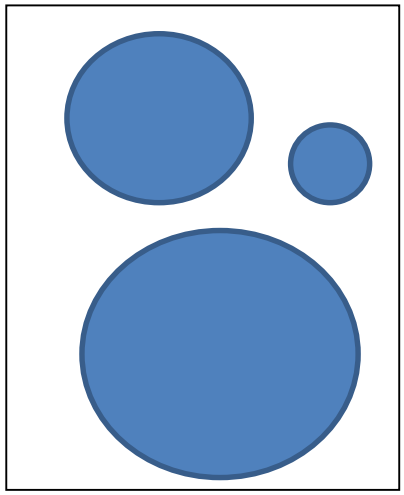


D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

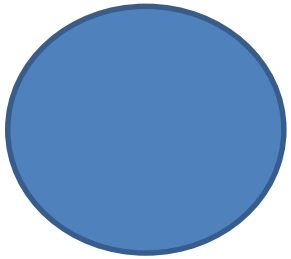
Object

- class provides the blueprints for objects. So basically an object is created from a class.
- Objects have states and behaviors
- The state of an object is stored in fields (variables), while methods (functions) display the object's behaviour

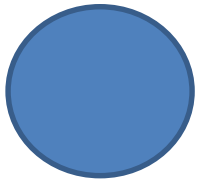
Class



Objects



radius= 4



radius= 3

radius= 1

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Class and Objects

Class

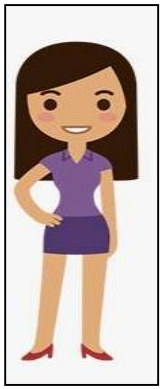
Person
name age gender

Objects

John
55
M

Mary
30
F

Jim
38
M



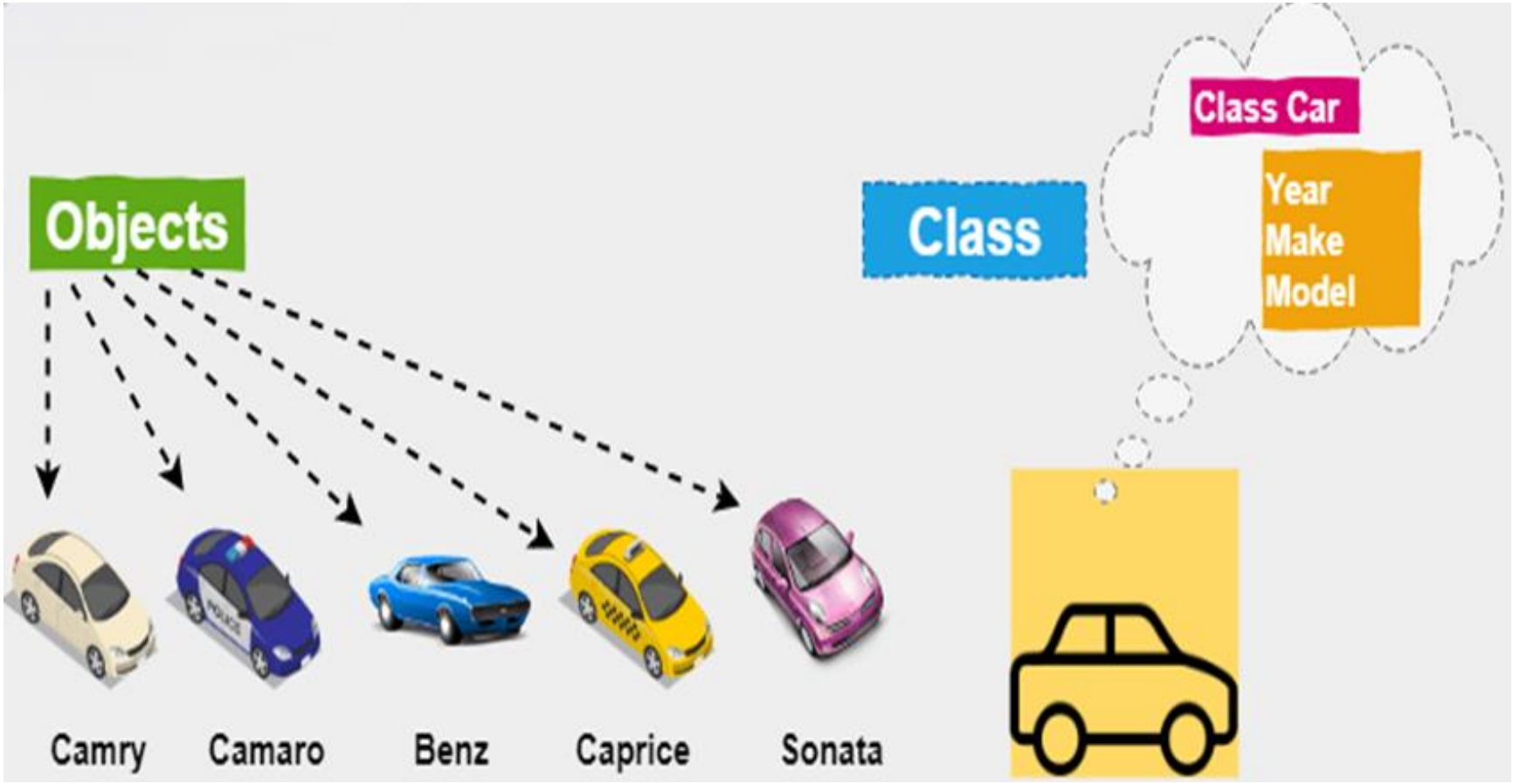
Mia
20
F

Class is a template representing common behavior of objects

Objects are instances of class

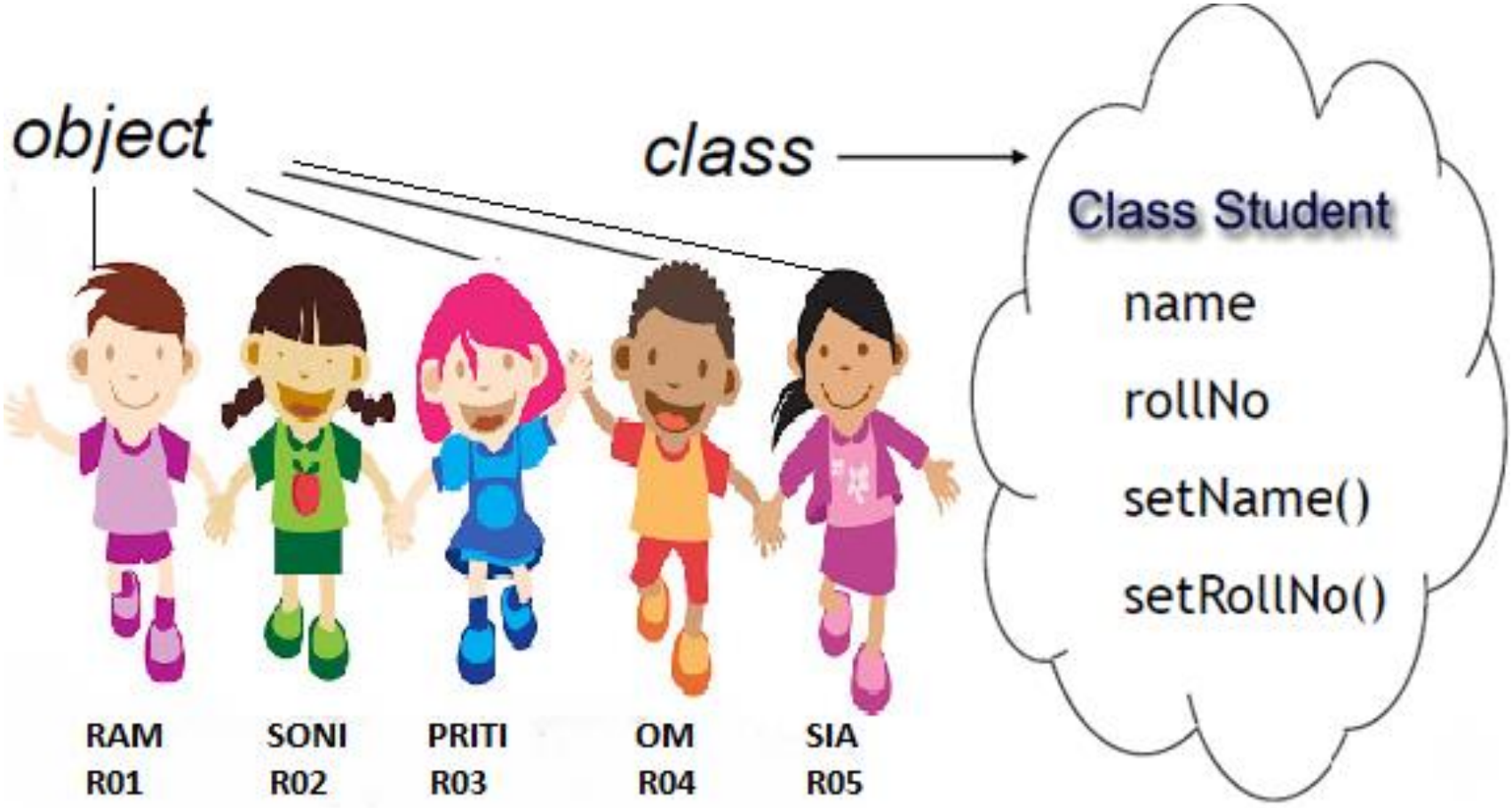
D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Class and Objects



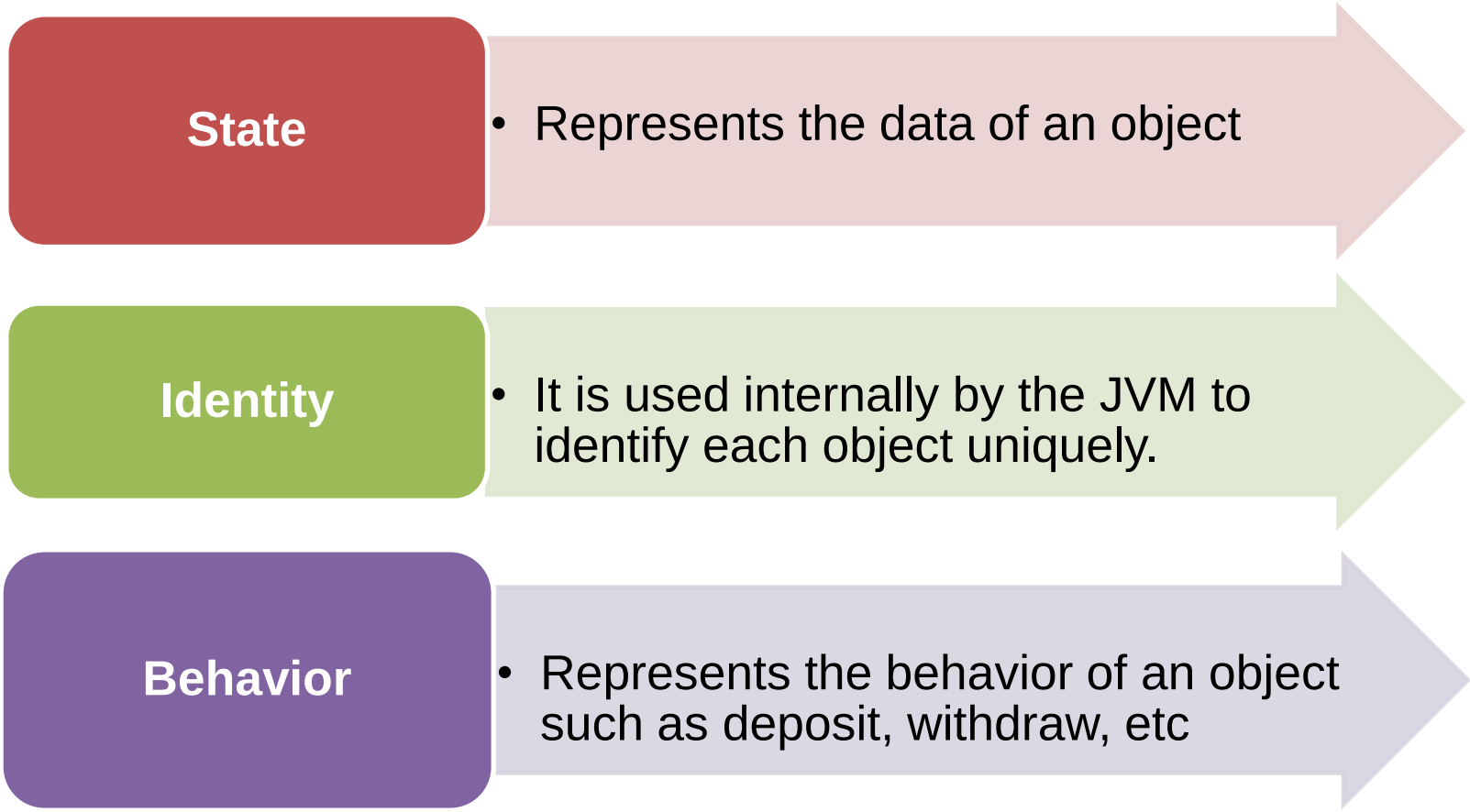
D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Class and Objects



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Characteristics of object



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Video on Class and Object

Source : https://www.youtube.com/watch?v=-MU0_MAi3to

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 8
Method Definition

Class Definition

```
class class-name {
        type instance-variable1;
        type instance-variable2;
        //…….
        type instance-variableN;

        return type methodname1(parameter-list)
                {
                // body of method
                }
         return type methodname2(parameter-list)
                {
                // body of method
                }
}
```

Fields/
Data members/
instance variables

methods/
Member functions

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Class Definition

A class can contain any of the following variable types.

- **Local variables:** Variables defined inside methods, constructors or blocks are called local variables. The variable will be declared and initialized within the method and the variable will be destroyed when the method has completed.
- **Instance variables:** Instance variables are variables within a class but outside any method. These variables are initialized when the class is instantiated. Instance variables can be accessed from inside any method, constructor or blocks of that particular class.
- **Class variables:** Class variables are variables declared within a class, outside any method, with the static keyword.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Adding Variables & Methods

```
class Circle {
        double r;           // radius of circle
        double c ,a;        //area and circumferen

        //Methods to return circumference and area
        void circumference()
                {
                        c=2*3.14*r;
                        System.out.println("Circumference =" +c);
                 }
        void area()
                {
                        double a;
                        a= 3.14 * r * r;
                        System.out.println("Area =" +a);
                }
}
```

Method Definition

Method Definition

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Creating objects of a class

There are three steps when creating an object from a class:

- **Declaration:** A variable declaration with a variable name with an object type.
- **Instantiation:** The 'new' key word is used to create the object.
- **Initialization:** The 'new' keyword is followed by a call to a constructor. This call initializes the new object.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Creating objects of a class

- Declaration of Circle class has created a new data type(user-defined) and we can define variables (objects) of that type.

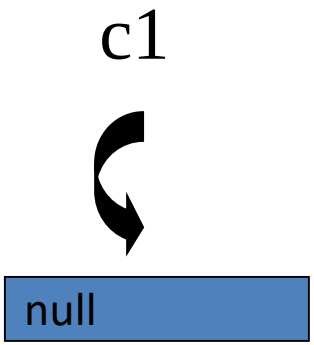
  Syntax:

  class-name object-name;

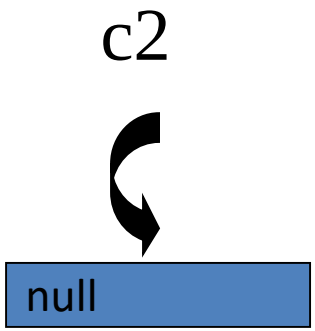
  e.g.

  Circle c1; //declaring object

  Circle c2;



Points to nothing (Null Reference)



Points to nothing (Null Reference)

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Creating objects of a class

- Objects are created dynamically using the *new* keyword.

  Syntax:

  class-name object-name;

  *object-name=*new *class-name();* or

  *class-name object-name=*new *class-name();* //combine 2 statements

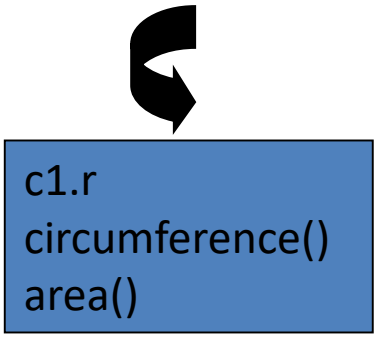
  e.g.

  Circle c1;

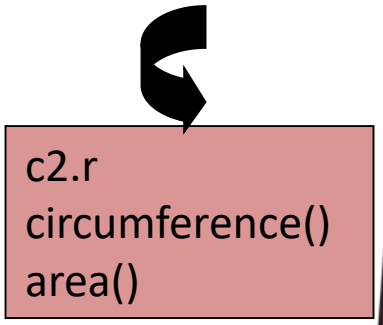
  c1 = new Circle() ; or

  Circle c1 = new Circle(); //combine 2 statements

c1 = new Circle() ;



c2= new Circle() ;



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 9
Access Specifier, Class scope and accessing class members

Class scope and accessing the class members

- In Java, ***scope*** defines where a certain variable or method is accessible in a program.
- Variables can be defined as having one of three types of scope:

1. **Class level scope** (instance variables):
   - any variable declared within a class is accessible by all methods in that class.
   - Depending on its *access modifier* (ie. public or private), it can sometimes be accessed outside the class.
2. **Method level scope** (local variables):
   - any variable declared within a method, arguments included, is NOT accessible outside that method.
3. **Block scope** (loop variables):
   - any variable declared in a loop condition is not accessible after the loop, unless you defined it beforehand.

Access Control/ Access Modifiers

- **Public :** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.
- **Private:** The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
- **Protected:** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
- **Default:** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default..

Accessing the class members

- Syntax is similar to C syntax of accessing data defined in a structure.

```
class Circle
{
     double r;    //instance variable

    void area()
{
    double a;   // local variable
    a= 3.14 * r * r;
    System.out.println("Area =" +a);
}
```

Object-Name.Variable-Name

Object-Name.Method-Name(parameter-list)

```
Circle c1= new Circle();

c1.r = 2.0 // initialize radius
c1.area();  //calling method
```

Accessing the class members

```
class Circle
{
      double r;      // radius of circle
      double c ,a; //area and circumference

      //method definition
      void circumference()
      {        c=2*3.14*r;
      System.out.println("Circumference =" +c);
      }

      void area()
      {        a=3.14 * r * r;
           System.out.println("Area =" +a);
     }
}
```

```
class MyMain
{
        public static void main(String args[])
        {
                Circle c1;  // create reference
                c1 = new Circle(); // create object

                c1.r = 5; // assign value to data
      field

                 // invoking method

                 c1.area();

                  c1.circumference();

       }
}
```

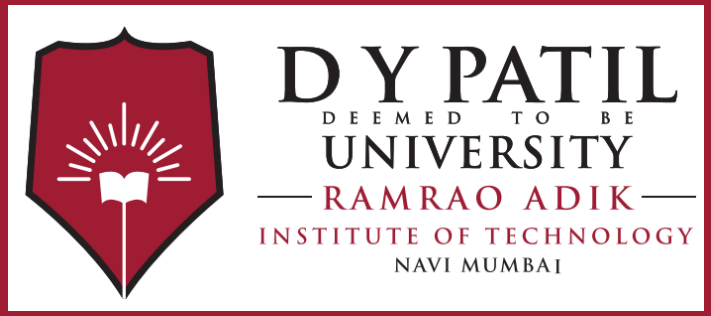
Method Returning Value

```
class Circle
{

      double r;    // radius of circle
      double c ,a; //area and circumference

     //Methods definitions
     double circumference()
     {
              c=2*3.14*r;
              return c ;
      }

      double area()
      {
              a=3.14 * r * r;
              return a;
      }
}
```

```
class MyMain
{
        public static void main(String args[])
        {
              double area, circumference;

               Circle c1;  // create reference
               c1 = new Circle(); // create object
               c1.r = 5; // assign value to data field

                // invoking method
               area=c1.area();
               circumference=c1.circumference();

               System.out.println("Area =" +area);
               System.out.println("Circumference
               =" +circumference);

      }
}
```

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You



Subject Name: OOP

Unit No:2

Unit Name: Classes and Objects

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 10 – Implementation of class and object concept using Java language	3
Lecture 11 – Argument Passing Mechanism	12
Lecture 12 – Method Overloading	18

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 10
Implementation of class and object concept using Java language

Example: Calculate the area of Circle, Triangle, Square and Rectangle

- **Aim:** Using classes and method in java write a program to calculate the area of circle, triangle, square and rectangle.

  - Objects are the basic runtime entities in an object oriented system.
  - A class may be thought of as a ‘data type’ and an object as ‘variable’ of that data type.
  - The data and code of the class are accessed by object of that class.

Example: Calculate the area of Circle, Triangle, Square and Rectangle

- **Syntax:**

```
class Class_Name
    {
        public static void main(String args[])
        {
                    // Statements;
         }
    }
```

Example: Calculate the area of Circle, Triangle, Square and Rectangle

- **Algorithm:**

1. Start.
2. Declare a class named Area.
3. Define a method circle() to calculate area of circle.
4. Define a method triangle() to calculate area of triangle.
5. Define a method square() to calculate area of square.
6. Define a method rectangle() to calculate area of rectangle.
7. Define a class named Calculate.
8. Create object of Area class and invoke methods of circle, triangle, square and rectangle methods inside main() method.
9. Stop.

Example: Calculate the area of Circle, Triangle, Square and Rectangle

- **Program:**

```
import java.util.*;
class Area
{
  void circle(double r)
  {
        double area1 = 3.14 * r * r;
        System.out.println("The area of the circle is :" + area1);
  }
  void triangle( double ht, double bs)
  {
        double area2= ht*bs*0.5;
        System.out.println("Area of traingle is "+area2);
  }
```

Example: Calculate the area of Circle, Triangle, Square and Rectangle

- **Program:**

```
void square(int s)
{
        int area3=s*s;
        System.out.println("Area of square is "+area3);
}
 void rectangle(int lth, int wd)
{
        int area4=lth*wd;
        System.out.println("Area of Rectangle is "+area4);
}
}
```

Example: Calculate the area of Circle, Triangle, Square and Rectangle

- **Program:**

```
class Calculate{
public static void main(String args[])
{
 Area ob=new Area();
 Scanner sc = new Scanner(System.in);
 System.out.println("Enter Radius of Circle:");
double radius = sc.nextDouble();
System.out.println("Enter Base of Triangle:");
double base = sc.nextDouble();
System.out.println("Enter Height of Triangle:");
double height = sc.nextDouble();
System.out.println("Enter Side of Square");
int side = sc.nextInt();
```

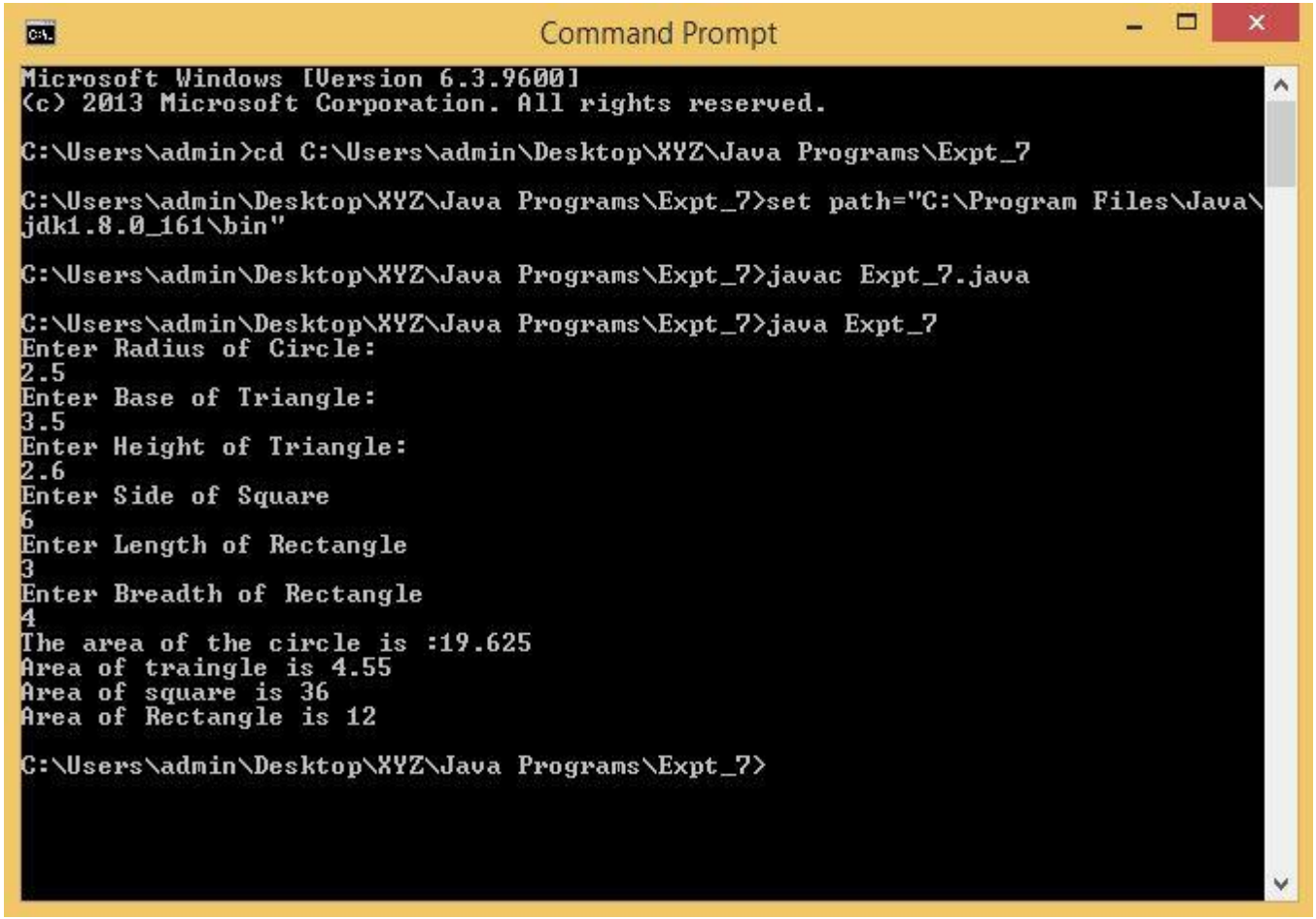
Example: Calculate the area of Circle, Triangle, Square and Rectangle

- **Program:**

```
System.out.println("Enter Length of Rectangle");
int length = sc.nextInt();
System.out.println("Enter Breadth of Rectangle");
int breadth = sc.nextInt();
ob.circle(radius);                    // Area of Circle
ob.triangle(base,height);             // Area of Triangle
ob.square(side);                      // Area of Square
ob.rectangle(length,breadth);   // Area of Rectangle
 }
}
```

Example: Calculate the area of Circle, Triangle, Square and Rectangle

- **Output:**



Lecture No: 11
Argument Passing Mechanism

Methods with Parameters/Arguments

```
class Circle
{
    double c ,a; //area and circumference

    //method definition
    void circumference( int r)
    {
        c=2*3.14*r;
    System.out.println("Circumference ="
    +c);
    }
    void area(int r)
    {
        a=3.14 * r * r;
      System.out.println("Area =" +a);
    }
}
```

```
class MyMain
{
    public static void main(String args[])
    {
        Circle c1;  // creating reference
        c1 = new Circle(); // creating object

         int radius= 5;

         // invoking method

         c1.circumference(radius);

         c1.area(radius);


    }
}
```

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Methods with Parameters/Arguments.... Impact of access specifiers

```
class Test {
int a;                  // default access
public int b;           // public access
private int c;          // private access

// methods to access c
void setc(int i) {      // set c's value
c = i;
}
int getc() {            // get c's value
return c;
}
}
```

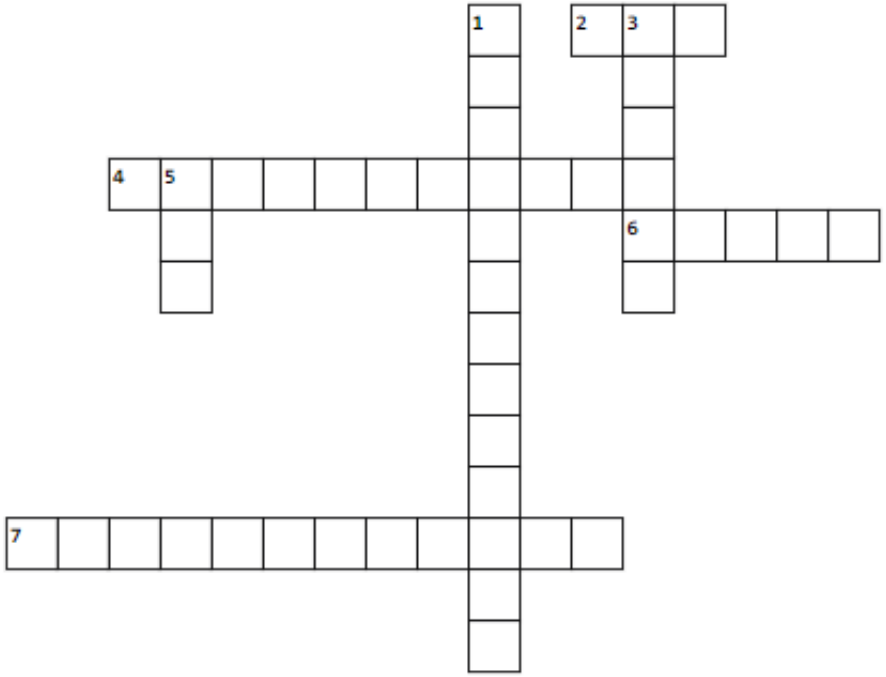
```
class AccessTest {
public static void main(String args[]) {
Test ob = new Test();
ob.a = 10;                  // OK
ob.b = 20;                  // OK
ob.c = 100;                 // Error!
ob.setc(100);               // OK
System.out.println("a, b, and c: " + ob.a + "
    " +ob.b + " " + ob.getc());
}}
```

Excercise

- WAP to create class rectangle and calculate area and perimeter of it.
- WAP to create class Student. Add methods to read and display student information

**Passing Objects as Arguments will be covered after introducing Constructors

Activity



Across

2. operator used to access class members

4. deriving properties from parent class

6. a template representing common behavior

7. multiple forms of same method

Down

1. binding data and methods together

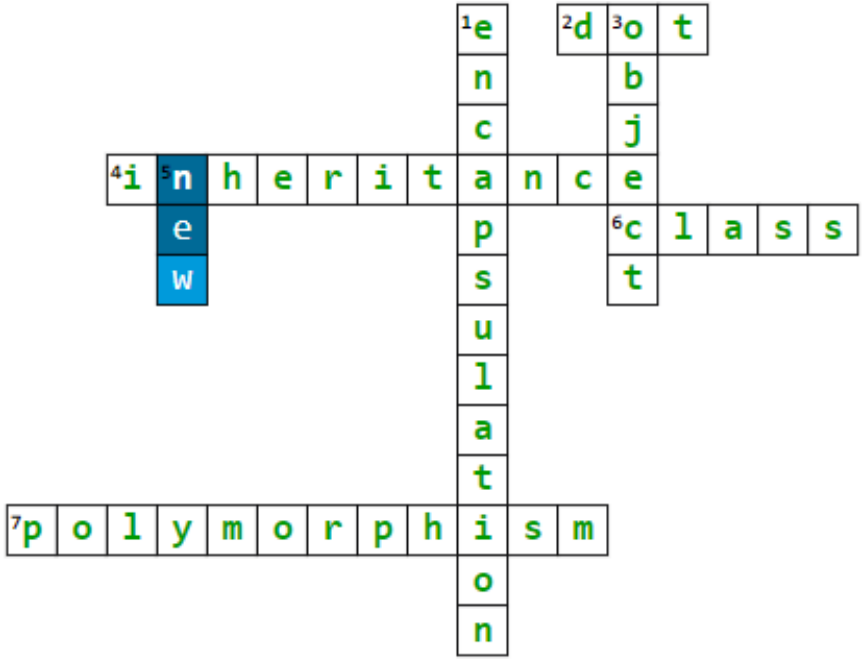
3. an instance of class

5. keyword used to create objects

https://crosswordlabs.com/view/crossword-puzzle-on-oop-concepts

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Activity- Answer Key



Across

✓**2.** operator used to access class members

✓**4.** deriving properties from parent class

✓**6.** a template representing common behavior

✓**7.** multiple forms of same method

Down

✓**1.** binding data and methods together

✓**3.** an instance of class

5. keyword used to create objects

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Lecture No: 12
Method Overloading

‘static’ Members

- Normally a class member must be accessed only with an object.
- When a member is declared static, it can be accessed before any objects of its class are created, and without reference to any object.
- There can be a static method, static variable and a static block

- static keyword is used to declare static variables.

  Eg: static int counter;

- When objects of class are declared, no copy of a static variable is made.
- Instead, all instances of the class share the same static variable.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

‘static’ Members –Example 1

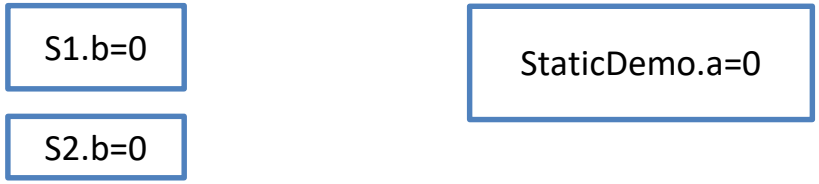
```
class StaticDemo {
static int a; //static variable
int b;   //instance variable
void increment() {
a++;
b++;
}
void display()
{
System.out.println(“a= “+a);
System.out.println(“b= “+b);
}
public static void main(String
   args[]) {
StaticDemo s1=new StaticDemo();
StaticDemo s2=new StaticDemo();
s1.increment();
s2.increment();
s1.display();
}
}
```

```
OUTPUT
a=2
b=1
```

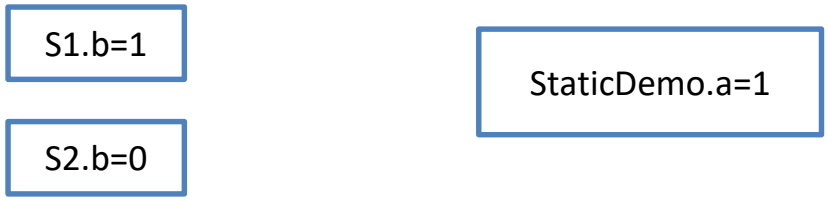

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

‘static’ Members

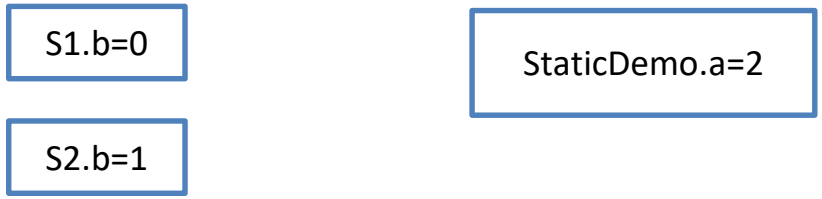
- Initially when object S1 and S2 are created



- When S1.increment() is called



- When S2.increment() is called



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

‘static’ Members

- Static methods can only call other static methods
- Static methods must only access static data
- Static block gets executed exactly once, when the class is first loaded

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

‘static’ Members- Example 2

```
class UseStatic {
static int a = 3;
static int b;
static void meth(int x) {
System.out.println("Static block initialized.");
b = a * 4;
System.out.println("x = " + x);
System.out.println("a = " + a);
System.out.println("b = " + b);
}
public static void main(String args[]) {
meth(42);
}
}
```

```
OUTPUT
Static block initialized.
x = 42
a = 3
b = 12
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

‘static’ Members

- Outside of the class in which they are defined, **static methods and variables can be used** independently of any object.
- To call a **static method and variable from** outside its class, class name is used .
- General form:

  classname.method( );

  Classname.variable1;

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

‘static’ Members

```
class StaticDemo {
static int a = 42;
static int b = 99;
static void callme() {
System.out.println("a = " + a);
}
}
class StaticByName {
public static void main(String
   args[]) {
StaticDemo.callme();
System.out.println("b = " +
   StaticDemo.b);
}
}
```

```
output :
a = 42
b = 99
```

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Case Study on Static members

A XYZ company wants to maintain employee details like EID, name, salary and department. Company will read and display details of employees. When a new employee is joined the company an Employee ID (EID) is automatically incremented and assigned to him/her.

Class: Employee

Data Members: EID, name, salary , department

Methods: getData()
display()

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Method Overloading

- In Java we can define number of methods in a class with the same name.
- Defining two or more methods with the same name in a class is called **method overloading**.
- Method overloading is one of the ways that Java implements polymorphism
- When an overloaded method is invoked, Java uses the type and/or number of arguments to determine which version of the overloaded method to actually call.
- Compile determine which method **to execute automatically.**
- The return type has no effect on the signature of a method.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

What is Signature of a method?

- The name of a method along with the types and sequence of the parameters is called **signature of the method**
- Signature of the method should be unique.

- **Signature of the method includes –**
  1. No of Parameters
  2. Different Parameters
  3. Sequence of the parameters

- **For Example :**

```
int display(int num1,int num2);                    OR
int display(double num1,int num2);       OR
void display(double num1,double num2);
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Method Overloading

```
class OverloadDemo {
void test()
{  System.out.println("No parameters");
}
void test(int a)
{   System.out.println("a: " + a);
}
void test(int a, int b)
 {
System.out.println("a and b: " + a + " "
    + b);
}
double test(double a)
{   System.out.println("double a: " + a);
return a*a;
}
}
```

```
class Overload {
public static void main(String args[])
 {

OverloadDemo ob = new OverloadDemo();
double result;
// call all versions of test()
ob.test();
ob.test(10);
ob.test(10, 20);
result = ob.test(123.2);
System.out.println("Result of ob.test(123.2): "
+ result);
}
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

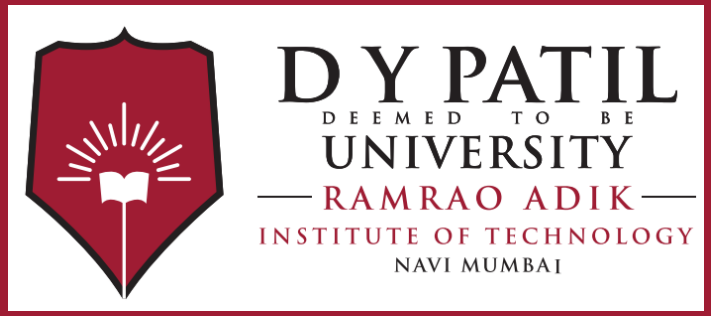
Excercise

- Write a program to overload the **plusMethod()** method to work for both int and double.
- Write a program to overload the **area()** method to calculate area of different shapes.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You



Subject Name: OOP

Unit No:2

Unit Name: Classes and Objects

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 13 – Constructor Overloading	3
Lecture 14 – Recursion	24

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 13
Constructor Overloading

Constructors

- Java allows objects to initialize themselves when they are created through the use of a constructor.
- Constructors play the role of initializing objects
- Constructors are a special kind of methods that are invoked using the new operator when an object is created.
- Constructor in java is used to create the instance of the class.

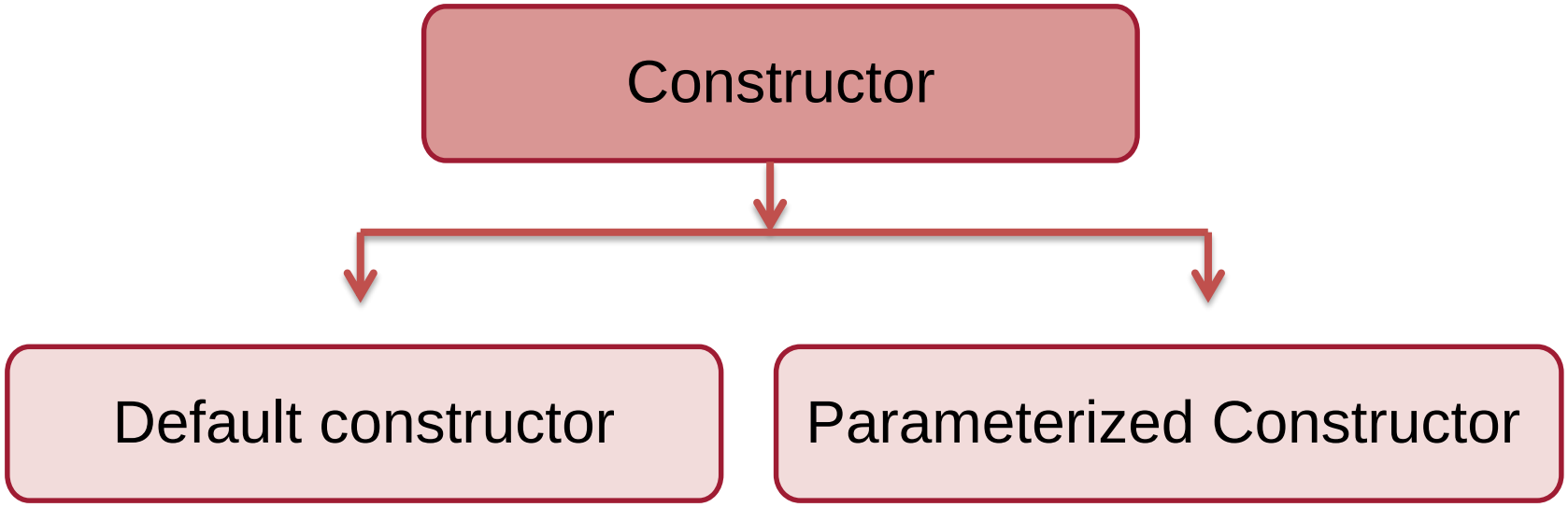
D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Constructors

- Constructors have following properties:
  - Constructors must have the same name as the class itself.
  - Constructors do not have a return type— not even void.
  - The implicit return type is class itself
  - Constructors are invoked when the objects are created.
  - A constructor with no parameters is referred to as a default constructor and constructor with parameters is called as parameterized constructor

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Types of Constructor



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Default Constructor

- If we don’t provide a constructor, then java provides default constructor implementation for us to use.
- Default constructor only role is to initialize the object and return it to the calling code.
- Default constructor is always without argument and provided by java compiler only when there is no existing constructor defined.
- Most of the time we are fine with default constructor itself as other properties can be accessed and initialized through getter setter methods.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Program 1 for Default constructor

```
class Bike1{
//creating a default constructor
Bike1(){System.out.println("Bike is created");}
//main method
public static void main(String args[]){
//calling a default constructor
Bike1 b=new Bike1();
}
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Program 2 for Default constructor

```
class Box {
double width;
double height;
double depth;
// This is the constructor for Box.
Box() {
System.out.println("Constructing Box");
width = 10;
height = 10;
depth = 10;
}
// compute and return volume
double volume() {
return width * height * depth;
}
}
```

```
class BoxDemo1 {
public static void main(String args[]) {

// declare, allocate, and initialize Box objects
Box mybox1 = new Box(); //constructor invoked
Box mybox2 = new Box(); //constructor invoked

double vol;
// get volume of first box
vol = mybox1.volume();
System.out.println("Volume is " + vol);

// get volume of second box
vol = mybox2.volume();
System.out.println("Volume is " + vol);
}
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Parameterized Constructor

- Constructor with arguments is called parameterized constructor.
- The purpose of a parameterized constructor is to assign user-wanted specific values to the instance variables of different objects.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Program 1 for Parameterized constructor

```
class Student4 {
    int id;
    String name;
    Student4 (int i,String n) {     //creating a parameterized constructor
    id = i;
    name = n;
    }
    void display(){System.out.println(id+" "+name);}   //method to display the values

    public static void main(String args[]){
   Student4 s1 = new Student4(111,"Karan");    //creating objects and passing values
   Student4 s2 = new Student4(222,"Aryan");
    s1.display();       //calling method to display the values of object
    s2.display();
   }
}
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Program 2 for Parameterized constructor

```
class Box {
double width;
double height;
double depth;
// This is the constructor for Box.
Box(double w, double h, double d) {
width = w;
height = h;
depth = d;
}
// compute and return volume
double volume() {
return width * height * depth;
}
}
```

```
class BoxDemo2 {
public static void main(String args[]) {
// declare, allocate, and initialize Box objects
Box mybox1 = new Box(10, 20, 15);
Box mybox2 = new Box(3, 6, 9);
double vol;

// get volume of first box
vol = mybox1.volume();
System.out.println("Volume is " + vol);

// get volume of second box
vol = mybox2.volume();
System.out.println("Volume is " + vol);
}
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Constructor Overloading

- Similar to method overloading we can also overload constructor .
- Overloaded constructors have the same name but must differ in the type and/or number of their parameters.
- They are differentiated by the compiler by the number of parameters in the list and their types.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Example-1

```
class Box {
double width;
double height;
double depth;
// constructor used when all dimensions specified
Box(double w, double h, double d) {
width = w;
height = h;
depth = d;
}
// constructor used when no dimensions specified
Box() {
width = -1; // use -1 to indicate
height = -1; // an uninitialized
depth = -1; // box
}
// constructor used when cube is created
Box(double len) {
width = height = depth = len;
}
// compute and return volume
double volume() {
return width * height * depth;
}
}
```

```
class OverloadCons {
public static void main(String args[]) {
// create boxes using the various constructors
Box mybox1 = new Box(10, 20, 15);
Box mybox2 = new Box();
Box mycube = new Box(7);
double vol;
// get volume of first box
vol = mybox1.volume();
System.out.println("Volume of mybox1 is " + vol);
// get volume of second box
vol = mybox2.volume();
System.out.println("Volume of mybox2 is " + vol);
// get volume of cube
vol = mycube.volume();
System.out.println("Volume of mycube is " + vol);
}
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Example-2

```
class Add {
int a,b;
double x,y,z;
Add() {
a=0;
b=0;
x=0.0;
y=0.0;
z=0.0;
}
Add(int m, int n) {
a=m;
b=n;
}
Add(double p, double q, double r) {
x=p;
y=q;
z=r;
}
void addition() {
int res=a+b;
double res1=x+y+z;
System.out.println("Result1="+ res);
System.out.println("Result2="+ res1);
}
}

class OverloadCons {
public static void main(String args[]) {

Add a1=new Add();
a1.addition();

Add a2=new Add(3,4);
a2.addition();

Add a3=new Add(1.2,4.0,3.2);
a3.addition();
}}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Exercise

- WAP to create class rectangle and calculate area and perimeter of it by using constructor.
- WAP to create class Student. Add methods to read and display student information by using constructor.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Passing Object as an Argument

- We can pass class's objects as arguments and also return them from a function the same way we pass and return other variables.
- No special keyword or header file is required to do so.
- To pass an object as an argument we write the object name as the argument while calling the function the same way we do it for other variables.
- **Syntax:**

  function_name(object_name);

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Example: Passing Object as an Argument

```
class Rectangle
{
  int length, width;
  Rectangle(int l, int b)
    { length = l; width = b; }
  void area(Rectangle r1)
    {
      int areaOfRectangle = r1.length * r1.width;
      System.out.println("Area of Rectangle : " + areaOfRectangle);
    }
 }
class RectangleDemo
 {
       public static void main(String args[])
           {
                Rectangle r1 = new Rectangle(10, 20);
                r1.area(r1);
           }
 }
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Explanation

- We can pass Object of any class as parameter to a method in java.
- We can access the instance variables of the object passed inside the called method.

  area = r1.length * r1.width
- It is good practice to initialize instance variables of an object before passing object as parameter to method otherwise it will take default initial values.
- **Output :**
  Area of Rectangle = 200

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Example: Passing Instance Variable as an Argument

```
class Rectangle
 {
    int length,width;
    void area(int length, int width)
     {
        int areaOfRectangle = length * width;
        System.out.println("Area of Rectangle : " + areaOfRectangle);
     }
  }
class RectangleDemo
  {
    public static void main(String args[])
     {
        Rectangle r1 = new Rectangle();
        Rectangle r2 = new Rectangle();
        r1.length = 20;
        r1.width = 10;
        r2.area(r1.length, r1.width);
     }
   }
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Explanation

- We can pass Instance variable of any class as parameter to a method in java.
- We can access the instance variables of the object passed inside the called method.

  area = length * width
- **Output :**
  Area of Rectangle = 200

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Example: Returning object as an Argument

```
class Rectangle
 {
  int length,breadth;
  Rectangle(int l,int b)
   {
    length = l;
    breadth = b;
   }
  Rectangle getRectangleObject()
   {
    Rectangle rect = new rectangle(10,20);
    return rect;
   }
 }
```

```
class RetOb
 {
  public static void main(String args[])
   {
    Rectangle ob1 = new Rectangle(40,50);
    Rectangle ob2;
    ob2 = ob1.getRectangleObject();
    System.out.println("ob1.length : " +ob1.length);
    System.out.println("ob1.breadth: " +ob1.breadth);
    System.out.println("ob2.length : " +ob2.length);
    System.out.println("ob2.breadth: " +ob2.breadth);
   }
 }
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Explanation

- In the program we have called a method getRectangleObject() and the method creates object of class from which it has been called.
- All objects are dynamically allocated using new, you don’t need to worry about an object going out-of-scope because the method in which it was created terminates.
- The object will continue to exist as long as there is a reference to it somewhere in your program. When there are no references to it, the object will be reclaimed the next time garbage collection takes place.
- **Output :**

  ob1.length : 40
  ob1.breadth: 50
  ob2.length : 10
  ob2.breadth: 20

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Lecture No: 14
Recursion

Recursion

- Recursion is the technique of making a function call itself.
- This technique provides a way to break complicated problems down into simple problems which are easier to solve.

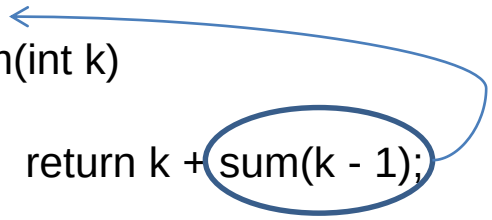
```
returntype methodname(){
//code to be executed
methodname();//calling same method
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Recursion Example

- Using recursion add a range of numbers together by breaking it down into the simple task of adding two numbers.

```
public class Main
{       public static void main(String[] args)
        {
                int result = sum(10);
                System.out.println(result);
        }

        public static int sum(int k)
        {       if (k > 0)
                {       return k + sum(k - 1);
                }
                else
                {       return 0;
                }
        }
}
```



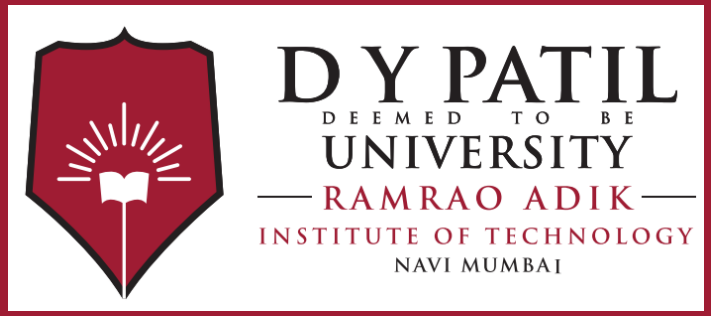
```
10 + sum(9)
10 + ( 9 + sum(8) )
10 + ( 9 + ( 8 + sum(7) ) )
...
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + sum(0)
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 + 0
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Excercise

- Write a program to print Factorial of a number using recursion
- Write a program to print Fibonacci series using recursion.
- Write a program to count the digits of a given number using recursion.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI



Subject Name: OOP

Unit No:3

Unit Name: Arrays and String Class

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 15 – Definition of Array 30

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 12
Definition of Arrays

Arrays

- Is a collection of **similar type of elements**
- Syntax: **1-D array**

```
        data_type array_name[]=new data_type[size];
        data_type[] array_name=new data_type[size];
Or
        data_type array_name[];
    array_name=new data_type[size];
```

Eg:-

```
        int A[]=new int[10];
        int[] A=new int[10];
Or
        int B[];
        B=new int[10];
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

1-D array initialization

Data_type array_name[]={list of values};

Eg:-
int A[]={10,20,30};

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

WAP to calculate sum and average of array elements

```
import java.util.*;
class ArrayAvg
{public static void main(String args[])
{     int n,i,a[];
      float avg,sum=0.0f;
Scanner sc=new Scanner(System.in);
System.out.println("Enter the number of
      elements:");
      n=sc.nextInt();
      a=new int[n];
      for(i=0;i<=n-1;i++)
      {                    a[i]=sc.nextInt();
      } for(i=0;i<=n-1;i++)
      {                    sum=sum+a[i];
      }
      avg=sum/n;
System.out.println("The sum is
      "+sum+" and average is "+avg);
}
}
```

```
Output
Enter the number of elements: 5
Enter a value:10
Enter a value:20
Enter a value:30
Enter a value:40
Enter a value:50
The sum is 150 and average is 30.0
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Practice Program

- WAP to search an element in array
- WAP to sort array elements in ascending order

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

2-D array

- **Syntax**

data_type array_name[][]=new data_type[row][col];

data_type[][] array_name=new data_type[row][col];

Or

```
        data_type array_name[][];
    array_name=new data_type[row][col];
```

Eg:-

```
        int A[][]=new int[3][3];
        int[][] A=new int[3][3];
```

Or

```
        int B[][];
        B=new int[3][3];
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

2-D array initialization

Data_type array_name[][]={{list of values in row1},
{list of values in row2}};

Eg:-
int A[][]={{10,20},{30,40}};

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

WAP to display addition of two matrices of size mxn

```
import java.util.*;
class MatAdd
{public static void main (String args[])
{
int A[][],B[][],C[][];
int i,j,m,n;
Scanner sc=new Scanner(System.in);
System.out.println("Enter size of
    matrices");
m=sc.nextInt();
n=sc.nextInt();
A[][]=new int[m][n];
B[][]=new int[m][n];
C[][]=new int[m][n];
```

```
System.out.println("ENTER ARRAY
    ELEMENTS FOR A:");
for(i=0;i<m;i++)
    {       for(j=0;j<n;j++)
            {A[i][j]=sc.nextInt();
    }}
System.out.println("ENTER ARRAY
    ELEMENTS FOR B:");
for(i=0;i<m;i++)
    {       for(j=0;j<n;j++)
            { B[i][j]=sc.nextInt();
            }
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

WAP to display addition of two matrices of size mxn

```
for(i=0;i<m;i++)
    {
            for(j=0;j<n;j++)
            {
            C[i][j]=A[i][j]+B[i][j];
            }
}
System.out.println("Matrix addition
    is\n");
```

```
for(i=0;i<m;i++)
    {
            for(j=0;j<n;j++)
            {
            System.out.print(C[i][j]+" ");
            }
System.out.println();
    }
}
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

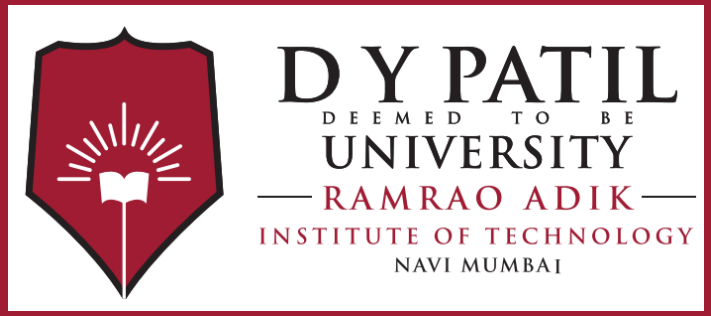
Practice Program

- WAP to perform transpose of a matrix
- WAP to perform addition of column elements of a matrix
- WAP to perform addition of diagonal elements of a matrix
- WAP to perform matrix multiplication

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You



Subject Name: OOP

Unit No:3

Unit Name: Arrays and String Class

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 16 – String Class-I	3
Lecture 17 – String Class-II	10
Lecture 18 – String Buffer Class	16

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 16
String Class-I

String

- Is a class in JAVA
- implements strings as objects of String class.
- Can not modify characters of the original string, hence its immutable.
- each time you need an altered version of an existing string, a new String object is created that contains the modifications

Eg:-

String s1=“HELLO”;

https://www.youtube.com/watch?v=xcEe9GYEcA4

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

String constructors

- **Syntax:**

  String s2=new String(); //creates empty string

  String s3=new String(char A[]);

Eg:- char A[]={‘n’,’e’,’r’,’u’,’l’};

String city=new String(A);

String s4=new String(char A[], int startIndex, int numChars)

Eg:- char A[]={‘n’,’e’,’r’,’u’,’l’};

String city=new String(A,0,3);

String s5=new String(String s4)

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Methods of String class

Function	Explanation
int length()	To find the number of characters from a string
char charAt(int)	To extract a single character from a String
int indexOf(int *ch)*	To search for the first occurrence of a character,
int lastIndexOf(int *ch)*	To search for the last occurrence of a character
void getChars(int sourceStart, int sourceEnd, char target[ ], int targetStart)	to extract more than one character at a time
Char[] toCharArray()	to convert all the characters in a String object into a character array

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Methods of String class

Function	Explanation
boolean equals(Object str)	To compare two strings for equality, It returns true if the strings contain the same characters in the same order, and false otherwise.
boolean equalsIgnoreCase(String str)	To perform a comparison that ignores case differences, When it compares two strings, it considers A-Z to be the same as a-z.
String substring(int *startIndex)*	returns a copy of the substring that begins at *startIndex and runs to the end of the invoking string*
String substring(int *startIndex, int endIndex)*	string returned contains all the characters from the beginning index, up to, but not including, the ending index.

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Methods of String class

Function	Explanation
int compareTo(String str)	For sorting applications, you need to know which is less than, equal to, or greater than the next. A string is less than another if it comes before the other in dictionary order. A string is greater than another if it comes after the other in dictionary order. **Less than zero :**The invoking string is less than str. **Greater than zero :**The invoking string is greater than str. **Zero :** The two strings are equal.
String concat(String str)	creates a new object that contains the invoking string with the contents of **str** appended to the end

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Methods of String class

Function	Explanation
String replace(char original, char replacement)	replaces all occurrences of one character in the invoking string with another character
String trim( )	returns a copy of the invoking string from which any leading and trailing whitespace has been removed.
String toLowerCase()	return a String object that contains the lowercase equivalent of the invoking String
String toUpperCase()	return a String object that contains the uppercase equivalent of the invoking String

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 17
String Class-II

Demo of String Methods

```
class StringDemo{
public static void main(String args[])
{
String s0="JAVA PROGRAM    ";
String s1="Java Program    ";
String s3="    Welcome To ";
System.out.println("String s0:"+s0);
System.out.println("String s1:"+s1);
System.out.println("String s3:"+s3);
```

```
//TO FIND LENGTH OF A STRING
System.out.println("Length of String
    s1:"+s1.length());

//TO FIND CHARACTER AT SPECIFIC
    INDEX
System.out.println("Character at Index 2
    in String s1:"+s1.charAt(2));

//TO FIND INDEX OF FIRST
    CHARACTER OCCURENCE
System.out.println("First occurrence of
    Character 'a' in String
    s1:"+s1.indexOf("a"));
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Demo of String Methods

```
//TO FIND INDEX OF LAST CHARACTER OCCURENCE
System.out.println("Last occurrence of Character 'a' in String s1:"+s1.lastIndexOf("a"));

//TO FIND SUBSTRING FROM SPECIFIC INDEX
int start=5,end=8;
char A[]=new char[end-start];
s1.getChars(start,end,A,0);
String s2=new String(A);
System.out.println("String s2:"+s2);
```

```
//TO CONVERT STRING TO CHARACTER ARRAY
A=s1.toCharArray();
System.out.print("String Conversion into Array: ");
for(int i=0;i<A.length;i++)
{
System.out.print(A[i]);
}
System.out.println();
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Demo of String Methods

```
//TO FIND OUT SUBSTRING
System.out.println("Sub-String of String s1 from Index 5:"+s1.substring(5));
System.out.println("Sub-String of String s1 from Index 9 to Index 12:"+s1.substring(9,12));

//TO COMPARE TWO STRINGS FOR EQUALITY
if(s1.equals(s2))
{System.out.println("Compare String s1 and s2 for Equality: "+s1+" == "+s2);
}
else
{System.out.println("Compare String s1 and s2 for Equality: "+s1+" != "+s2);
}
```

```
//TO COMPARE TWO STRINGS WITH IGNORE CASE
if(s1.equalsIgnoreCase(s0))
{ System.out.println("Compare String s1 and s0 with Ignore Case: "+s1+" == "+s0);
}
else
{ System.out.println("Compare String s1 and s0 with Ignore Case:"+s1+" != "+s0);
}
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Demo of String Methods

```
//TO COMPARE TWO STRINGS
int a=s2.compareTo(s1);
System.out.println("Comparison Result
    of String s2 and s1:"+a);

//TO CONCAT TWO STRINGS
System.out.println("Concatenate String
    s3 and s1:"+s3.concat(s1));

//TO REMOVE LEADING AND
    TRAILING WHITE SPACES
System.out.println("Remove Leading
    and Trailing Spaces of String
    s3:"+s3.trim());
```

```
//TO REPLACE ONE CHARACTER BY
    OTHER
System.out.println("Replace character A
    with a in String
    s0:"+s0.replace('A','a'));

//CONVERT TO LOWER CASE
System.out.println("Convert String s1
    into Lower case:
    "+s1.toLowerCase());

//CONVERT TO UPPER CASE
System.out.println("Convert String s1
    into upper case:
    "+s1.toUpperCase());
}
}
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Demo of String Methods

OUTPUT

```
String s0:JAVA PROGRAM
String s1:Java Program
String s3:    Welcome To
Length of String s1:16
Character at Index 2 in String s1:v
First occurrence of Character 'a' in String s1:1
Last occurrence of Character 'a' in String s1:10
String s2:Pro
String Conversion into Array: Java Program
Sub-String of String s1 from Index 5:Program
Sub-String of String s1 from Index 9 to Index 12:ram
Compare String s1 and s2 for Equality: Java Program     != Pro
Compare String s1 and s0 with Ignore Case: Java Program     == JAVA PROGRAM
Comparison Result of String s2 and s1:6
Concatenate String s3 and s1:    Welcome To Java Program
Remove Leading and Trailing Spaces of String s3:Welcome To
Replace character A with a in String s0:JaVa PROGRaM
Convert String s1 into Lower case: java program
Convert String s1 into upper case: JAVA PROGRAM
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 18
String Buffer class

StringBuffer

- represents growable and writeable character sequences
- may have characters and substrings inserted in the middle or appended to the end
- has more characters pre-allocated than are actually needed, to allow room for growth

- https://www.youtube.com/watch?v=_bX7jslx6rY

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

StringBuffer Constructors

- StringBuffer( )

Eg:- StingBuffer s1=new StringBuffer();

- StringBuffer(int size)

Eg:- StingBuffer s1=new StringBuffer(15);

- StringBuffer(String str)

Eg:- String s=new String(“java”);

StingBuffer s1=new StringBuffer(s);

Note- allocates room for 16 additional characters when no specific buffer length is requested

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

StringBuffer class methods

Functions	Explanation
int length()	Returns the current length of the StringBuffer class
int capacity()	Returns total allocated capacity of StringBuffer
void ensureCapacity(int *capacity)*	to set the size of the buffer
void setCharAt(int where, char ch)	where specifies the index of the character being set, and ch specifies the new value of that character.
StringBuffer append(String str)	concatenates the string representation of any other type of data to the end of the invoking StringBuffer object

UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

StringBuffer class methods

Functions	Explanation
StringBuffer insert(int index, String str)	index specifies the index at which point the string will be inserted into the invoking StringBuffer object
StringBuffer reverse( )	reverse the characters within a StringBuffer object
StringBuffer delete(int startIndex, int endIndex)	deletes a sequence of characters from the invoking object
StringBuffer deleteCharAt(int loc)	deletes the character at the index specified by loc
StringBuffer replace(int startIndex, int endIndex, String str)	The substring being replaced is specified by the indexes startIndex and endIndex. Thus, the substring at startIndex through endIndex–1 is replaced

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Demo of StringBuffer Methods

```
class StringBufferDemo
{   public static void main(String
    args[])
{   StringBuffer s1=new
    StringBuffer(15);
String s=new String("java");
StringBuffer s2=new StringBuffer(s);
System.out.println("String s1: "+s1);
System.out.println("String s2: "+s2);

//TO DISPLAY LENGTH
System.out.println("Length of String
    s1: "+s1.length());
System.out.println("Length of String
    s2: "+s2.length());
```

```
//TO DISPLAY CAPACITY
System.out.println("Capacity of String
    s1: "+s1.capacity());
System.out.println("Capacity of String
    s2: "+s2.capacity());

//TO SET BUFFER CAPACITY
s1.ensureCapacity(20);
System.out.println("Set Buffer
    Capacity of String s1:
    "+s1.capacity());
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Demo of StringBuffer Methods

```
//TO SET/REPLACE CHARACTER AT GIVEN POSITION
s2.setCharAt(1,'A');
System.out.println("Replace character at position 1 with A in String s2: "+s2);

//TO APPEND STRING
s2.append(" program");
System.out.println("Append 'program' at the end of String s2: "+s2);
```

```
//TO INSERT STRING
s1.insert(0,"hello");
System.out.println("Insert 'hello' at the beginning of String s1: "+s1);

//TO REVERSE A STRING
s1.reverse();
System.out.println("Reverse the String s1: "+s1);
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Demo of StringBuffer Methods

```
//TO DELETE A SUBSTRING
s2.delete(10,12);
System.out.println("Delete a substring
    from position 10 to position 12 in
    String s2: "+s2);

//TO DELETE CHARACTER AT
    SPECIFIC INDEX'
s2.deleteCharAt(1);
System.out.println("Delete a character
    at position 1 in String s2: "+s2);
```

```
//TO REPLACE SUBSTRING
s2.replace(0,4,"java hi ");
System.out.println("Replace a
    substring from position 0 to position
    4 in String s2 with 'java hi': "+s2);
}
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Demo of StringBuffer Methods

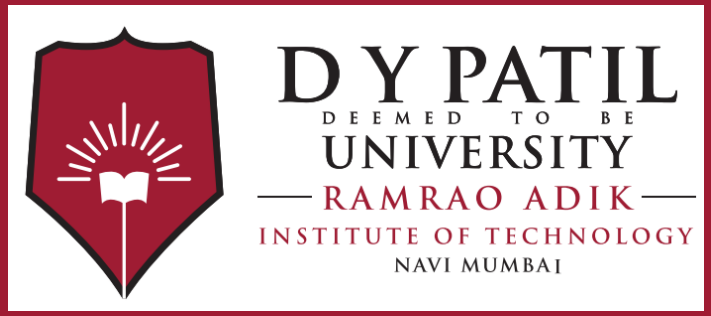
OUTPUT

```
String s1:
String s2: java
Length of String s1: 0
Length of String s2: 4
Capacity of String s1: 15
Capacity of String s2: 20
Set Buffer Capacity of String s1: 32
Replace character at position 1 with A in String s2: jAva
Append 'program' at the end of String s2: jAva program
Insert 'hello' at the beginning of String s1: hello
Reverse the String s1: olleh
Delete a substring from position 10 to position 12 in String s2: jAva progr
Delete a character at position 1 in String s2: jva progr
Replace a substring from position 0 to position 4 in String s2 with 'java hi': java hi progr
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You



Subject Name: OOP

Unit No:4

Unit Name: Inheritance and Packages

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 19 – Importance of Inheritance, Different Types of Inheritance	3
Lecture 20 – Super keyword	18
Lecture 21 – Abstract class and Abstract method	27

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 19
Importance of Inheritance, Different Types of Inheritance

Importance of Inheritance

- Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object.
- The idea behind inheritance in Java is that you can create new classes that are built upon existing classes.
- When you inherit from an existing class, you can reuse methods and fields of the parent class.
- Moreover, you can add new methods and fields in your current class also

Why to use inheritance in java

- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

Defining a Subclass

- Syntax :

```
class <subclass name> extends <superclass name>
{
 variable declarations;
 method declarations;
}
```

- The extends keyword indicates that you are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.
- Extends keyword signifies that properties of the super class are extended to sub class
- Sub class will not inherit private members of super class

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Program 1

```
class Employee{
 float salary=40000;
}
class Programmer extends Employee{
 int bonus=10000;
public static void main(String args[]){
 Programmer p=new Programmer();
  System.out.println("Programmer salary is:"+p.salary);
  System.out.println("Bonus of Programmer is:"+p.bonus);
}
}
```

Output

Programmer salary is:40000.0

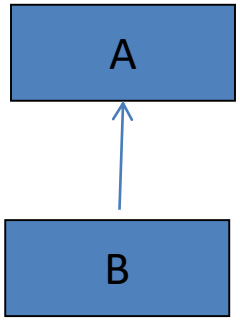
Bonus of programmer is:10000

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Inheritance and it's Types

- A class can inherit the properties of another class and add its own modification.
- The class whose properties are extended is known as super or base or parent class.
- The class which extends the properties of super class is known as sub or derived or child class
- To declare inherited classes 'extends' keyword is used

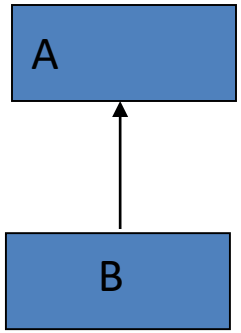
- class B extends A { ….. }
- A is super class
- B is sub class



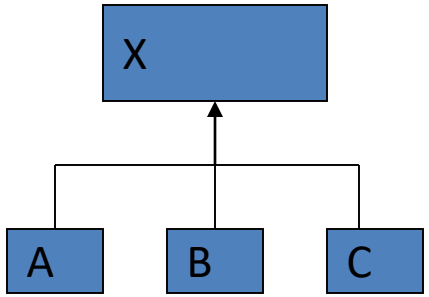
D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Types of Inheritance

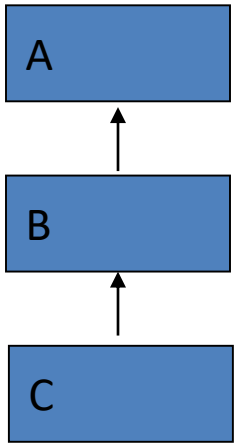
Single Inheritance



Hierarchical Inheritance



MultiLevel Inheritance



D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Program 2:Single Inheritance

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class TestInheritance{
public static void main(String args[]){
Dog d=new Dog();
d.bark();
d.eat();
}}
```

OUTPUT

```
barking...
eating...
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Explanation (Single Inheritance)

- When a class inherits another class, it is known as a *single inheritance*. In the example given below, Dog class inherits the Animal class, so there is the single inheritance.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Program 3: Multilevel Inheritance

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```

OUTPUT:

```
weeping...
barking...
eating...
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Explanation (Multilevel Inheritance)

- When there is a chain of inheritance, it is known as multilevel inheritance. As you can see in the example given below, BabyDog class inherits the Dog class which again inherits the Animal class, so there is a multilevel inheritance.

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Program 4: Hierarchical Inheritance

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
public static void main(String args[]){
Cat c=new Cat();
c.meow();
c.eat();
}}
```

OUTPUT

```
meowing...
eating...
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Explanation (Hierarchical Inheritance)

- When two or more classes inherits a single class, it is known as hierarchical inheritance. In the example given below, Dog and Cat classes inherits the Animal class, so there is hierarchical inheritance.

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Program 4

```
class A{
void msg(){System.out.println("Hello");}
}
class B{
void msg(){System.out.println("Welcome");}
}
class C extends A,B{//suppose if it were

 public static void main(String args[]){
   C obj=new C();
   obj.msg();//Now which msg() method would be invoked?
}
}
```

OUTPUT

Compile Time Error

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Explanation (Multiple Inheritance)

- Multiple inheritance is not supported in java.
- Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class.
- Since compile-time errors are better than runtime errors, Java renders compile-time error if you inherit 2 classes. So whether you have same method or different, there will be compile time error.

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Lecture No: 20
Super keyword

Using super

super keyword is used :

1. To call the superclass' constructor.
2. To access a member of the superclass that has been hidden by a member of a subclass.

first use of super

number of parameters in subclass constructor is equal to sum of number of data members in super class and its own, if all are distinct.

super must be a very first statement in subclass constructor.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Super Keyword

- The super keyword in Java is a reference variable which is used to refer immediate parent class object.
- Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

Usage of Java super Keyword

1. super can be used to refer immediate parent class instance variable.
2. super can be used to invoke immediate parent class method.
3. super() can be used to invoke immediate parent class constructor.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

1.super can be used to refer immediate parent class instance variable.

- We can use super keyword to access the data member or field of parent class. It is used if parent class and child class have same fields.

- In the given example, Animal and Dog both classes have a common property color. If we print color property, it will print the color of current class by default. To access the parent property, we need to use super keyword.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

1.super can be used to refer immediate parent class instance variable.

```
class Animal{
String color="white";
}
class Dog extends Animal{
String color="black";
void printColor(){
System.out.println(color);//prints color of Dog class
System.out.println(super.color);//prints color of Animal class
}
}
class TestSuper1{
public static void main(String args[]){
Dog d=new Dog();
d.printColor();
}}
```

OUTPUT:

```
black
white
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

2. Super can be used to invoke parent class method

- The super keyword can also be used to invoke parent class method. It should be used if subclass contains the same method as parent class. In other words, it is used if method is overridden.
- In the given example Animal and Dog both classes have eat() method if we call eat() method from Dog class, it will call the eat() method of Dog class by default because priority is given to local.
- To call the parent class method, we need to use super keyword.

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

2. Super can be used to invoke parent class method

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void eat(){System.out.println("eating bread...");}
void bark(){System.out.println("barking...");}
void work(){
super.eat();
bark();
}
}
class TestSuper2{
public static void main(String args[]){
Dog d=new Dog();
d.work();
}}
```

OUTPUT

```
eating...
barking...
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

3. super() can be used to invoke immediate parent class constructor.

- super() is added in each class constructor automatically by compiler if there is no super()

- As we know well that default constructor is provided by compiler automatically if there is no constructor. But, it also adds super() as the first statement.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

3. super() can be used to invoke immediate parent class constructor.

```
class Animal{
Animal(){System.out.println("animal is created");}
}
class Dog extends Animal{
Dog(){
super();
System.out.println("dog is created");
}
}
class TestSuper3{
public static void main(String args[]){
Dog d=new Dog();
}}
```

OUTPUT

```
animal is created
dog is created
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 21
Abstract class and Abstract method

Abstract Class in Java

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It needs to be extended and its method implemented. It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

```
public abstract class Product
                    {
                    // contents
                    }
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Abstract Method in Java

- A method which is declared as abstract and does not have implementation is known as an abstract method.

abstract void printStatus(); //no method body and abstract

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Example 1

```
abstract class Bike{
abstract void run();
}

class Honda4 extends Bike{
void run(){System.out.println("running safely");}

public static void main(String args[]){
Bike obj = new Honda4();
obj.run();
}
}
```

OUTPUT

running safely

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Example 2

```
abstract class Shape{
abstract void draw();
}

class Rectangle extends Shape{
void draw(){System.out.println("drawing rectangle");}
}

class Circle1 extends Shape{
void draw(){System.out.println("drawing circle");}
}

class TestAbstraction1{
public static void main(String args[]){
Shape s=new Circle1();
s.draw();
}
}
```

OUTPUT

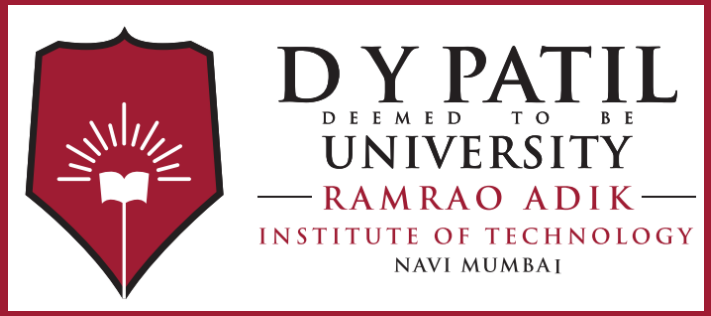
drawing circle

- In this example, Shape is the abstract class, and its implementation is provided by the Rectangle and Circle class
- In this example, if you create the instance of Rectangle class, draw() method of Rectangle class will be invoked.

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You



Subject Name: OOP

Unit No:4

Unit Name: Inheritance and Packages

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 22 – Final keyword	3
Lecture 23 – Multiple inheritance using interface	11
Lecture 24 – Method Overriding	23

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 22
Final keyword

Final Keyword

- The final keyword in java is used to restrict the user. The java final keyword can be used in many context. Final can be:

1) variable
2) method
3) Class

- The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable. It can be initialized in the constructor only.
- The blank final variable can be static also which will be initialized in the static block only.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Final keyword

- Declaring constants

final int MAX=100;

- Preventing method overriding

final void show (final int x)

- Preventing inheritance

final class Demo {}

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Final keyword

```
class A {
final float PI=3.14f;
final void show() {
……….
}
}
final class B extends A {
void show() { //error
……..
}
}
class C extends B{ //error
{
……
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Final variable

- When a variable is declared with final keyword, its value can't be modified, essentially, a constant.
- This also means that you must initialize a final variable.
- A final variable can only be initialized once

E.g.

```
class ABC
{       // a final variable
        // direct initialize
  final int THRESHOLD = 5;
      // a blank final variable
 final int CAPACITY;
    // initializing CAPACITY
  {
  CAPACITY = 25;
  }
}
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Final Method

- Methods can be qualified with the final modifier
- Final methods cannot be overridden.
- This can be useful for security purposes.

E.g.

```
class A
{
 final void m1()
{
 System.out.println("This is a final method.");
 }
 }
class B extends A
 {
 void m1() { // COMPILE-ERROR! Can't override. System.out.println("Illegal!");
 } }
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Final Class

- Classes can be qualified with the final modifier
- The class cannot be extended (inheritated).
- This can be used to improve performance. Because there can be no subclasses, there will be no polymorphism

E.g.

```
final class A
{
        // methods and fields
 }
// The following class is illegal.
class B extends A
{
         // COMPILE-ERROR! Can't subclass A
 }
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Final Class Example

```
final class XYZ
        {
         }
        class ABC extends XYZ
        {
                 void demo()
                {
                                System.out.println("My Method");
                }
                 public static void main(String args[])
                {
                                 ABC obj= new ABC();
                                obj.demo();
                }
         }
```

Output:

The type ABC cannot subclass the final class XYZ

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Lecture No: 23
Multiple inheritance using interface

Interfaces

- Java does not allow such multiple inheritance directly. So one class can not extend two super classes
- The reason is to just keep the language simple.
- Java allows multiple inheritance through the use of interface concept.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Multiple Inheritance

```
class A
{
 int i=1;
}
class B
{
int i=2;
}
class C extends A,B
{
//two copies of i are available
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Interfaces

- Interfaces are syntactically similar to classes, but they lack instance variables, and their methods are declared without any body.
- Interfaces defines only abstract methods and final variables.
- Once it is defined, any number of classes can implement an interface
- It is responsibility of class which implements the interface to define the abstract methods
- Can’t create object for interface, but we can have reference for it.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Defining an Interface

- *interface* keyword is used to define interface.

```
interface Interfacename {
  return-type method-name1(parameter-list);
  return-type method-name2(parameter-list);
  final type varname1 = value;
  final type varname2 = value;
……..
                              }
```

- Methods end with a semicolon after the parameter list i.e. they are abstract methods
- Variables are implicitly final and static.

D Y PATIL RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Implementing Interfaces

- To implement an interface, the **implements** keyword is used

- General form

```
class classname implements interfacename1, interfacename2….
{
    // class-body
}
                    or
class classname extends classname implements interfacename1,
    interfacename2….
{
    // class-body
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

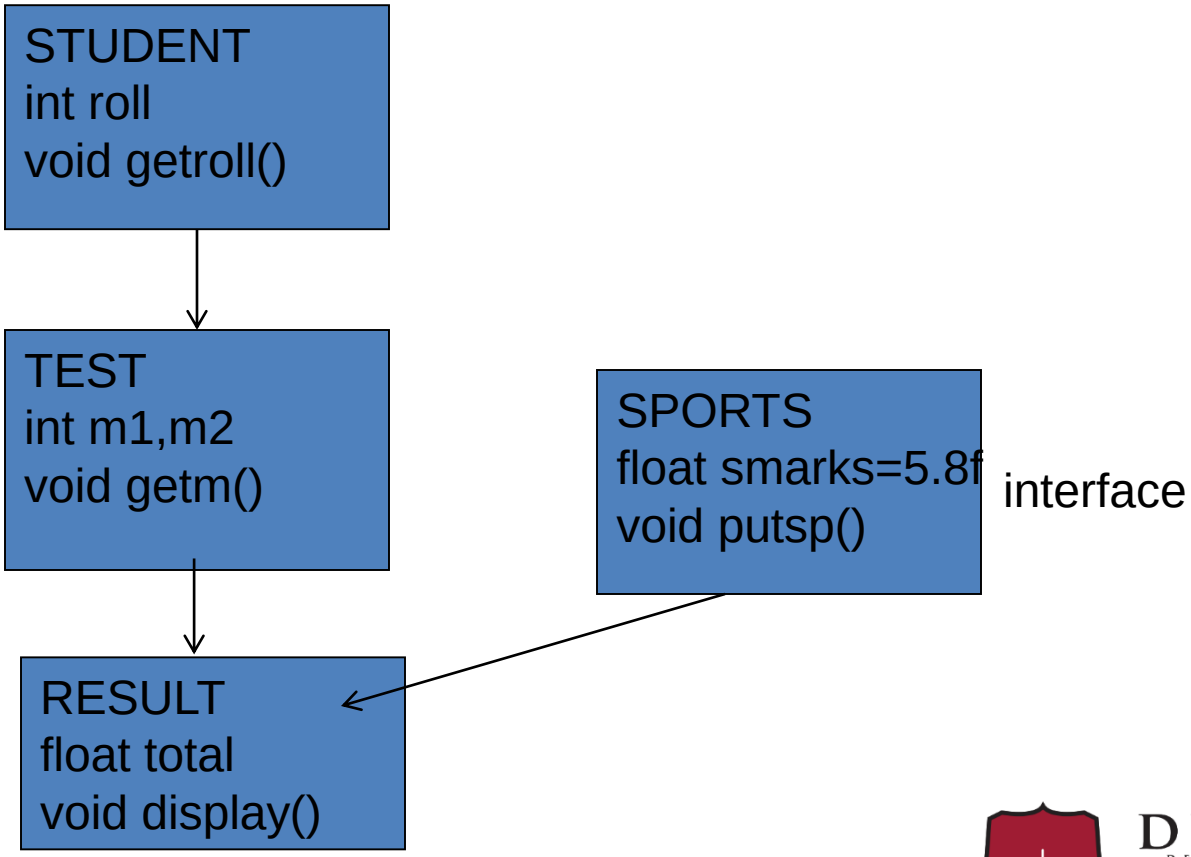
Example of Interface

```
interface A {
    void show();
    int a=100;
}

class B implements A {
public void show() {
a=20;
System.out.println("B
    implements method");
}
void display()
{
System.out.println("B's own
    method");
}
}
```

```
public class InterfaceDemo {
public static void main(String args[]) {
B b1 = new B();
b1.display();
b1.show();      //error
A a1 = new A(); //error
A a2;
a2=b1;
a2.show();
}
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Write program for following inheritance hierarchy



D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Example of Interface

```
import java.util.*;

class Student{
int roll;
Scanner sc =new Scanner(System.in);
void getroll()
{
System.out.println("Enter roll no");
roll=sc.nextInt();
}
}
class Test extends Student
{
int m1,m2;
void getm()
{
System.out.println("Enter marks of 2 subjects");
m1=sc.nextInt();
m2=sc.nextInt();
}
}
interface Sports
{
float m=5.8f;
void puts();
}
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Example of Interface

```
class Result extends Test implements
    Sports
{
float total;
public void puts()
{
System.out.println("sports marks="+m);
}
void display()
{
total=m1+m2+m;
System.out.println("Roll No="+roll);
System.out.println("Marks1="+m1);
System.out.println("Marks2="+m2);
System.out.println("sports marks="+m);
System.out.println("Total="+total);
}
}
public class MainClass
{
public static void main(String args[])
{
Result r1=new Result();
r1.getroll();
r1.getm();
r1.puts();
r1.display();
}
}
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Interfaces Can Be Extended

- One interface can inherit another by use of the keyword extends.
- The syntax is the same as for inheriting classes.
- When a class implements an interface that inherits another interface, it must provide implementations for all methods defined within the interface inheritance chain.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Write program for following inheritance hierarchy

```
interface A {
void show();
}
interface B extends A {
void display();
}
// This class must implement all of A and B methods
class MyClass implements B {
public void show() {
System.out.println("Implement show().");
}
public void display() {
System.out.println("Implement display().");
}}
class IFExtend {
public static void main(String arg[]) {
MyClass ob = new MyClass();
ob.show();
ob.display();
}}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 24
Method Overriding

Method Overriding

- What is method overriding?
- If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in Java**.
- In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.

- Usage of Java Method Overriding
- Method overriding is used to provide the specific implementation of a method which is already provided by its super class.
- Method overriding is used for runtime polymorphism

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Method Overriding

Rules for Java Method Overriding

- The method must have the same name as in the parent class
- The method must have the same parameter as in the parent class.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Method Overriding

```
class Vehicle{
  //defining a method
  void run(){System.out.println("Vehicle is running");}
}
//Creating a child class
class Bike2 extends Vehicle{
  //defining the same method as in the parent class
  void run(){System.out.println("Bike is running safely");}

  public static void main(String args[]){
  Bike2 obj = new Bike2();//creating object
  obj.run();//calling method
  }
}
```

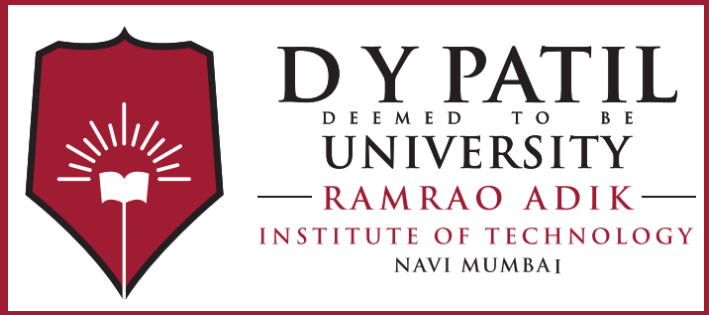
OUTPUT:

Bike is running safely

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You



Subject Name: OOP

Unit No:4

Unit Name: Inheritance and Packages

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 25 – Built in Packages	3
Lecture 26 – User Defined Packages	8

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 25
Built in Packages

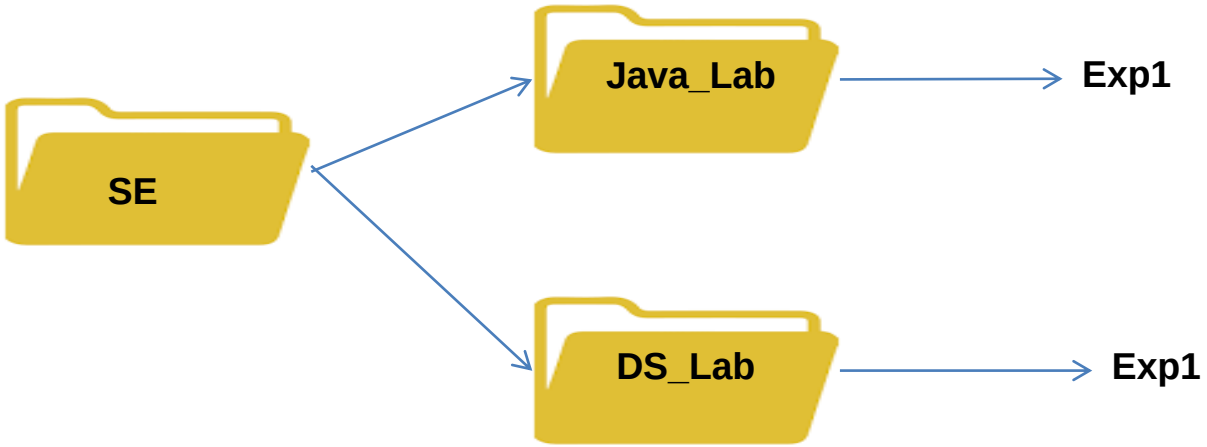
Packages

- A **java package** is a group of similar types of classes, interfaces and sub-packages.
- Package in java can be categorized in two form, built-in package and user-defined package.
- There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.
- Here, we will have the detailed learning of creating and using user-defined packages.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Packages

- Packages are nothing but “folders”



- collection of classes and interfaces
- Used to organize properly, to increase readability and reusability of code
- provides a unique namespace for the classes.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Packages

- Declaration resides at the top of a Java source file.
- Eliminates naming conflicts
- Class that reside inside a package cannot be referred by their own name alone.
- The package name has to precede the name of the class of which it is a part of.
- All classes are part of some or the other package.
- If the package keyword is not used in any class for mentioning the name of the package, then it becomes part of the default/unnamed package

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Built-In package

- Those which are available in java library
- Used to make use of existing classes and interfaces
  - java.lang
  - java.util
  - java.awt
  - java.applet etc…

- To import in-built package in JAVA programs
  - import java.util.*;
  - import java.util.Scanner;

- Any number of import statements can be given

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 26
User Defined Packages

To create user defined package

- Use *package* keyword

- Is very first statement in JAVA program
- package pkg_name;

- if you want to create sub-packages
  - package rootPackage.subPackage1;

- Must have exactly one public class

- If you want more than one public class in same package then create different files for each public class.

- Save JAVA program with public class class_name
- . public class class_name { }

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

How to write package program

File1: class_name.java

```
package mypack;
public class class_name
{
//Body of class
}
class another_class_name
{
}
```

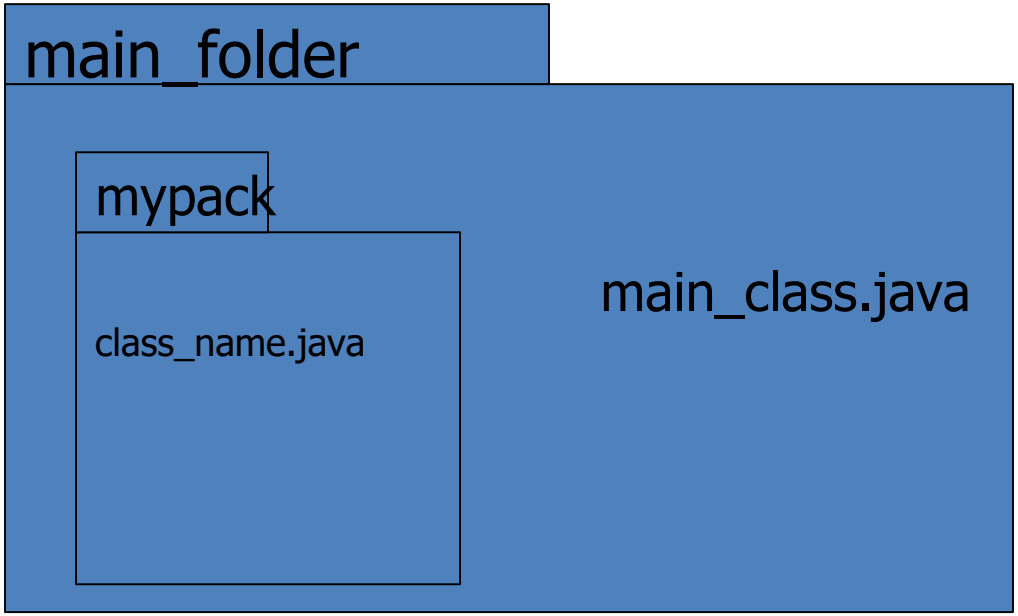
File2: main_class.java

```
import mypack.*;
class main_class
{
//Body of main class
}
```

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

How to save package program

- Create folder with name similar to package name.
- Save the program with file name similar to the name of the public class in your package.
- Save the program which imports the package outside the package folder.



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Compiling and executing packages

- compiling the class from our package
  - mypack>javac class_name.java

- executing the class from main_folder
  - To compile main program
- main_folder>javac main_class.java

  - To execute main program
- \main_folder>java main_class

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

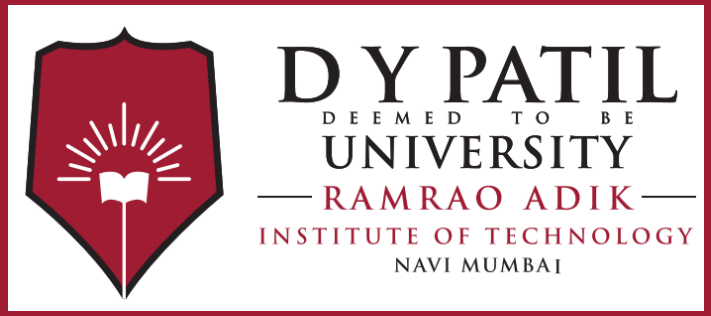
WAP to create user defined package with Student class and display students information.

```
package studentpkg;
public class Student
{
        public int rno;
        public String name;

public Student(int r, String n)
        {
                rno=r;
                name=n;
        }
        public void display()
        {
System.out.println("Roll No= "+rno);
System.out.println("Name="+name);
        }
}
```

```
import studentpkg.Student;
import java.util.*;
class PackageDemo
{
public static void main(String args[])
        {
Student s1=new Student(101,"JOHN");
s1.display();
        }
}
```

```
Output
Roll No=101
Name=JOHN
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI



Subject Name: OOP

Unit No:5

Unit Name: Exception Handling and Multithreading

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 27 – Error vs Exception 16

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 27
Error vs Exception

Exception

- An exception is an unexpected event that occurs during program execution. It affects the flow of the program instructions which can cause the program to terminate abnormally.
- An exception can occur for many reasons. Some of them are:
  - Invalid user input
  - Device failure
  - Loss of network connection
  - Physical limitations (out of disk memory)
  - Code errors
  - Opening an unavailable file

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Exception(cont..)

- **Exceptions** can be caught and handled by the program.
- When an exception occurs within a method, it creates an object. This object is called the exception object.
- It contains information about the exception such as the name and description of the exception and state of the program when the exception occurred.
- **Exception Types**

  → RuntimeException

  → IOException

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

RuntimeException

- A **runtime exception** happens due to a programming error. They are also known as **unchecked exceptions**.
- These exceptions are not checked at compile-time but run-time.
- Some of the common runtime exceptions are:

  → Improper use of an API - IllegalArgumentException

  → Null pointer access (missing the initialization of a variable) - NullPointerException

  →Out-of-boundsarray access - ArrayIndexOutOfBoundsException

  → Dividing a number by 0 - ArithmeticException

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

IOException

- An IOException is also known as a **checked exception**.
- They are checked by the compiler at the compile-time and the programmer is prompted to handle these exceptions.
- Some of the examples of checked exceptions are:

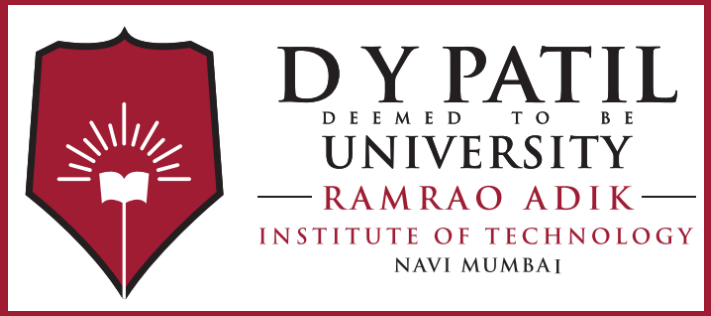
  → Trying to open a file that doesn't exist results in FileNotFoundException

  → Trying to read past the end of a file

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You



Subject Name: OOP

Unit No:5

Unit Name: Exception Handling and Multithreading

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 28 – Exception handling using try, catch	3
Lecture 29 – finally, throw and throws, Multiple try and catch blocks	7
Lecture 30 – User defined exception	19

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 28
Exception handling using try, catch

Handling Exceptions

- To catch and handle an exception, we place the try...catch...finally block around the code that might generate an exception.
- The finally block is optional.
- The syntax for try...catch...finally is:

```
try
 {
    // code
 }
catch (ExceptionType e)
 {
    // catch block
  }
 finally
  {
    // finally block
  }
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

try....catch Block

- The code that might generate an exception is placed in the **try** block.
- Every try block should be immediately followed by the catch or finally block. When an exception occurs, it is caught by the catch block that immediately follows it.
- **catch** blocks cannot be used alone and must always be preceded by a try block.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Program on try catch block

```
class Main
{
   public static void main(String[] args)
    {
       try
        {
            int divideByZero = 5 / 0;
            System.out.println("Rest of code in try block");
         } // end of try block
       catch (ArithmeticException e)
        {
            System.out.println("ArithmeticException => " +
    e.getMessage());
         }   // end of catch block
    }       }
```

Output:

ArithmeticException =>
Divide by zero

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Lecture No: 29
finally, throw and throws, Multiple try and catch blocks

finally

- Java supports another statement known as finally statement that can be used to handle an exception that is not caught by any of the previous catch statements.
- We can put finally block after the try block or after the last catch block.
- The finally block is optional. However, if defined, it is always executed.
- Even if a try block completes without problems, the finally block executes.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Syntax of Finally Block

```
try
    {
        //code
    }
catch (ExceptionType1 e1)
    {
        // catch block
    }
catch (ExceptionType1 e2)
    {
        // catch block
    }
finally
    {
        // finally block always executes
    }
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Program on try catch finally block

```
class Main
{
   public static void main(String[] args)
    {
       try
        {
            int divideByZero = 5 / 0;
            System.out.println("Rest of code in try block");
         } // end of try block
       catch (ArithmeticException e)
        {
            System.out.println("ArithmeticException => " + e.getMessage());
         }   // end of catch block
       finally { System.out.println("Finally block is always executed"); }
    }       }
```

Output:

ArithmeticException => Divide by zero
Finally block is always executed

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

THROW keyword

- Java throw keyword is used throw an exception explicitly in the code, inside the function or the block of code.
- We are allowed to throw only one exception at a time i.e. we cannot throw multiple exceptions.
- The syntax for throw keyword is as follows:

  throw new ThrowableInstance;
- For example:

  throw new NullPointerException();

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Example on throw keyword :

```
class Main
{
    public static void divideByZero()
     {
        throw new ArithmeticException("Trying to divide by 0");
      }
     public static void main(String[] args)
     {
         divideByZero();
      }
}
```

Output :

```
Exception in thread "main“ java.lang.ArithmeticException: Trying to divide by 0 at
        Main.divideByZero(Main.java:3)
        at Main.main(Main.java:7)
exit status 1
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

throws keyword

- We use the throws keyword in the method declaration to declare the type of exceptions that might occur within it.
- The throws keyword is used in signature of the method.
- The throws keyword is followed by class names of Exceptions to be thrown.
- We can declare multiple exceptions using throws keyword that can be thrown by the method. For example, main() throws IOException, SQLException.
- **Syntax :**

  accessModifier returnType methodName() throws ExceptionType1, ExceptionType2 …

  { // code }

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Example on throws keyword

```
public class TestThrows
 {
     //defining a method
     public static int divideNum(int m, int n) throws ArithmeticException
      {
          int div = m / n;
          return div;
       }
      public static void main(String[] args)
      {
         TestThrows obj = new TestThrows();
         try
         {
            System.out.println(obj.divideNum(45, 0));
          }
         catch (ArithmeticException e)
         {
             System.out.println("\nNumber cannot be divided by 0");
          }
          System.out.println("Rest of the code..");
     }
 }
```

```
C:\Users\Anurati\Desktop\abcDemo>javac TestThrows.java

C:\Users\Anurati\Desktop\abcDemo>java TestThrows

Number cannot be divided by 0
Rest of the code..
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Multiple Catch Blocks

- For each try block, there can be zero or more catch blocks.
- The argument type of each catch block indicates the type of exception that can be handled by it.
- Multiple catch blocks allow us to handle each exception differently.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

try…with multiple catch

First write catch with subclass exception objects and then super class object

Syntax :

```
        try
        {
                //statements to be monitored
        }
        catch(Exception_type ExOb1)
        {
                //exception handling code
        }
catch(Exception_type ExOb2)
        {
                //exception handling code
        }
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Program on Multiple Catch Block

```
class Main {
 public static void main(String[] args)
  {
      //try block containing exception prone code
      try{
         System.out.println("try Block:: Begin");
         int myArray[]=new int[5];
         myArray [5]=10/0;
           }
      //multiple catch blocks
       catch(ArithmeticException e)
        {
           System.out.println("Arithmetic Exception :: Divide by
    zero!!");
        }
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Program on Multiple Catch Block (cont..)

```
catch(ArrayIndexOutOfBoundsException e)
{
   System.out.println("ArrayIndexOutOfBounds :: Accessed index
    out of bounds");
}
catch(Exception e)
{
   System.out.println("Exception :: " + e.getMessage ());
}
System.out.println("rest of the code");
}
}
```

Output :

Arithmetic Exception :: Divide by zero!!
rest of the code

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 30
User defined exception

User defined exception

- Used to create our own exceptions
- Just define a subclass of Exception class to create your own exception
- throw keyword is used to throw an exception

  String toString()
- is a method of Throwable class which Returns a String object containing a description of the exception.
- This method is called by println( ) when outputting a Throwable object.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Syntax

```
class exception_name extends Exception
  {
        public String toString()
        {
           //body of method
        }
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Creating your own exception

```
class MyException extends Exception {
private int detail;
MyException(int a) {
detail = a;
}
public String toString() {
return "MyException[" + detail + "]";
}
}
class ExceptionDemo {
static void compute(int a) throws MyException {
System.out.println("Called compute(" + a + ")");
if(a > 10)
throw new MyException(a);
System.out.println("Normal exit");
}
```

```
public static void main(String args[]) {
try {
compute(1);
compute(20);
} catch (MyException e) {
System.out.println("Caught " + e);
}
}
```

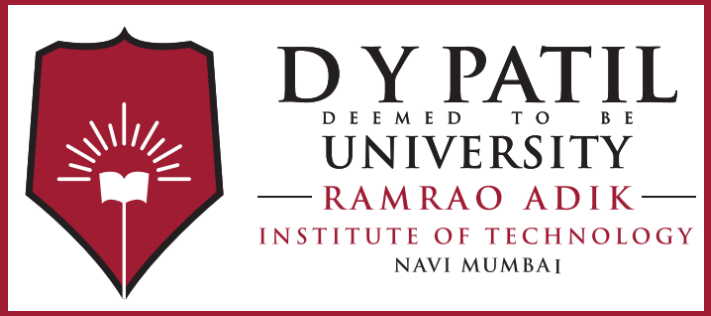
Output:

```
Called compute(1)
Normal exit
Called compute(20)
Caught MyException[20]
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You



Subject Name: OOP

Unit No:5

Unit Name: Exception Handling and Multithreading

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 31 – Thread lifecycle	3
Lecture 32 –Thread class methods	14
Lecture 33 –Creating threads using extends keyword	23

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 31
Thread lifecycle

Introduction

- Multitasking
  - Computer seems to work on multiple tasks concurrently or simultaneously

- Approach
  - Computer does some work on a particular task
  - Computer then quickly switch to next task
  - Tasks managed by operating system (scheduler)
  - Can improve performance by reducing waiting

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Introduction

- Multitasking can be
  - Process based
    - A process is, a program that is executing.
    - Run two or more programs concurrently
    - e.g. running the Java compiler and at the same time using a text editor.
    - Processes are heavyweight tasks
  - Thread Based
    - The thread is the smallest unit of program.
    - Threads are light weight tasks
    - A single program can have multiple threads and can perform multiple tasks simultaneously.
    - e.g. a text editor can format text at the same time that it is printing a document

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Multithreading

- Multithreading in java is a process of executing multiple threads simultaneously
- A program is divided into two or more subprograms, which can be implemented at the same time in parallel.
- Multiprocessing and multithreading, both are used to achieve multitasking.
- Java Multithreading is mostly used in games, animation etc.

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

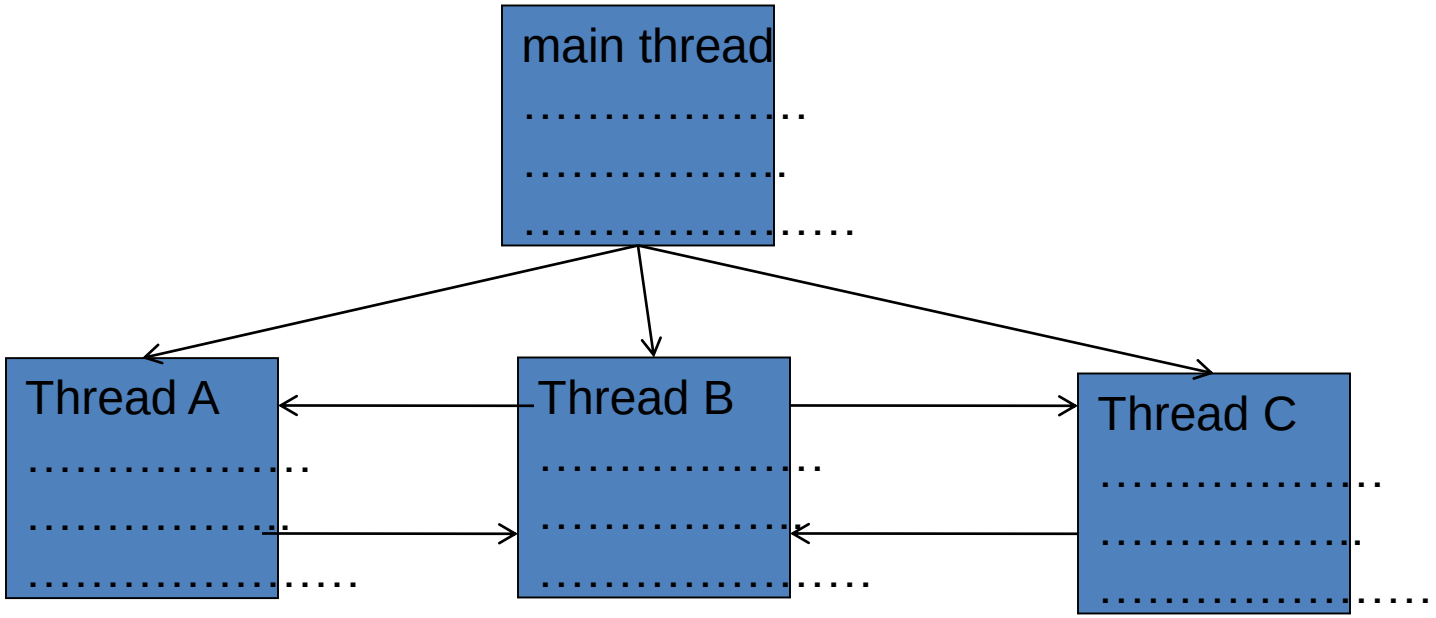
Advantage

- It doesn't block the user
- can perform many operations together so it saves time.
- Threads are independent so it doesn't affect other threads

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

The Java Thread Model

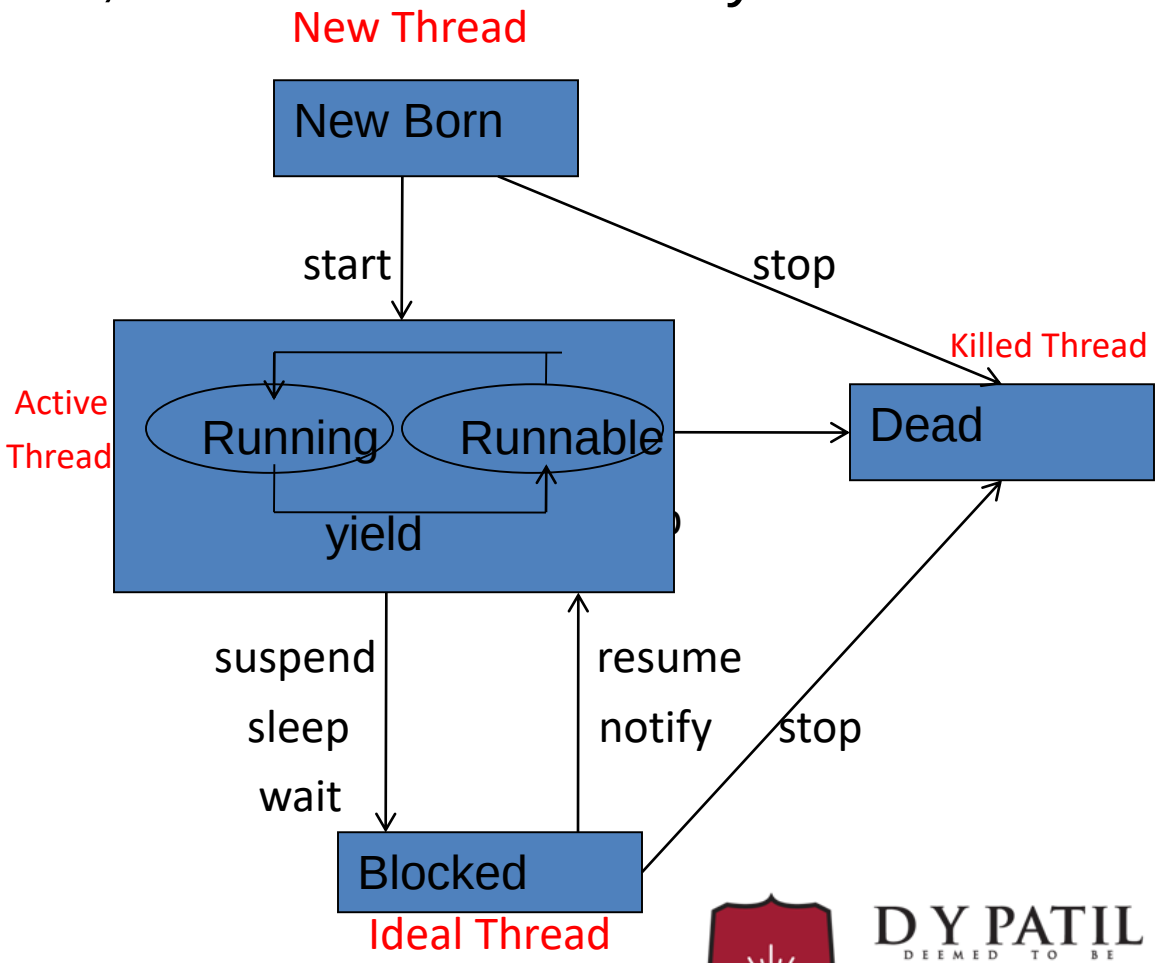
- Every java program has at least one thread which is called as main thread which is actually a main method module.



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Life cycle of thread

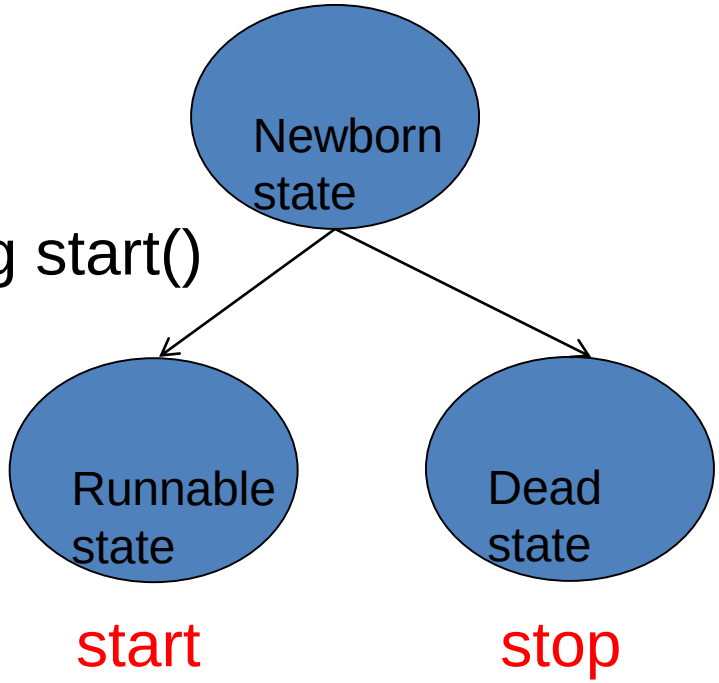
- During the life time ,thread can be in any one of the following states:
  - Newborn State
  - Runnable State
  - Running State
  - Blocked State
  - Dead State



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Newborn State

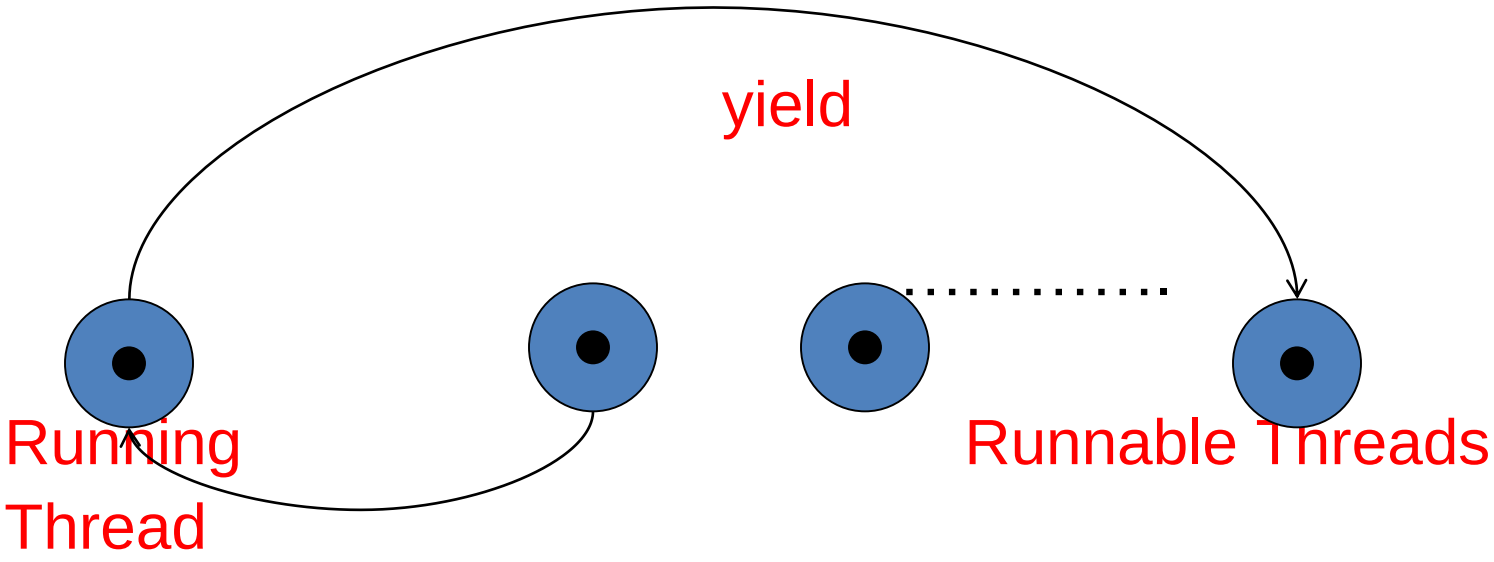
- When we create a thread object, the thread is born and it is said to be in newborn state
- At this state, we can do
  - schedule it for running using start()
  - kill it using stop()



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Runnable state

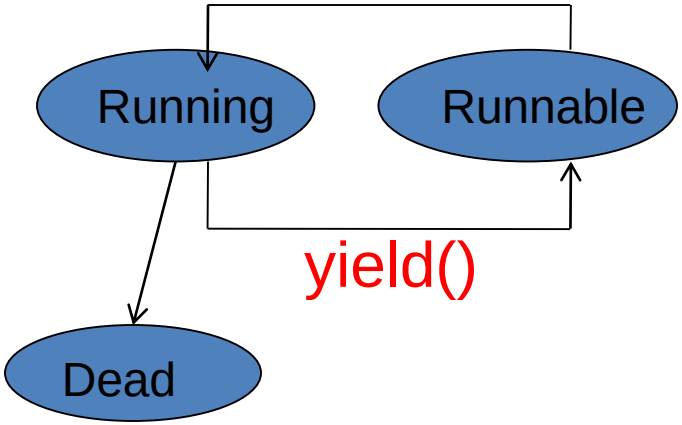
- Runnable state means the thread is ready for execution and is waiting for availability of the processor.
- Thread joins queue of threads waiting for processor.



D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Running state

- Running state means processor has given time to thread for execution.
- Thread runs till it stop control by its own or preempted by any high priority thread



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Blocked state and Dead state

- Blocked state:
  - Thread is said to be blocked when it is prevented from entering into runnable state and subsequently in running state
  - Blocked thread is considered as ”not runnable” but “not dead” and it is fully qualified to run again
- Dead state:
  - A running thread ends its life when it has completed its execution.
  - We can kill the thread by calling stop() method

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Lecture No: 32
Thread class methods

Thread class

- Data Members:
  - static int MAX_PRIORITY : This is the maximum priority (10) that a thread can have
  - static int MIN_PRIORITY : This is the minimum priority (1) that a thread can have.
  - static int NORM_PRIORITY : This is the default priority (5) that is assigned to a thread.
- Constructors:
  - Thread t1= new Thread()
    - Constructs a new Thread.
  - Thread t1= new Thread(String name)
    - Constructs a new Thread with the specified name

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thread class Methods

Method	Meaning
static int activeCount()	returns number of active threads
static Thread currentThread()	returns reference to the currently executing thread object
String getName()	returns current thread's name
void setName(String name)	sets the name for the thread
int getPriority	returns current thread's priority
void setPriority(int no)	sets new priority for the thread
boolean isAlive()	checks whether the thread is alive or not
void join()	wait for this thread to die
void run()	Entry point for the thread
void start()	Start a thread by calling its run method
void stop()	stop the execution of thread
void yield()	relinquish control to another thread
void static sleep(long milisec)	Causes the currently executing thread to sleep for the specified number of milliseconds

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

The Main Thread

- When a Java program starts up, one thread begins running immediately, which is called as the *main thread of the program*
- The main thread is important for two reasons:
  - It is the thread from which other "child" threads will be spawned.
  - It must be the last thread to finish execution. When the main thread stops, the program terminates.
- Although the main thread is created automatically when the program is started, it can be controlled through a **Thread object.**
- To do so, obtain a reference to it by calling the method currentThread( )
- This method returns a reference to the thread in which it is called.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Controlling the main Thread

```
class CurrentThreadDemo {
public static void main(String args[])
{
Thread t = Thread.currentThread();
System.out.println("Current thread: "
    + t);

// change the name of the thread
t.setName("My Thread");
System.out.println("After name
    change: " + t);
for(int n = 5; n > 0; n--) {
System.out.println(n);
}
}
}
```

OUTPUT:

Current thread: Thread[main,5,main]

After name change: Thread[My Thread,5,main]

5

4

3

2

1

//The values in output are:

[Thread name, Priority, Thread Group]

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Use of yield(),stop() &sleep()Methods

```
class A extends Thread{
public void run(){
for(int i=1;i<=5;i++){
if(i==2)
yield();
System.out.println("\t From Thread A :"+i);
}
System.out.println("Exit from A");
}}
class B extends Thread{
public void run(){
for(int j=1;j<=5;j++)
{
System.out.println("\t From Thread B :"+j);
if(j==3)
stop();
}
System.out.println("Exit from B");
}}
class C extends Thread{
public void run(){
for(int k=1;k<=5;k++)
{
try{
sleep(3000);
System.out.println("\t From Thread C :"+k);
}catch(Exception e){}
}
System.out.println("Exit from C");}}
```

```
class ThreadDemo
{
public static void main(String args[])
{ A a1=new A();
 B b1=new B();
C c1=new C();
a1.start();
b1.start();
c1.start();
}}
```

OUTPUT:

From Thread A :1
From Thread B :1
From Thread B :2
From Thread B :3
From Thread A :2
From Thread A :3
From Thread A :4
From Thread A :5
Exit from A
From Thread C :1
From Thread C :2
From Thread C :3
From Thread C :4
From Thread C :5
Exit from C

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Use of join() method

- In a multithreaded program, the main thread must be the last thread to finish running.
- If the main thread finishes before a child thread has completed ,then the Java run-time system may "hang.“
- To ensure ,that main thread is the last to stop , join() method is used.
- Syntax:
  - final void join( ) throws InterruptedException
- This method waits until the thread on which it is called terminates

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Using isAlive( ) and join( )

```
class A extends Thread{
public void run(){
for(int i=1;i<=5;i++){
System.out.println("\t From Thread A :"+i);
}
System.out.println("Exit from A");
}}
class B extends Thread{
public void run(){
for(int j=1;j<=5;j++){
System.out.println("\t From Thread B :"+j);
}
System.out.println("Exit from B");
}}
class ThreadDemo1
{
public static void main(String args[])
{
A a1=new A();
B b1=new B();
a1.start();
System.out.println("Thread A is alive:
"+a1.isAlive());
b1.start();
System.out.println("Thread B is alive:
"+b1.isAlive());

for(int n = 5; n > 0; n--) {
System.out.println("Main Thread"+n);
}
```

```
try{ a1.join();
b1.join();     }catch(Exception e){}
System.out.println("End of Main Thread");
System.out.println("Thread A is alive:  "+a1.isAlive());
System.out.println("Thread B is alive:  "+b1.isAlive());
}}
Output:
From Thread A :1
Thread A is alive:  true
        From Thread A :2
        From Thread A :3
        From Thread A :4
        From Thread A :5
Exit from A
        From Thread B :1
Thread B is alive:  true
Main Thread5
Main Thread4
Main Thread3
Main Thread2
Main Thread1
        From Thread B :2
        From Thread B :3
        From Thread B :4
        From Thread B :5
Exit from B
End of Main Thread
Thread A is alive:  false
Thread B is alive:  false
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thread Priorities

- Thread priorities are used by the thread **scheduler** to decide when each thread should be allowed to run.
- The higher-priority threads get more CPU time than lower-priority threads.

- There are 3 predefines values of priorities
  - static int MAX_PRIORITY =10
  - static int MIN_PRIORITY =1
  - static int NORM_PRIORITY = 5

- To set a thread's priority, use the **setPriority( ) method.**
  - final void setPriority(int level)
- The value of level must be in the range 1 to 10.
- To obtain the current priority setting use the **getPriority( ) method**
  - final int getPriority( )

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 33
Creating threads using extends keyword

Creating Thread

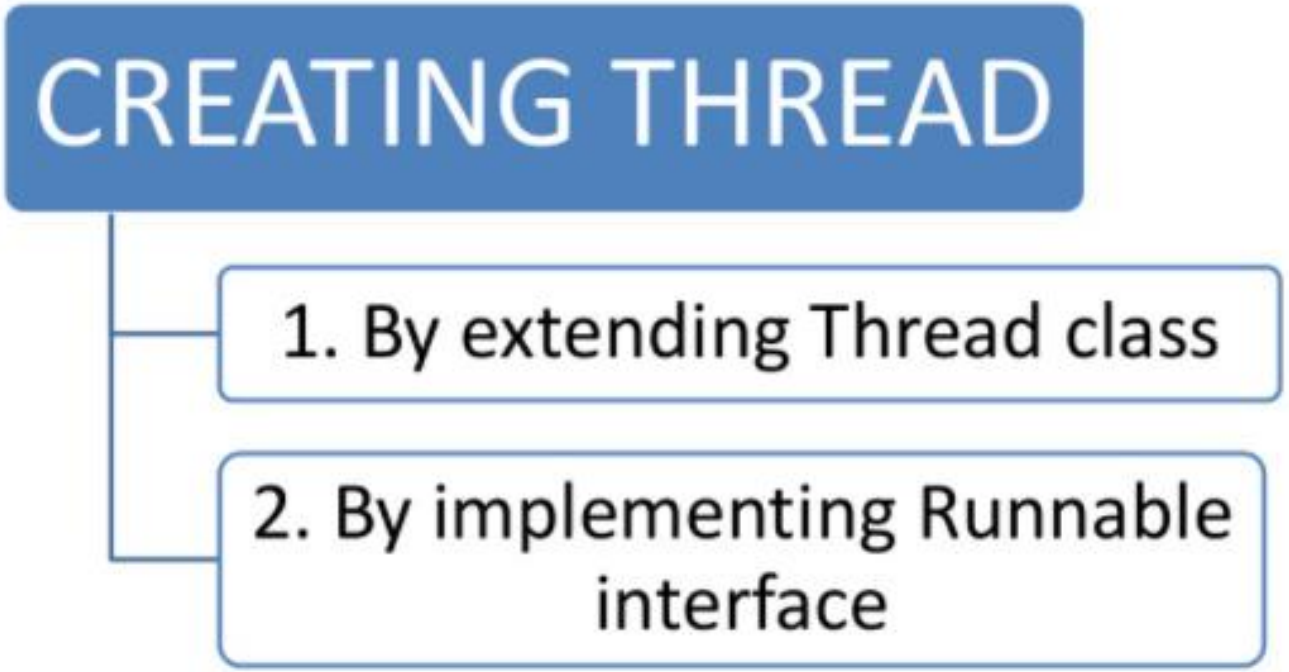
- Threads are implemented in the form of objects.
- The run() and start() are two inbuilt methods which helps to thread implementation
- The run() method is the heart and soul of any thread – It makes up the entire body of a thread

```
public void run()
{
        ……….
}
```

- The run() method can be initiating with the help of start() method.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Creating Thread…



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Why two ways?

- class A extends Thread{……..}
  - It will work fine
- class B extends A extends Thread{…..error..}
  - one class can not extend two classes
- class B extends A implements Runnable
  - This class definition is correct as one class can extend one class and implements an interface.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Extending Thread class

1.Declare a class extending the Thread class

```
class MyThread extends Thread
{
   ……..
}
```

2. Implement run() method

- override this method to implement code to be executed by the thread

```
public void run()
{……..//thread code
}
```

3. Starting a new thread

- To actually create and run the thread ,create its object and call the start() method.

```
MyThread t1=new MyThread()
t1.start()
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Example

1. By Extending Thread class

```
class Multi extends Thread                          // Extending thread class
{
public void run()                                   // run() method declared
{
System.out.println("thread is running...");
}
public static void main(String args[])
{
Multi t1=new Multi();                    //object initiated
t1.start();                              // run() method called through start()
 }
}
```

Output: thread is running...

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Example

```
class A extends Thread
{
public void run()
{
for(int i=1;i<=5;i++)
      System.out.println("\t From Thread A :"+i);
System.out.println("Exit from A");
}}
class B extends Thread
{
public void run()
{
for(int j=1;j<=5;j++)
     System.out.println("\t From Thread B :"+j);
System.out.println("Exit from B");
}  }
class ThreadDemo{
public static void main(String args[]){
A a1=new A();
B b1=new B();
a1.start();
b1.start();      }}
```

OUTPUT 1:

```
 From Thread A :1
        From Thread B :1
        From Thread A :2
        From Thread B :2
        From Thread A :3
        From Thread B :3
        From Thread A :4
        From Thread B :4
        From Thread A :5
        From Thread B :5
Exit from B
Exit from A
```

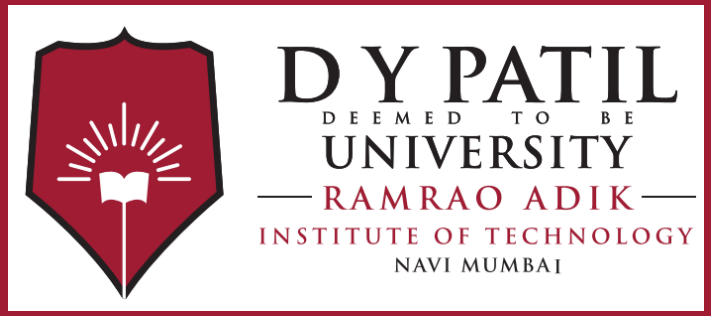
OUTPUT 2:

```
 From Thread A :1
        From Thread B :1
        From Thread B :2
        From Thread B :3
        From Thread B :4
        From Thread B :5
Exit from B
        From Thread A :2
        From Thread A :3
        From Thread A :4
        From Thread A :5
Exit from A
```

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You



Subject Name: OOP

Unit No:5

Unit Name: Exception Handling and Multithreading

Faculty Name : Mr. Dayanand Dhongade

Index

Lecture 34 – Creating threads using implements keyword 3

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 34
Creating threads using implements keyword

Implementing Runnable Interface

1.Declare a class implementing the Runnable interface

```
class A implements Runnable
{ ……..
  }
```

2. Implement run() method

- override this method to implement code to be executed by the thread

```
public void run()
{……..//thread code
}
```

3. Create object of Thread class and pass object of class which implements Runnable interface to it

- A a1=new A();
- Thread t1=new Thread(a1)

3. Call start() method to run a thread

```
t1.start()
```


D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Example

2. By implementing Runnable interface

```
class Multi3 implements Runnable                    // Implementing Runnable interface
{
public void run()
{
System.out.println("thread is running...");
}
public static void main(String args[])
{
Multi3 m1=new Multi3();                              // object initiated for class
Thread t1 =new Thread(m1);                           // object initiated for thread
t1.start();
 } }
```

Output: thread is running...

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Example

```
class A implements Runnable
{
public void run()
{
for(int i=1;i<=5;i++)
{
System.out.println("\t From Thread A :"+i);
}
System.out.println("Exit from A");
}
}
class RunnableDemo
{
public static void main(String args[])
{
A a1=new A();
Thread t1=new Thread(a1);
t1.start();
System.out.println("End of Main Thread");
}  }
```

OUTPUT:

From Thread A :1

From Thread A :2

From Thread A :3

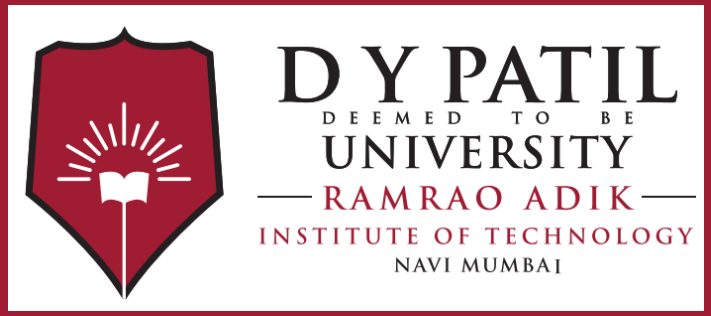
From Thread A :4

From Thread A :5

Exit from A

End of Main Thread

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI



Subject Name: OOP

Unit No: 6

Unit Name: GUI Programming

Faculty Name : Mr. Dayanand Dhongade

Index

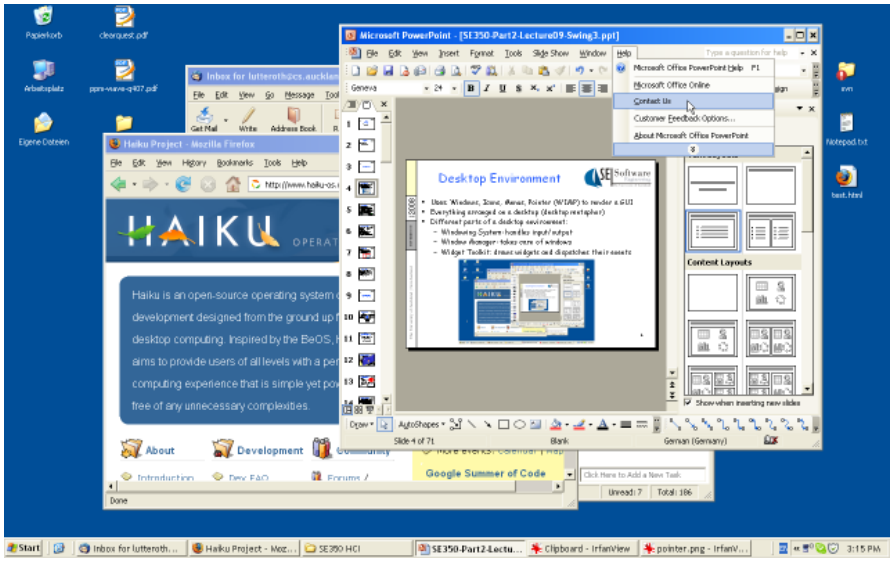
Lecture 35 – Introduction to Java Swing Package, Container Class	9
Lecture 36 – Difference Between AWT And Swing	24

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 35
Introduction to Java Swing Package, Container Class

Desktop Environment

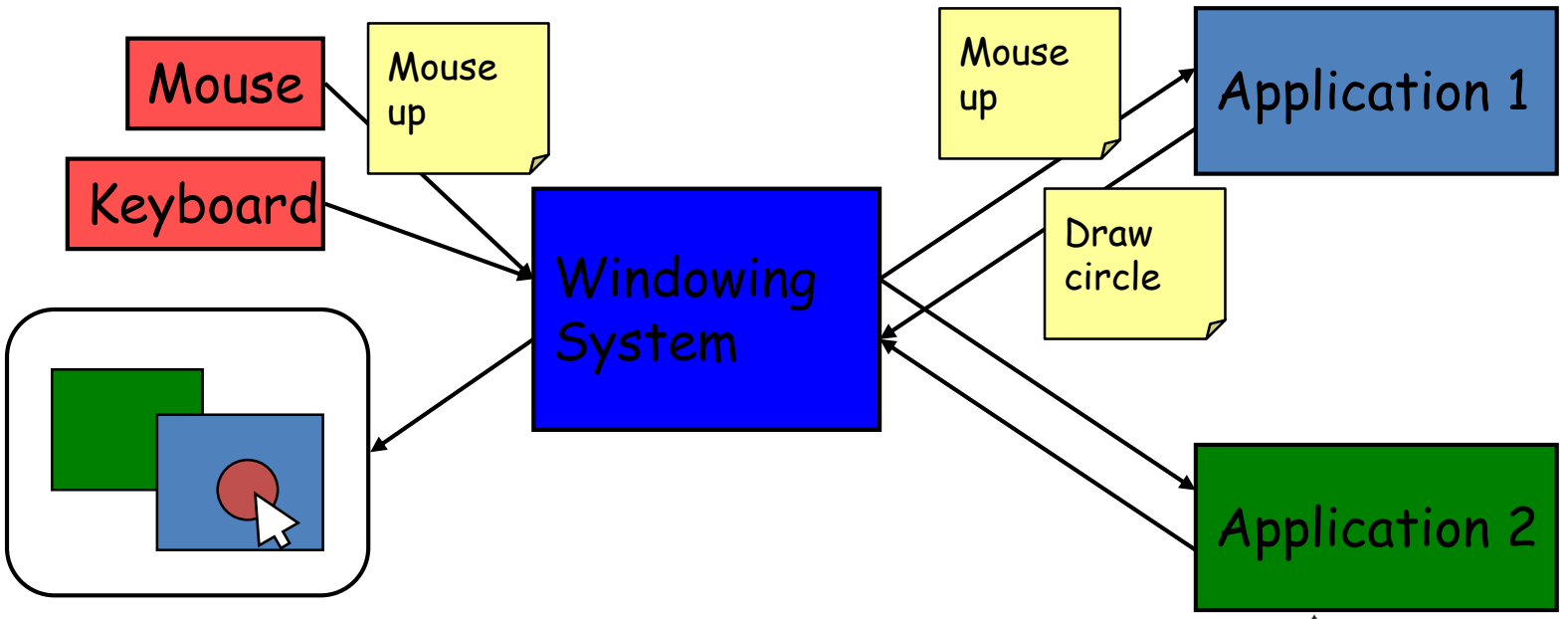
- Uses **W**indows, **I**cons, **M**enus, **P**ointer to render a GUI
- Everything arranged on a desktop
- Different parts of a desktop environment (may be merged):
  - Windowing System: handles input/output
  - Widget Toolkit: draws widgets and dispatches their events
  - Window Manager: takes care of windows



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Windowing System

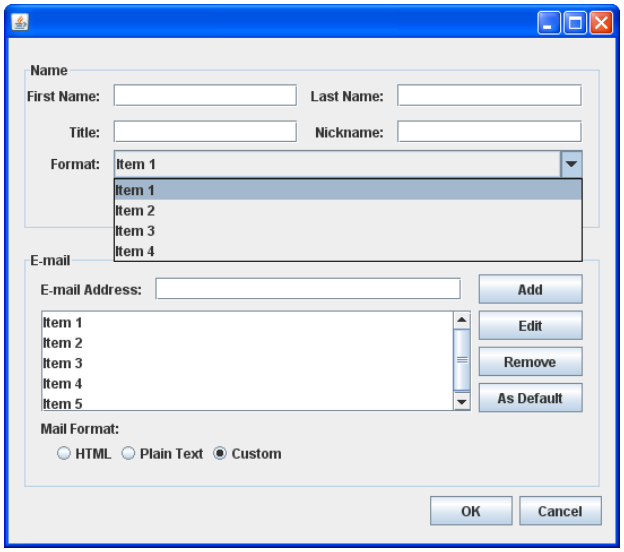
- Manages input and output devices: graphics cards, screens, mice, keyboards
- Redirects data from input devices to applications
- Receives and processes drawing commands from applications

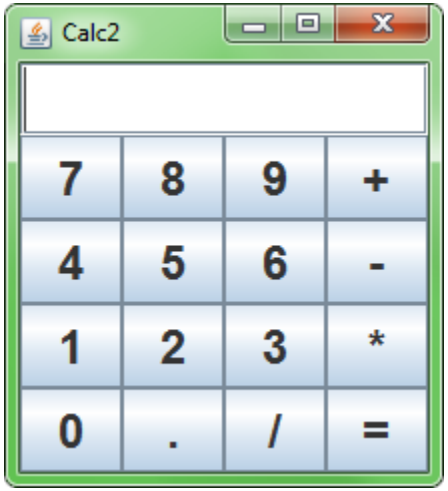


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

GUI in Java

- **GUI (Graphical User Interface) in Java** is an easy-to-use visual experience builder for Java applications.
- It is mainly made of graphical components **like buttons, labels, windows, etc.** through which the user can interact with an application.
- GUI plays an important role to build easy interfaces for Java applications.





D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Swing in Java

- Swing in Java is a Graphical User Interface (GUI) toolkit that includes the GUI components.
- Swing provides a rich set of widgets and packages to make sophisticated GUI components for Java applications.
- Swing is a part of Java Foundation Classes(JFC), which is an API for Java GUI programing that provide GUI.
- The Java Swing library is built on top of the Java Abstract Widget Toolkit (**AWT**), an older, platform dependent GUI toolkit.
- Swing offers much-improved functionality over AWT, new components, expanded components features, excellent event handling with drag and drop support.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Swing in Java

- Swing has about four times the number of User Interface [UI] components as AWT and is part of the standard Java distribution.
- By today's application GUI requirements, AWT is a limited implementation, not quite capable of providing the components required for developing complex GUI's required in modern commercial applications.
- The AWT component set has quite a few bugs and really does take up a lot of system resources when compared to equivalent Swing resources.

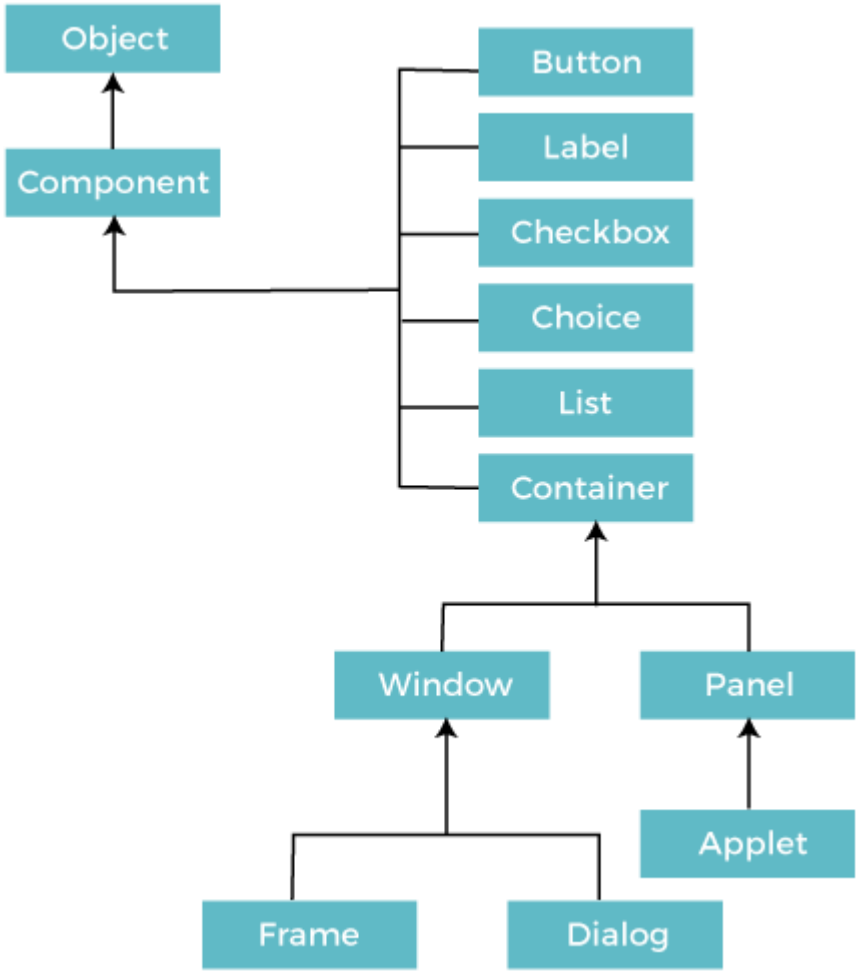
D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java AWT

- Java AWT (Abstract Window Toolkit) is an API to develop Graphical User Interface (GUI) or windows-based applications in Java.
- Java AWT components are platform-dependent i.e. components are displayed according to the view of operating system. AWT is heavy weight i.e. its components are using the resources of underlying operating system (OS).
- The java.awt package provides classes for AWT API such as TextField, Label, TextArea, RadioButton, CheckBox, Choice, List etc.

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Java AWT Hierarchy



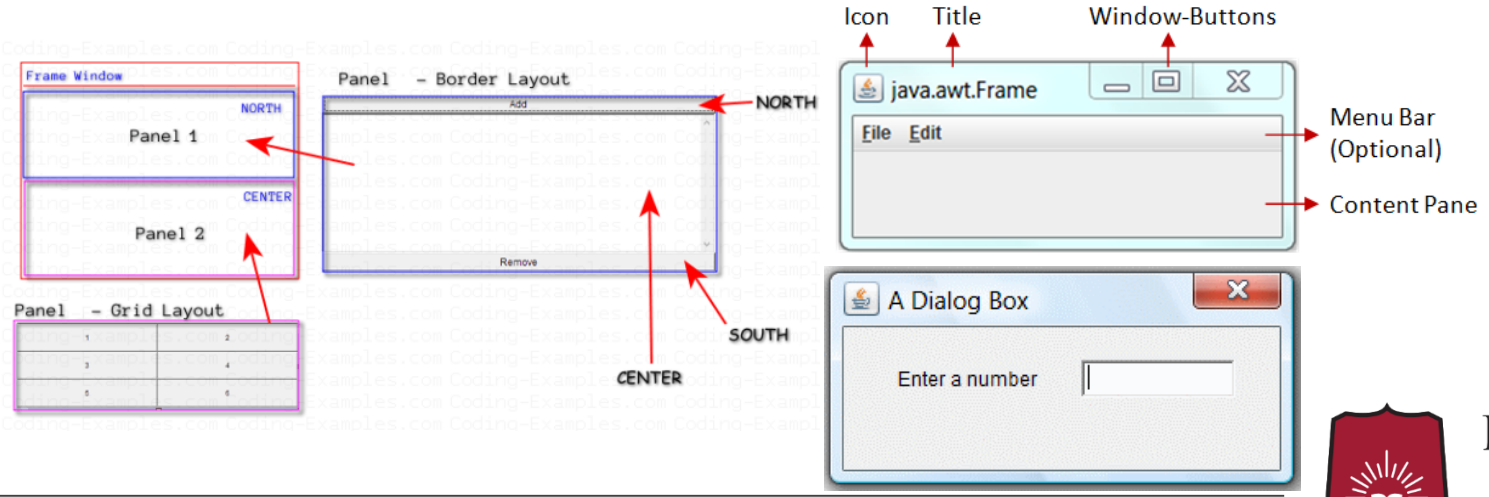
Components:

- All the elements like the button, text fields, scroll bars, etc. are called components.
- In Java AWT, there are classes for each component as shown in above diagram.
- In order to place every component in a particular position on a screen, we need to add them to a container.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Container in AWT

- The Container is a component in AWT that can contain another components like buttons, textfields, labels etc.
- It is basically a screen where the where the components are placed at their specific locations.
- Thus it contains and controls the layout of components.
- There are four types of containers in Java AWT: Window, Panel, Frame, Dialog



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Types of containers

- **Window:** The window is the container that have no borders and menu bars. You must use frame, dialog or another window for creating a window. We need to create an instance of Window class to create this container.
- **Panel:** The Panel is the container that doesn't contain title bar, border or menu bar. It is generic container for holding the components. It can have other components like button, text field etc. An instance of Panel class creates a container, in which we can add components.
- **Frame:** The Frame is the container that contain title bar and border and can have menu bars. It can have other components like button, text field, scrollbar etc. Frame is most widely used container while developing an AWT application.

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Introduction to Java Swing

- Swing is a Set Of API (API- Set Of Classes and Interfaces)
- Swing is Provided to Design a Graphical User Interfaces
- Swing is an Extension library to the AWT (Abstract Window Toolkit)
- Includes New and improved Components that have been enhancing the looks and Functionality of GUI's
- It Employs model/view design architecture
- Swing is more portable and more flexible than AWT
- Swing is Entirely written in Java

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Features Of Swing Class

- Lightweight Components
- Pluggable look and feel
- Uses MVC (Model View Controller) architecture
- Advance features such as JTable, JTabbedPane, JScollPane etc
- Platform Independent

[Java is a platform-independent language and runs on any client machine, the GUI look and feel, owned and delivered by a platform specific O/S, simply does not affect an application's GUI constructed using Swing components]

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Features Of Swing Class

- **Lightweight Components:** Starting with the JDK 1.1, its AWT supported lightweight component development. For a component to qualify as lightweight, it must not depend on any non-Java [O/s based) system classes. Swing components have their own view supported by Java's look and feel classes
- **Pluggable Look and Feel:** This feature enables the user to switch the look and feel of Swing components without restarting an application. The Swing library supports components look and feel that remains the same across all platforms wherever the program runs. The Swing library provides an API that gives real flexibility in determining the look and feel of the GUI of an application

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Features Of Swing Class

- **The MVC Connection:** Over the years, one component architecture has proven itself to be exceptionally effective:- Model-View-Controller or MVC for short.
  - In MVC terminology, the model corresponds to the state information associated with the Component
  - The view determines how the component is displayed on the screen, including any aspects of the view that are affected by the current state of the model.
  - The controller determines how the component reacts to the user

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Container Class of Swing

- Container classes are classes that can have other components on it. So for creating a Java Swing GUI, we need at least one container object. There are 3 types of Java Swing containers.
  1. **Panel**: It is a pure container and is not a window in itself. The sole purpose of a Panel is to organize the components on to a window.
  2. **Frame**: It is a fully functioning window with its title and icons.
  3. **Dialog**: It can be thought of like a pop-up window that pops out when a message has to be displayed. It is not a fully functioning window like the Frame.

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Lecture No: 36
Difference Between AWT And Swing

Difference between AWT and Swing

No.	Java AWT	Java Swing
1)	AWT components are **platform-dependent**.	Java swing components are **platform-independent**.
2)	AWT components are **heavyweight**.	Swing components are **lightweight**.
3)	AWT **doesn't support pluggable look and feel**.	Swing **supports pluggable look and feel**.
4)	AWT provides **less components** than Swing.	Swing provides **more powerful components** such as tables, lists, scrollpanes, colorchooser, tabbedpane etc.
5)	AWT **doesn't follows MVC**(Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swing **follows MVC**.

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Difference between AWT and Swing

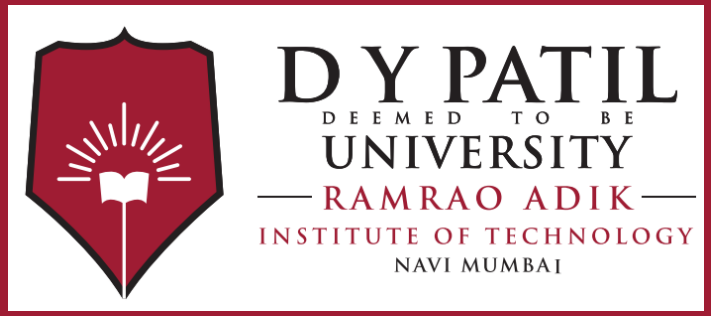
- Abstract Windowing Toolkit (AWT)
  - Original Java GUI toolkit
  - Wrapper API for native GUI components
  - Lowest-common denominator for all Java host environments

- Swing
  - Implemented entirely in Java on top of AWT
  - Richer set of GUI components
  - Pluggable look-and-feel support

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You



Subject Name: OOP

Unit No: 6

Unit Name: GUI Programming

Faculty Name : Mr. Dayanand Dhongade

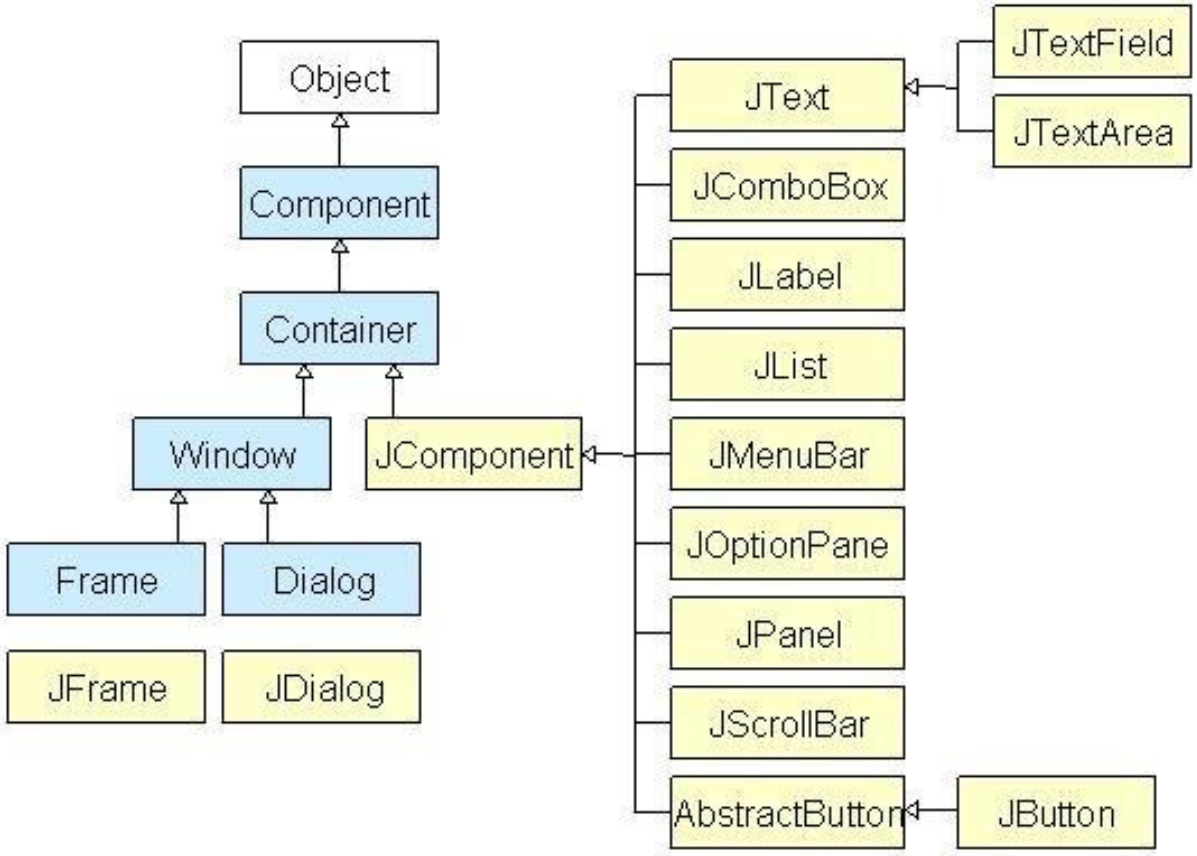
Index

Lecture 37 – Java Swing Class Hierarchy	3
Lecture 38 – Layout Manager	11
Lecture 39 – Designing GUI and building applications using swing in Java	16

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 37
Java Swing Class Hierarchy

Java Swing Class Hierarchy



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Swing Components

- The simplest Swing components have capabilities far beyond AWT components as follows:
  - Swing buttons and the labels can be displaying images instead of or in addition to text
  - The borders around most Swing components can be changed easily. For example: it is easy to put a 1 pixel border around the outside of a Swing label
  - Swing components do not have to be rectangular. Buttons, for example, can be round
  - Now The Latest Assertive technologies such as screen readers can easily getting the information from Swing components. For example: A screen reader tool can easily capture the text that is displayed on a Swing button or label

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Swing Components

Class	Description
Component	A Component is the Abstract base class for about the non menu user-interface controls of SWING. Components are represents an object with graphical representation
Container	A Container is a component that can contain SWING Components
JComponent	A JComponent is a base class for all swing UI Components In order to use a swing component that inherits from JComponent, component must be in a containment hierarchy whose root is a top-level Swing container
JLabel	A JLabel is an object component for placing text in a container
JButton	This class creates a labeled button
JColorChooser	A JColorChooser provides a pane of controls designed to allow the user to manipulate and select a color
JCheckBox	A JCheckBox is a graphical(GUI) component that can be in either an on-(true) or off-(false) state

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Swing Components

Class	Description
JRadioButton	The JRadioButton class is a graphical(GUI) component that can be in either an on-(true) or off-(false) state. in the group
JList	A JList component represents the user with the scrolling list of text items
JComboBox	A JComboBox component is Presents the User with a show up Menu of choices
JTextField	A JTextField object is a text component that will allow for the editing of a single line of text
JPasswordField	A JPasswordField object it is a text component specialized for password entry
JTextArea	A JTextArea object s a text component that allows for the editing of multiple lines of text

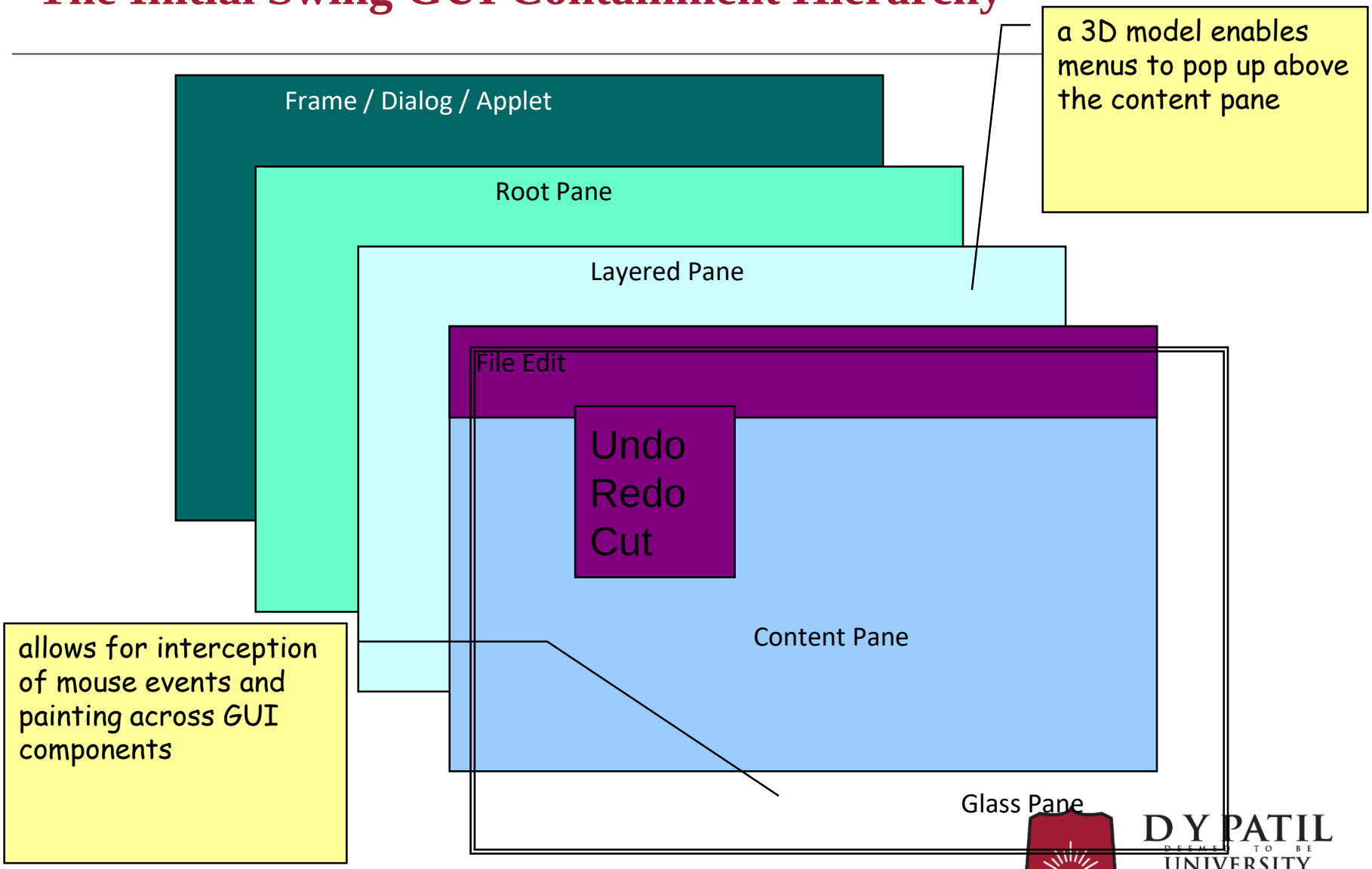
D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Swing Components

Class	Description
ImageIcon	A ImageIcon control is an implementation of the Icon interface that paints Icons from Images
JScrollbar	A JScrollbar control represents a scroll bar component in order to enable users to Select from range values
JOptionPane	JOptionPane provides set of standard dialog boxes that prompt users for a value or Something
JFileChooser	A JFileChooser it Controls represents a dialog window from which the user can select a file.
JProgressBar	As the task progresses towards completion, the progress bar displays the tasks percentage on its completion
JSlider	A JSlider this class is lets the user graphically(GUI) select by using a value by sliding a knob within a bounded interval.
JSpinner	A JSpinner this class is a single line input where the field that lets the user select by using a number or an object value from an ordered sequence

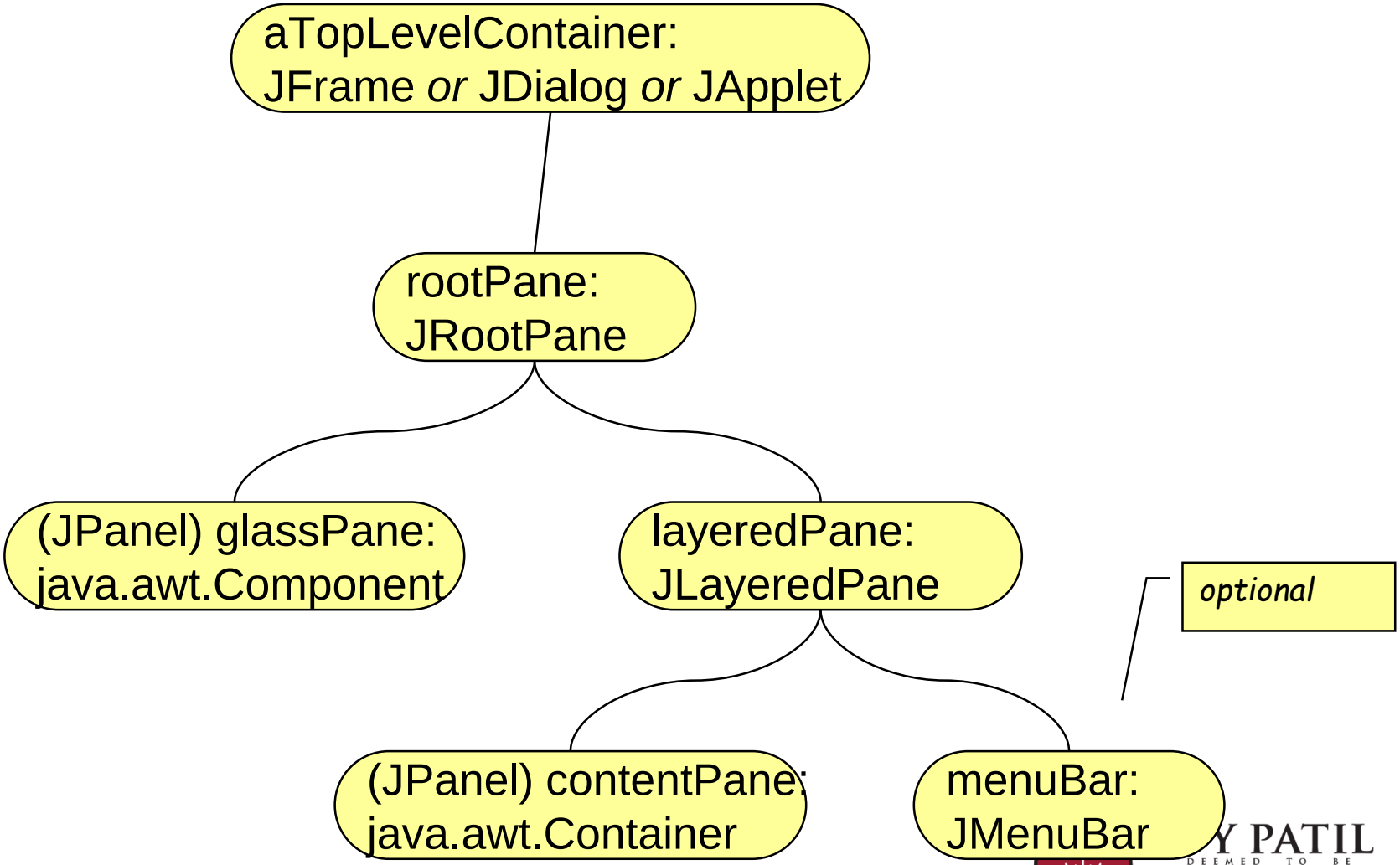
D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

The Initial Swing GUI Containment Hierarchy



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

The Initial Swing GUI Containment Hierarchy



Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 38
Layout Manager

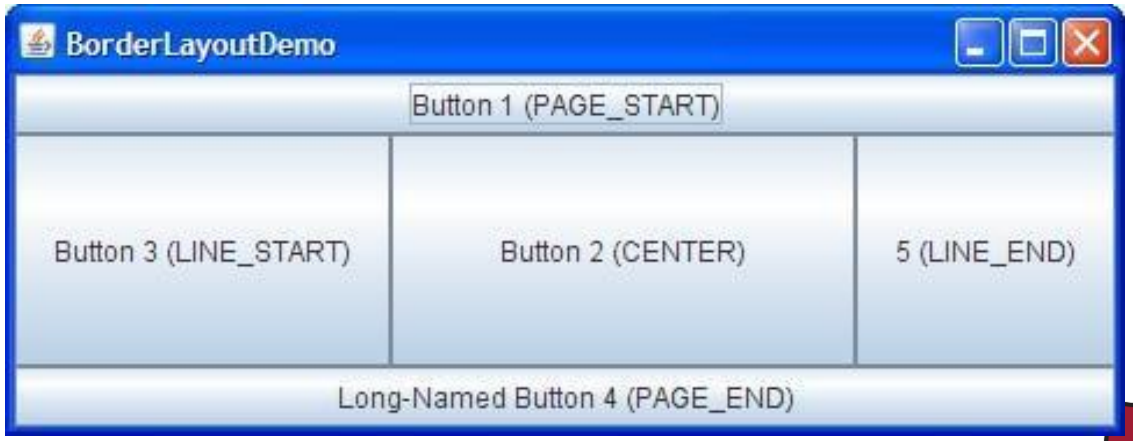
Java Layout Manager

- The Layout manager is used to layout (or arrange) the GUI Java components inside a container. There are many layout managers, but the most frequently used are-
- **Java BorderLayout**
- **Java FlowLayout**
- **Java GridBagLayout**

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java BorderLayout

- A **Java BorderLayout** BorderLayout is the default layout for the content panes of JWindow and JFrame objects. It places components in up to five areas: top, bottom, left, right, and center.
- Because each component is associated with a direction, BorderLayout can manage at most five components; it squashes or stretches those components to fit its constraints.



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java FlowLayout

- The **FlowLayout** class puts components in a row, sized at their preferred size.
- If the horizontal space in the container is too small to put all the components in one row, the FlowLayout class uses multiple rows.
- If the container is wider than necessary for a row of components, the row is, by default, centered horizontally within the container.

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Java GridBagLayout

- **GridBagLayout** class is one of the most flexible and complex layout managers the Java platform provides.
- It places components in a grid of rows and columns, allowing specified components to span multiple rows or columns.
- As you can see from figure, the grid has three rows and three columns. The button in the second row spans all the columns; the button in the third row spans the two right columns.

UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Lecture No: 39
Designing GUI and building applications using swing in Java

Swing Design Principles

- GUI is built as **containment hierarchy** of widgets (i.e. the parent-child nesting relation between them)
- Event objects and event listeners
  - **Event object**: is created when event occurs (e.g. click), contains additional info (e.g. mouse coordinates)
  - **Event listener**: object implementing an interface with an event handler method that gets an event object as argument
- Separation of Model and View:
  - **Model**: the data that is presented by a widget
  - **View**: the actual presentation on the screen

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Different Kinds of Swing Events

Low-level events

- **`MouseEvent`**: Component got mouse-down, mouse-move, etc.
- **`KeyEvent`**: Component got key-press, key-release, etc.
- **`ComponentEvent`**: Component resized, moved, etc.
- **`ContainerEvent`**: Container's contents changed because a component was added or removed
- **`FocusEvent`**: Component got focus or lost focus
- **`WindowEvent`**: Window opened, closed, etc.

High-level semantic events

- **`ActionEvent`**: Main action of control invoked (e.g. JButton click)
- **`AdjustmentEvent`**: Value was adjusted (e.g. JScrollBar moved)
- **`ItemEvent`**: Item was selected or deselected (e.g. in JList)
- **`TextEvent`**: Text in component has changed (e.g in JTextField)

D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

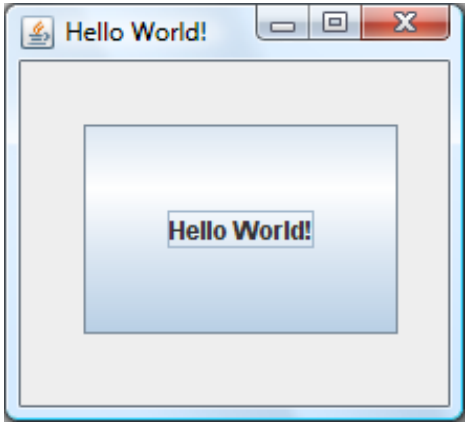
Events, Listeners / Adapters and Handler Methods

Event	Listener / Adapter	Handler Methods
ActionEvent	ActionListener	actionPerformed
AdjustmentEvent	AdjustmentListener	adjustmentValueChanged
MouseEvent	MouseListener MouseAdapter	mouseClicked mouseEntered mouseExited mousePressed mouseReleased
KeyEvent	KeyListener KeyAdapter	keyPressed keyReleased keyTyped
ComponentEvent	ComponentListener ComponentAdapter	componentShown componentHidden componentMoved componentResized

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Swing Example: Hello World

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class HelloWorld {
    public static void main(String[] args) {
          JFrame frame = new JFrame("Hello World!");
          frame.setSize(220, 200);
          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

          Container contentPane = frame.getContentPane();
          contentPane.setLayout(null);

          JButton button = new JButton("Hello World!");
          button.setLocation(30, 30);
          button.setSize(150, 100);
          contentPane.add(button);

          frame.setVisible(true);
    }
}
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Swing Example: Calculator

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class Calculator implements ActionListener {

private static JTextField inputBox;
Calculator(){}
public static void main(String[] args) {
createWindow();
}
private static void createWindow() {
    JFrame frame = new JFrame("Calculator");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    createUI(frame);
    frame.setSize(200, 200);
    frame.setLocationRelativeTo(null);
    frame.setVisible(true);
}
private static void createUI(JFrame frame) {
    JPanel panel = new JPanel();
    Calculator calculator = new Calculator();
    GridBagLayout layout = new GridBagLayout();
    GridBagConstraints gbc = new GridBagConstraints();
    panel.setLayout(layout);
    inputBox = new JTextField(10);
    inputBox.setEditable(false);
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Swing Example: Calculator

```
JButton button0 = new JButton("0");JButton button1 = new JButton("1");
JButton button2 = new JButton("2");JButton button3 = new JButton("3");
JButton button4 = new JButton("4");JButton button5 = new JButton("5");
JButton button6 = new JButton("6");JButton button7 = new JButton("7");
JButton button8 = new JButton("8");JButton button9 = new JButton("9");
JButton buttonPlus = new JButton("+");JButton buttonMinus = new JButton("-");
JButton buttonDivide = new JButton("/");JButton buttonMultiply = new JButton("x");
JButton buttonClear = new JButton("C");JButton buttonDot = new JButton(".");
JButton buttonEquals = new JButton("=");

button1.addActionListener(calculator);button2.addActionListener(calculator);
button3.addActionListener(calculator);button4.addActionListener(calculator);
button5.addActionListener(calculator);button6.addActionListener(calculator);
button7.addActionListener(calculator);button8.addActionListener(calculator);
button9.addActionListener(calculator);button0.addActionListener(calculator);
buttonPlus.addActionListener(calculator);buttonMinus.addActionListener(calculator);
buttonDivide.addActionListener(calculator);buttonMultiply.addActionListener(calculator);
buttonClear.addActionListener(calculator);buttonDot.addActionListener(calculator);
buttonEquals.addActionListener(calculator);
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Swing Example: Calculator

```
    gbc.fill = GridBagConstraints.HORIZONTAL;
    gbc.gridx = 0; gbc.gridy = 0; panel.add(button1, gbc);
    gbc.gridx = 1; gbc.gridy = 0; panel.add(button2, gbc);
    gbc.gridx = 2; gbc.gridy = 0; panel.add(button3, gbc);
    gbc.gridx = 3; gbc.gridy = 0; panel.add(buttonPlus, gbc);
    gbc.gridx = 0; gbc.gridy = 1; panel.add(button4, gbc);
    gbc.gridx = 1; gbc.gridy = 1; panel.add(button5, gbc);
    gbc.gridx = 2; gbc.gridy = 1; panel.add(button6, gbc);
    gbc.gridx = 3; gbc.gridy = 1; panel.add(buttonMinus, gbc);
    gbc.gridx = 0; gbc.gridy = 2; panel.add(button7, gbc);
    gbc.gridx = 1; gbc.gridy = 2; panel.add(button8, gbc);
    gbc.gridx = 2; gbc.gridy = 2; panel.add(button9, gbc);
    gbc.gridx = 3; gbc.gridy = 2; panel.add(buttonDivide, gbc);
    gbc.gridx = 0; gbc.gridy = 3; panel.add(buttonDot, gbc);
    gbc.gridx = 1; gbc.gridy = 3; panel.add(button0, gbc);
    gbc.gridx = 2; gbc.gridy = 3; panel.add(buttonClear, gbc);
    gbc.gridx = 3; gbc.gridy = 3; panel.add(buttonMultiply, gbc);
    gbc.gridwidth = 3;
    gbc.gridx = 0; gbc.gridy = 4; panel.add(inputBox, gbc);
    gbc.gridx = 3; gbc.gridy = 4; panel.add(buttonEquals, gbc);
    frame.getContentPane().add(panel, BorderLayout.CENTER);
}
```


D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

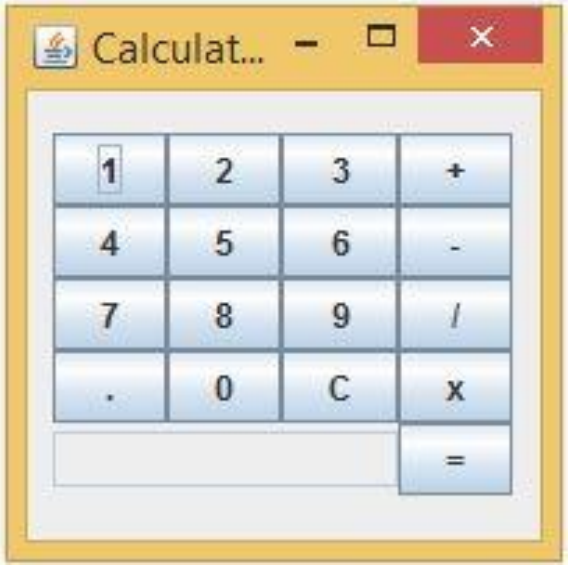
Swing Example: Calculator

```
public void actionPerformed(ActionEvent e) {
    String command = e.getActionCommand();
    if (command.charAt(0) == 'C') {
       inputBox.setText("");
    }else if (command.charAt(0) == '=') {
       inputBox.setText(evaluate(inputBox.getText()));
    }else {
       inputBox.setText(inputBox.getText() + command);
    }
 }

 public static String evaluate(String expression) {
    char[] arr = expression.toCharArray();
    String operand1 = "";String operand2 = "";String operator = "";
    double result = 0;
    for (int i = 0; i < arr.length; i++) {
       if (arr[i] >= '0' && arr[i] <= '9' || arr[i] == '.') {
          if(operator.isEmpty()){
             operand1 += arr[i];
          }else{
             operand2 += arr[i];
          }
       }
       if(arr[i] == '+' || arr[i] == '-' || arr[i] == '/' || arr[i] == '*') {
          operator += arr[i];
       }
    }
```


D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Swing Example: Calculator

```
        if (operator.equals("+"))
            result = (Double.parseDouble(operand1) + Double.parseDouble(operand2));
        else if (operator.equals("-"))
            result = (Double.parseDouble(operand1) - Double.parseDouble(operand2));
        else if (operator.equals("/"))
            result = (Double.parseDouble(operand1) / Double.parseDouble(operand2));
        else
            result = (Double.parseDouble(operand1) * Double.parseDouble(operand2));
        return operand1 + operator + operand2 + "=" +result;
    }
}
```



D Y PATIL DEEMED TO BE UNIVERSITY
RAMRAO ADIK INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Thank You